Mikros: Bitwidth-Aware SNARKs over Extensions of Small Fields

Mingshu Cong

Shenzhen University

Abstract. Provers for succinct non-interactive arguments of knowledge (SNARKs) perform arithmetic over a large field for soundness, even when the witness entries are a few bits wide, which inflates both prover time and memory usage. Recent work makes individual SNARK components *bitwidth-aware*, so that a lower witness bitwidth means faster proving and lower memory usage, but these techniques cover only part of the prover and exploit the structure of specific fields, typically binary ones.
We introduce `Mikros`, a framework for end-to-end bitwidth-aware SNARK construction based on *algebraic vector packing*. We pack vectors of length $N = m\kappa$ over a base field $\mathbb{F}$ into length-m vectors over a degree-κ extension field $\mathbb{E}$, which turns length-N relations over $\mathbb{F}$ into length-m relations over $\mathbb{E}$ and makes existing linear-time extension-field protocols bitwidth-aware. This technique thus unifies bitwidth-aware inner-product proofs, Hadamard-product proofs, and polynomial commitment schemes, previously obtained by separate constructions, and yields the first bitwidth-aware grand-product proof, hence the first bitwidth-aware permutation argument. Composing them gives bitwidth-aware SNARKs for circuit satisfiability over extensions of arbitrary small fields.
For a single instance of circuit satisfiability, the prover cost is proportional to the *effective bitwidth*, the combined bitwidth of the witness entries and the wiring-position encodings; across instances of the same circuit, the position encoding is amortized and the per-instance cost is proportional to the witness bitwidth. Although `Mikros` does not rely on binary-field structure, its prover is 2.19×, 2.55×, and 1.85× faster on batches of 256 SHA-256 compressions, Keccak permutations, and BLAKE3 compressions than Flock (Bünz *et al.*, IACR ePrint 2026), the state-of-the-art bitwidth-aware SNARK over binary-extension fields.

1 Introduction

A succinct non-interactive argument of knowledge (SNARK) allows a prover to establish knowledge of a witness using a short proof that can be verified much faster than the computation it certifies (*e.g.*, [13, 14, 16, 21–23, 32, 37, 38, 58, 59]). While the proof is succinct and the verifier efficient, the prover's cost of producing these SNARK proofs significantly exceeds the cost of the computation itself.

Working over a large field for soundness guarantees is a key contributor to prover overhead. The soundness of essentially all SNARK protocols relies, at some step, on the Schwartz–Zippel lemma: a nonzero polynomial evaluated at a

random point sampled from a large field is unlikely to vanish. For these tests, the error bound is proportional to the polynomial degree and inversely proportional to the challenge-field size. This large field is costly. The memory needed to store a field element is proportional to the field bitwidth, while the time to compute a field multiplication is quadratic in the bitwidth under schoolbook arithmetic.

Computations with low-bitwidth integers arise in modern applications of SNARKs. First, many blockchain applications of SNARKs require proving hash computations involving bitwise operations, such as Merkle-tree membership proofs for anonymous payments [8], hash-based commitments in privacy-preserving smart contracts [45], and light-client verification for cross-chain bridges [66]. Second, a growing body of zero-knowledge machine learning (zkML) research develops proofs for machine-learning computations and their arithmetic components (*e.g.*, [1,24,33,40,47,54–56,61]). For efficient inference, neural networks can quantize their weights and activations to 8-bit integers [43]. Third, a recent line of work combines SNARKs with fully homomorphic encryption (FHE) to achieve verifiable computation on encrypted data (*e.g.*, [3, 12, 19, 30, 31, 34, 52, 63, 69]). FHE schemes often exploit small-field arithmetic, with plaintext coordinates in fields such as $\mathbb{F}_{65537}$ [3] and ciphertext coefficients represented via the Chinese remainder theorem (CRT) over word-sized prime fields [42].

For computations over small fields, we seek prover time and memory usage that scale with the bitwidth of the base field, a property we call *bitwidth awareness*. Drawing challenges from a large extension field can erase these savings, since witness-dependent intermediate values are lifted into the large field. Recent work addresses this overhead in both polynomial commitment schemes (PCSs) (*e.g.*, [27, 28]) and polynomial interactive oracle proofs (PIOPs) (*e.g.*, [5, 6, 25, 26, 39]), and an end-to-end bitwidth-aware SNARK has been constructed over binary fields [17]. However, existing bitwidth-aware techniques are either limited to individual SNARK components or tied to specific field structures. Constructing an end-to-end bitwidth-aware SNARK over arbitrary small fields remains an open problem. In this paper, we remove these restrictions. Given a small field, an extension of it, and a linear-prover PCS over the extension field, we show how to construct a bitwidth-aware SNARK for circuit satisfiability (Circ-SAT).

Our result. Let λ be the security parameter, and consider an N-gate arithmetic circuit over a small field of order q. We choose a packing factor $\kappa \simeq \lambda / \log_2 q$; for fixed λ, κ scales inversely with the base-field bitwidth $\log_2 q$. For a single instance, our prover uses $O(N\kappa \log_q N \log_2 \kappa)$ base-field operations and stores $O(N \log_q N)$ base-field elements. For a batch of κ instances of the same circuit, the amortized costs decrease to $O(\kappa N)$ base-field operations and $O(N)$ base-field elements of memory per instance. Under schoolbook arithmetic, multiplication in a degree-κ extension costs $\Theta(\kappa^2)$ base-field operations, while each extension-field element occupies κ base-field coordinates. Thus, the amortized per-instance complexity corresponds to $O(N/\kappa)$ extension-field multiplication equivalents and the storage occupied by $O(N/\kappa)$ extension-field elements.

Our core technique. `Mikros` is built on a new technique we call *algebraic vector packing*. We partition a vector of length $N = m\kappa$ over $\mathbb{F}$ into κ blocks

of length m and pack these blocks into a single length-m vector using either (i) a basis $\overline{\beta}_1, \ldots, \overline{\beta}_\kappa$ of $\mathbb{E}$ over $\mathbb{F}$, or (ii) κ linearly independent polynomials $\phi_1(X), \ldots, \phi_\kappa(X) \in \mathbb{F}[X]$, each of degree at most $\kappa - 1$. In the first case, the original length-N vector relation over $\mathbb{F}$ translates directly into a length-m vector relation over $\mathbb{E}$. In the second case, it first becomes a polynomial-vector identity, whose evaluation at a random point reduces to a length-m vector relation over $\mathbb{E}$. Since proof systems for vector relations over extension fields are well developed, existing protocols can be applied directly to the packed relation.

Concrete performance. While our construction is generic and relies on no specific field structure beyond the existence of an extension field, it achieves lower prover time than Flock [17], the state-of-the-art bitwidth-aware SNARK over binary fields. For a binary Hadamard (element-wise) product of length 2^{30}, the single-threaded PIOP prover in `Mikros` runs in 684.48 ms, 1.45× faster than Flock. For batches of 256 independent SHA-256 compressions, Keccak permutations, and BLAKE3 compressions, `Mikros` reduces single-threaded end-to-end prover time by 2.19×, 2.55×, and 1.85×, respectively.

1.1 Related Work

PIOP–PCS framework for SNARKs. Since Supersonic [16] and Marlin [22], SNARK design has converged on a common framework that combines a polynomial interactive oracle proof (PIOP) with a polynomial commitment scheme (PCS). Circuit satisfiability is first arithmetized as constraints, such as rank-one constraint systems (R1CS) [35] or PLONK-style gates [21, 32], which are then encoded as polynomial identities. The PIOP uses random verifier challenges to reduce these identities to polynomial evaluation claims, and the PCS binds the prover to the committed polynomials and proves the claimed evaluations [44,50]. The Fiat–Shamir transform makes the resulting argument non-interactive in the random-oracle model [10, 29].

Early arithmetizations encoded constraints using univariate polynomials [32, 35, 51], whereas later works increasingly adopt multilinear extensions (MLEs) over the Boolean hypercube [21,58,64,67]. MLE-based identities are typically reduced to sumcheck claims. The sumcheck protocol, introduced by Lund *et al.* [49], has since become a fundamental building block of efficient proof systems [36,62]. Building on sumcheck, HyperPlonk encodes circuit wiring through a permutation argument [21], whereas Spartan [58] proves the evaluation of sparse R1CS matrices via memory checking, later generalized into lookup arguments [60].

Within this PIOP–PCS framework, a SNARK for Circ-SAT can be assembled from four protocol components: (i) a degree-1 zerocheck, which proves the inner product between a private vector and a tensor-structured vector; (ii) a degree-2 zerocheck, which proves a Hadamard-product relation; (iii) a lookup or permutation argument, which proves the wiring relation; and (iv) a PCS, which proves the MLE claims produced by the reductions. Prior work on small-field SNARKs makes a subset of these components bitwidth-aware, as summarized in Table 1. As the table shows, no prior framework is fully bitwidth-aware across all four components.

Table 1. Bitwidth awareness of SNARK components in prior work and `Mikros`.

	Degree-1 zerocheck	Degree-2 zerocheck	Lookup argument	PCS
Group-based PCSs [44, 46, 64]	○	○	○	◑
Lasso/BabySpartan [57, 60]	○	○	◑	◑
Binius/FRI-Binius [27, 28]	●	○	◑	●
SFF [5, 6, 25]	●	◑	◑	○
Dao–Thaler [26]	●	◑	◑	○
Tower zerocheck [48]	●	◑	○	●
RMFE–Aurora [20]	●	●	○	●
Gruen [39]	●	●	○	○
Packed sumcheck [65]	●	●	◑	●
Flock [17]	●	●	○	●
`Mikros`	●	●	●	●

●: fully bitwidth-aware; ◑: partially bitwidth-aware (some prover computations are not bitwidth-aware); ○: no dedicated bitwidth-aware construction.

Bitwidth-aware PCSs. Group-based PCSs such as KZG, Hyrax, and Dory [44, 46, 64] commit using multiscalar multiplications, where short scalars reduce the commitment cost [60]. However, the scalar field remains large, and the opening algorithm, which depends on scalar-field random challenges, remains full-width.

Hash-based PCSs [7, 11, 37, 68] encode a vector with an error-correcting code such that any two distinct vectors yield codewords that differ in many positions. Some of these constructions, such as Brakedown and BaseFold [37, 68], are field-agnostic, requiring no special field structure, yet they still need a sufficiently large field for soundness. Other systems operate over small, efficient fields, such as the 64-bit Goldilocks field [53] and the 31-bit Mersenne field [41], but draw random challenges from an extension field to reach the required soundness level. In either case, the prover computes over a large field regardless of the witness bitwidth, so hash-based PCSs were long not bitwidth-aware.

This problem was resolved recently for binary fields. Binius commits to multilinear polynomials over binary towers, including $\mathbb{F}_2$, without embedding each coefficient into the challenge field [27]. FRI-Binius further introduces *ring switching* [28], which packs κ base-field coordinates into one element of a degree-κ extension $\mathbb{E}$, so that the commitment cost depends only on the total bit size of the input. A sumcheck-based reduction then turns MLE claims over $\mathbb{F}$ into claims about the packed polynomial over $\mathbb{E}$, keeping the opening proof succinct as well.

Bitwidth-aware PIOPs. The PIOP reduces constraint relations to evaluation claims about the MLEs of the witness and the wiring encoding. Gate constraints are proved by degree-1 and degree-2 zerochecks, while wiring constraints are proved with a lookup or permutation argument.

Table 2. Prover costs for N-Hadamard-product proofs and N-of-M lookup arguments

	Prover time	Prover RAM	Applicability
Hadamard-product proofs (degree-2 zerocheck)			
Classical sumcheck	$O(N\kappa^2)$	$O(N\kappa)$	General extensions
SFF [5, 6, 25]	$O(N\kappa(3/2)^{t_{\mathsf{skip}}} + N\kappa^2/2^{t_{\mathsf{skip}}})$	$O(N+N\kappa/2^{t_{\mathsf{skip}}})$	General extensions
Dao–Thaler [26]	$O(N\kappa^2)$	$O(N\kappa)$	Binary towers
Tower zerocheck [48]	$O(N\kappa^{3/2})$	$O(N\sqrt{\kappa})$	Wiedemann towers
RMFE–Aurora [20]	$O(N\kappa^2 + N\log N\kappa)$	$O(N\kappa)$	Binary RMFE
Packed sumcheck [65]	$O(Nk)$	$O(N)$	$\lvert\mathbb{F}\rvert > 3(2k-1)+1$
Gruen [39]	$O(Nt_{\mathsf{skip}} + N\kappa + N\kappa^2/2^{t_{\mathsf{skip}}})$	$O(N+N\kappa/2^{t_{\mathsf{skip}}})$	$\lvert\mathbb{F}\rvert \geq 2\kappa - 1$
Flock [17]	$O(N2^{t_{\mathsf{skip}}} + N\kappa + N\kappa^2/2^{t_{\mathsf{skip}}})$	$O(N+N\kappa/2^{t_{\mathsf{skip}}})$	Binary extensions
`Mikros`	$O(N\kappa)$	$O(N)$	General extensions
Lookup argument			
Classical Lasso [60]	$O((M+N)\kappa^2)$	$O((M+N)\kappa)$	General extensions
Binius [27]	$O((M+N)\kappa^2)$	$O((M+N)\kappa)$	General extensions
Packed sumcheck [65]	$O((M+N)\kappa^2)$	$O((M+N)\kappa)$	$\lvert\mathbb{F}\rvert > M+N$
`Mikros`	$O((M+N)d\kappa\log_2\kappa')$	$O((M+N)d)$	General extensions

Here $\kappa \approx \lambda/\log_2|\mathbb{F}|$ is the packing parameter for all schemes except packed sumcheck, whose parameter $k > \kappa$ depends on N. t_{skip} denotes the number of skipped rounds. For lookups, M is the table size and N the number of lookup values. The intermediate-field degree is $d = \Theta(1 + \log_{|\mathbb{F}|}\max\{M, N\})$, and the effective packing parameter is $\kappa' = \kappa/(2d)$. Time and RAM are measured in operations and elements over the base field, respectively. Assume $N \gg \kappa$ and omit lower-order terms of N; Appendix B.4 derives the Hadamard bounds with their parameter-only terms and absorption conditions, and Appendix B.5 derives the lookup bounds.

Zerocheck. A zerocheck proves that a low-degree multivariate polynomial vanishes at every point of the Boolean hypercube, and it is typically reduced to a sumcheck. Constraint systems such as R1CS and PLONK require both degree-1 and degree-2 zerochecks, which prove inner-product and Hadamard-product relations among the gate values, respectively. With classical sumcheck, a zerocheck over N gates costs $O(N)$ extension-field operations, even when the witness lies in a small field. To reduce this overhead, small-field-first (SFF) methods [5, 6, 25] precompute products of base-field entries and expand the extension-field operations of the first t_{skip} sumcheck rounds into base-field operations. This makes degree-1 zerochecks bitwidth-aware. For degree-2 zerochecks, however, the precomputation cost grows exponentially in t_{skip}, so only a few rounds can be optimized, and the remaining rounds still operate over the extension field. The univariate skip of Gruen [39] improves on this by handling several initial rounds

at once with a single higher-degree univariate check, which makes the prover time proportional to the bitwidth. Another line of work exploits the structure of binary fields, using either tower structures [26, 48] or reverse multiplication-friendly embeddings (RMFEs) [20]. Flock [17] builds on the approach of Gruen and further exploits the arithmetic of binary fields. Packed sumcheck [65] takes a different approach, avoiding extension fields entirely through parallel repetition over the small field. However, to meet the soundness requirement, its repetition count k must grow with N and can significantly exceed κ, eroding its efficiency gains.

Lookup arguments. Lookup arguments prove the wiring encoding, ensuring that the witness is consistent with the wiring of the committed circuit. Lasso [60] builds a lookup argument on top of a permutation argument, reducing a lookup to a multiset equality between the low-bitwidth lookup values and the table. To work over small fields, Binius and related systems [26, 27] first pack several base-field coordinates, including the position indices and the values, into a single extension-field element, and then run the permutation argument on the packed elements. However, each packed element is treated as a full-bitwidth extension-field element. The low bitwidth of its coordinates is ignored.

Table 2 compares the prover costs of Hadamard-product proofs and lookup arguments. Existing techniques either incur a coefficient of N that grows superlinearly in κ or apply only to restricted fields. In contrast, `Mikros` achieves $O(\kappa N)$ prover time and $O(N)$ memory for Hadamard-product proofs over general extensions, and extends the same packing technique to lookup and permutation arguments, whose prover cost then scales with the effective bitwidth.

Relation to our work. Prior work obtains bitwidth-aware inner-product proofs, Hadamard-product proofs, and PCSs through separate techniques. `Mikros` unifies all three components under a single algebraic vector packing technique with a uniform interface. Beyond these components, the same technique yields bitwidth-aware permutation and lookup arguments, and it packs multiple Circ-SAT instances of the same circuit. Together, these components give an end-to-end bitwidth-aware SNARK over extensions of arbitrary small fields.

1.2 Our Contributions

New technique: algebraic vector packing. We pack length-N base-field vectors into length-N/κ extension-field vectors. Packing is decoupled from the extension-field protocol that proves the packed relation, so the two can be optimized independently, and existing extension-field protocols can become bitwidth-aware for a broad range of relations.

New solution: end-to-end bitwidth-aware SNARKs over arbitrary small fields. We make every SNARK component bitwidth-aware, including inner-product and Hadamard-product proofs, permutation and lookup arguments, and the PCS. Composing these components yields end-to-end bitwidth-aware SNARKs for circuit satisfiability, the first that apply to arbitrary small fields.

New components of independent interest. Our packed inner-product, Hadamard-product, permutation, and lookup arguments apply beyond circuit satisfiability. For example, our bitwidth-aware lookup argument can prove nonlinear activation functions on low-bitwidth inputs in zkML protocols.

2 Technical Overview

Let $\mathbb{E}$ be a degree-κ extension of a small field $\mathbb{F}$, large enough to meet the soundness requirements at security parameter λ. Our central technique, *algebraic vector packing*, reduces length-$N = m\kappa$ base-field relations to length-m extension-field relations. The challenge is to preserve the base-field relation. We use fixed extension-field bases for linear relations and polynomial bases for bilinear relations. The two routes share a packed commitment interface, so their proof protocols compose into an end-to-end bitwidth-aware SNARK for Circ-SAT.

2.1 Algebraic Packing of Small-Field Vectors via Two Routes

We partition $\boldsymbol{a}, \boldsymbol{b} \in \mathbb{F}^N$ into κ blocks of length m, and arrange them as columns:

$$\boldsymbol{A} = \left[\boldsymbol{a}^{(1)} \mid \ldots \mid \boldsymbol{a}^{(\kappa)} \right] \in \mathbb{F}^{m \times \kappa}, \qquad \boldsymbol{B} = \left[\boldsymbol{b}^{(1)} \mid \ldots \mid \boldsymbol{b}^{(\kappa)} \right] \in \mathbb{F}^{m \times \kappa}.$$

There are two methods to combine these columns into length-m vectors.

Extension-field packing for linear relations. Fix an $\mathbb{F}$-basis $\overline{\boldsymbol{\beta}} = (\overline{\beta}_1, \ldots, \overline{\beta}_\kappa)^\top$ of $\mathbb{E}$ and set $\overline{\boldsymbol{a}} = \boldsymbol{A}\overline{\boldsymbol{\beta}}$. Any $\mathbb{F}$-linear map $\mathcal{L} : \mathbb{F}^m \to \mathbb{F}^{m'}$, applied across the columns, can be represented by a matrix $\boldsymbol{L} \in \mathbb{F}^{m' \times m}$. For $\boldsymbol{c}^{(j)} \in \mathbb{F}^{m'}$, linear independence of the basis gives:

$$\mathcal{L}(\boldsymbol{a}^{(j)}) = \boldsymbol{c}^{(j)} \quad \forall j \in [\kappa] \quad \iff \quad \boldsymbol{L}\overline{\boldsymbol{a}} = \textstyle\sum_{j=1}^{\kappa} \boldsymbol{c}^{(j)} \overline{\beta}_j.$$

Thus one extension-field vector equation certifies all κ base-field equations. Here, the requirement that $\boldsymbol{L}$ only contain base-field entries is essential.

Polynomial packing for bilinear relations. For a bilinear map $\mathcal{B}$, applying $\mathcal{B}(\boldsymbol{A}\overline{\boldsymbol{\beta}}, \boldsymbol{B}\overline{\boldsymbol{\beta}})$ introduces cross-column products weighted by $\overline{\beta}_i \overline{\beta}_j$. These do not, in general, encode the desired element-wise relations in the original basis.

We instead retain a formal indeterminate X. Choose linearly independent polynomials $\phi_1, \ldots, \phi_\kappa \in \mathbb{F}[X]$, tailored to the relation, and write $\boldsymbol{\phi} = (\phi_1, \ldots, \phi_\kappa)^\top$. The vector packings are then:

$$\boldsymbol{g}_A(X) = \boldsymbol{A}\boldsymbol{\phi}(X) \in \mathbb{F}[X]^m, \qquad \boldsymbol{g}_B(X) = \boldsymbol{B}\boldsymbol{\phi}(X) \in \mathbb{F}[X]^m. \tag{1}$$

Unlike multiplication in a fixed extension-field basis, polynomial multiplication does not reduce modulo an irreducible polynomial. We can therefore isolate the desired relation in a single coefficient, or in the values at a set of interpolation points, while auxiliary polynomials absorb the remaining terms.

Concretely, we construct a polynomial identity $\mathcal{F}$ such that

$$\mathcal{B}(\boldsymbol{A}\overline{\boldsymbol{\beta}}, \boldsymbol{B}\overline{\boldsymbol{\beta}}) = \boldsymbol{c} \iff \mathcal{F}\big(\boldsymbol{g}_A(X), \boldsymbol{g}_B(X), \boldsymbol{c}, \boldsymbol{h}(X)\big) = 0,$$

where $\mathcal{F}$ also depends on the claimed output $\boldsymbol{c}$ and on some auxiliary polynomials $\boldsymbol{h}(X)$. After the witness commitments are fixed, the prover sends the auxiliary polynomials, and then the verifier samples a challenge $\overline{x} \in \mathbb{E}$. Evaluating the identity at $\overline{x}$ reduces it to a bilinear relation on length-m vectors over $\mathbb{E}$, which we prove using an extension-field protocol.

2.2 Instantiations for Concrete Relations

Packed inner-product proof. For $c = \langle \boldsymbol{a}, \boldsymbol{b} \rangle = \sum_{j=1}^{\kappa} \langle \boldsymbol{a}^{(j)}, \boldsymbol{b}^{(j)} \rangle$, choose:

$$\boldsymbol{\phi}_A(X) = (1, X, \ldots, X^{\kappa-1})^\top, \qquad \boldsymbol{\phi}_B(X) = (1, X^{-1}, \ldots, X^{-(\kappa-1)})^\top.$$

In $\langle \boldsymbol{g}_A(X), \boldsymbol{g}_B(X) \rangle$, the matching columns contribute to the constant coefficient, while cross-column products have nonzero exponents. The claim therefore holds if and only if there exist $h_+, h_- \in \mathbb{F}[X]$, each of degree at most $\kappa - 2$, such that:

$$\langle \boldsymbol{g}_A(X), \boldsymbol{g}_B(X) \rangle = c + X h_+(X) + X^{-1} h_-(X^{-1}).$$

Neither auxiliary term can alter the constant coefficient. After h_+, h_- are fixed, a random $\overline{x} \xleftarrow{\$} \mathbb{E}^\times$ reduces this identity to one length-m inner-product claim.

Packed Hadamard-product proof. For $\boldsymbol{a}^{(j)} \circ \boldsymbol{b}^{(j)} = \boldsymbol{c}^{(j)}$, a random tensor row weight $\bar{\boldsymbol{l}}$ reduces each vector equality to a scalar claim. Writing $\overline{d}^{(j)} = \langle \bar{\boldsymbol{l}}, \boldsymbol{c}^{(j)} \rangle$, we must prove:

$$\overline{d}^{(j)} = \langle \bar{\boldsymbol{l}}, \boldsymbol{a}^{(j)} \circ \boldsymbol{b}^{(j)} \rangle, \qquad j \in [\kappa].$$

For $|\mathbb{F}| \geq \kappa$, choose distinct roots $\mathcal{H} = \{\gamma_1, \ldots, \gamma_\kappa\} \subseteq \mathbb{F}$ and their Lagrange basis $(\psi_1, \ldots, \psi_\kappa)$. Use $\boldsymbol{\phi} = (\psi_1, \ldots, \psi_\kappa)^\top$ in Eq. (1), and set $g_C(X) = \sum_{j=1}^{\kappa} \overline{d}^{(j)} \psi_j(X)$ and $\Psi(X) = \prod_{j=1}^{\kappa} (X - \gamma_j)$. Since evaluation at γ_j recovers column j, all scalar claims hold exactly when:

$$\bar{\boldsymbol{l}}^\top \big(\boldsymbol{g}_A(X) \circ \boldsymbol{g}_B(X)\big) - g_C(X) = \Psi(X) h(X), \qquad \deg h \leq \kappa - 2.$$

Here $h \in \mathbb{E}[X]$. The factor Ψ prevents the quotient from changing any claimed value at the roots. Fixing g_C, h before sampling $\overline{x} \xleftarrow{\$} \mathbb{E} \setminus \mathcal{H}$ leaves a length-m weighted inner product over $\mathbb{E}$.

In both reductions, the packed polynomial products in each row are computed over $\mathbb{F}$ before aggregation with extension-field weights. With precomputed base-polynomial evaluations, the prover uses $O(N\kappa + \kappa^3)$ base-field operations. For $m \geq \kappa$, this is $O(N\kappa)$: the final length-m proof costs $O(m)$ extension-field operations, or $O(N\kappa)$ base-field operations under schoolbook arithmetic. Section 4.2 also gives a tabulated intermediate-field variant for smaller fields.

Packed PCS. κ base-field vectors are committed to via a packed $\overline{\boldsymbol{a}} = \boldsymbol{A}\overline{\boldsymbol{\beta}}$. However, the inner-product and Hadamard-product reductions use challenge-dependent packing weights $\overline{\boldsymbol{\rho}} = (\phi_1(\overline{x}), \ldots, \phi_\kappa(\overline{x})) \in \mathbb{E}^\kappa$. To connect these different packing weights, consider the column MLE claims $\overline{\boldsymbol{\xi}}^\top \boldsymbol{A} = \overline{\boldsymbol{v}}^\top$, where $\overline{\boldsymbol{\xi}} = \mathsf{ts}(\overline{\boldsymbol{x}})$ and $\overline{\boldsymbol{x}} \in \mathbb{E}^{\log_2 m}$. We decompose $\overline{\boldsymbol{\xi}} = \boldsymbol{\Xi}\overline{\boldsymbol{\beta}}$ and $\overline{\boldsymbol{v}}^\top = \overline{\boldsymbol{\beta}}^\top \boldsymbol{U}$, with $\boldsymbol{\Xi} \in \mathbb{F}^{m\times\kappa}$ and $\boldsymbol{U} \in \mathbb{F}^{\kappa\times\kappa}$. The extension-field claims thus become a base-field matrix identity, to which the linear packing route applies:

$$\overline{\boldsymbol{\xi}}^\top \boldsymbol{A} = \overline{\boldsymbol{v}}^\top \quad\Longleftrightarrow\quad \boldsymbol{\Xi}^\top \boldsymbol{A} = \boldsymbol{U} \quad\Longleftrightarrow\quad \boldsymbol{\Xi}^\top \overline{\boldsymbol{a}} = \boldsymbol{U}\overline{\boldsymbol{\beta}}.$$

After $\overline{\boldsymbol{v}}$ is fixed, a random projection reduces the last identity to a length-m inner product and one extension-field PCS opening. The verifier computes the MLE value of $\boldsymbol{\Xi}$ in logarithmic time using the tensor structure of $\overline{\boldsymbol{\xi}}$, through a recursion on its $\kappa \times \kappa$ coordinate matrices.

Packed permutation proof. A permutation relation is reduced to equality of random grand-product fingerprints:

$$\textstyle\prod_{i=1}^N (\overline{z} + a_i) = \prod_{i=1}^N (\overline{z} + b_i), \qquad \overline{z} \xleftarrow{\$} \mathbb{E}.$$

For powers-of-two m, κ with $m \geq \kappa$, we retain Z as a variable during the first $\log_2 \kappa$ levels of the product tree. Each level multiplies pairs of rows, so its coefficients remain in $\mathbb{F}$. At that depth, only m polynomial entries remain. Evaluating them at $\overline{z}$ gives a length-m extension-field tree.

The proof traverses these levels in reverse, using the packed Hadamard construction for each transition. At each interpolation point, the prover forms the row-product polynomials over the small field, aggregates their coefficients with the row weights, and only then evaluates at $\overline{z}$.

Packed constraint system. These components support both single-instance Circ-SAT arguments and batched arguments for multiple executions of the same circuit. For a single execution, we prove a PLONK-style constraint system by combining our packed inner-product, Hadamard-product, and permutation proofs with a packed PCS.

For a batch of κ instances, we use R1CS with shared wiring-encoding sparse matrices $\boldsymbol{S}_L, \boldsymbol{S}_R, \boldsymbol{S}_O \in \mathbb{F}^{N\times N}$, each with $O(N)$ nonzero entries. Collect the assignments as columns of $\boldsymbol{W} = \left[\, \boldsymbol{w}^{(1)} \mid \ldots \mid \boldsymbol{w}^{(\kappa)} \,\right] \in \mathbb{F}^{N\times\kappa}$ and commit to $\overline{\boldsymbol{w}} = \boldsymbol{W}\overline{\boldsymbol{\beta}} \in \mathbb{E}^N$. All instances satisfy the R1CS if and only if:

$$(\boldsymbol{S}_L \boldsymbol{w}^{(j)}) \circ (\boldsymbol{S}_R \boldsymbol{w}^{(j)}) = \boldsymbol{S}_O \boldsymbol{w}^{(j)},\ \forall j \in [\kappa] \quad\Longleftrightarrow\quad (\boldsymbol{S}_L \boldsymbol{W}) \circ (\boldsymbol{S}_R \boldsymbol{W}) = \boldsymbol{S}_O \boldsymbol{W}.$$

The packed Hadamard proof reduces this matrix relation to weighted evaluations of the three linear images $\boldsymbol{S}_\star \boldsymbol{W}$, for $\star \in \{L, R, O\}$. Since the matrices have base-field entries, fixed-basis packing commutes with each linear map:

$$(\boldsymbol{S}_\star \boldsymbol{W})\overline{\boldsymbol{\beta}} = \boldsymbol{S}_\star \overline{\boldsymbol{w}}.$$

The column-projection step of the packed PCS reduces these linear claims to an MLE opening of $\overline{\boldsymbol{w}}$ and MLE claims on the shared sparse matrices. The latter can

be proved using the sparse-evaluation protocol of Spartan [58], so their prover cost is amortized over all κ instances. Under the parameters of Section 5.3, with circuit preprocessing and linear-time, linear-space PCS provers, the amortized prover costs are $O(N\kappa)$ base-field operations and $O(N)$ base-field elements of memory per instance.

3 Preliminaries

3.1 Notation

Let $\mathbb{F}$ be a small finite field of order q, and let $\mathbb{E} = \mathsf{GF}(q^\kappa)$ be a degree-κ extension of $\mathbb{F}$. Thus $q = |\mathbb{F}|$ is a prime power and $\mathbb{F} \subset \mathbb{E}$. For security parameter λ, we choose κ to satisfy the soundness bounds of our protocols, including $|\mathbb{E}| = q^\kappa \geq 2^\lambda$. We write N for the witness length over $\mathbb{F}$ and set $m = N/\kappa$, assuming $\kappa \mid N$.

We use lowercase letters for scalars, bold lowercase letters for column vectors, and bold uppercase letters for matrices. An overline denotes an element, vector, or matrix over $\mathbb{E}$, as in $\overline{a}$, $\overline{\boldsymbol{a}}$, and $\overline{\boldsymbol{A}}$. Transposition is denoted by $\top$.

Let $\overline{\beta}_1, \ldots, \overline{\beta}_\kappa$ be a basis of $\mathbb{E}$ over $\mathbb{F}$. The polynomial rings over $\mathbb{F}$ and $\mathbb{E}$ are denoted by $\mathbb{F}[X]$ and $\mathbb{E}[X]$, respectively. In cost expressions, we overload $\mathbb{F}$ and $\mathbb{E}$ to denote the size of a base-field and an extension-field element, respectively. We use X, Y, Z for formal indeterminates, τ for tree levels, and i, j, ℓ for indices. Among the extension-field challenges we distinguish *evaluation points* from *weight vectors*. The points $\overline{\boldsymbol{x}}, \overline{\boldsymbol{y}}, \overline{\boldsymbol{y}}'$ have length $\log_2 n$ and enter the protocols only through their tensor span (Definition 3), whereas the scalar $\overline{z}$ is a fingerprinting or claim-combining challenge. Weight vectors carry no such structure: $\overline{\boldsymbol{l}}$ denotes a row weight, supplied as a product of two-entry factors, and $\overline{\boldsymbol{r}}, \overline{\boldsymbol{\rho}}$ denote column weights and random projections, which are arbitrary vectors and are never tensor spans. Two intermediate fields appear between $\mathbb{F}$ and $\mathbb{E}$: the interpolation field $\mathbb{K}$, of degree $t = [\mathbb{K} : \mathbb{F}]$, used by the Hadamard and permutation proofs over small base fields; and the position-encoding field $\mathbb{K}_{\mathsf{pos}}$, of degree $d = [\mathbb{K}_{\mathsf{pos}} : \mathbb{F}]$, used by the copy and lookup arguments, with $\kappa_{\mathsf{pos}} = [\mathbb{E} : \mathbb{K}_{\mathsf{pos}}]$. In comparisons with round-skipping methods, t_{skip} denotes the number of skipped sumcheck rounds. Soundness errors are written ϵ, with ϵ_{pc} for the underlying extension-field PCS and ϵ_{PCS} for the packed PCS built from it. In Appendix B.2, ℓ abbreviates $\log_2 m$. We write $[n] = \{1, \ldots, n\}$ and $[a, b] = \{a, a+1, \ldots, b\}$ for integers $a \leq b$, and use $|\cdot|$ for cardinality.

We also use the following matrix and vector operations:

- inner product $c = \langle \boldsymbol{a}, \boldsymbol{b} \rangle$; addition ($\boldsymbol{C} = \boldsymbol{A} + \boldsymbol{B}$); transposition ($\boldsymbol{C} = \boldsymbol{A}^\top$);
- multiplication ($\boldsymbol{C} = \boldsymbol{A}\boldsymbol{B}$); Kronecker (tensor) product ($\boldsymbol{C} = \boldsymbol{A} \otimes \boldsymbol{B}$);
- Hadamard (element-wise) product $\boldsymbol{A} \circ \boldsymbol{B}$;
- concatenation of vectors $\boldsymbol{c} = (\boldsymbol{c}_1 \| \boldsymbol{c}_2 \| \cdots \| \boldsymbol{c}_\kappa)$; and
- horizontal packing of column vectors $\boldsymbol{C} = [\boldsymbol{c}_1 \mid \boldsymbol{c}_2 \mid \cdots \mid \boldsymbol{c}_\kappa]$.

We denote the prover and verifier by $\mathcal{P}$ and $\mathcal{V}$. We write $\mathcal{TP}$, $\mathcal{SP}$, $\mathcal{TV}$, and $\mathcal{S}$ for prover time, prover memory, verifier time, and proof size, respectively. We

charge bb, be, and ee for a base-field, mixed-field, and extension-field multiplication, respectively, and absorb additions into these costs. Schoolbook extension-field arithmetic gives $\mathsf{be} = \kappa\mathsf{bb}$ and $\mathsf{ee} = \kappa\mathsf{be} = \kappa^2\mathsf{bb}$. We specify relations as $\mathcal{R} = \{\text{public input} : \text{private input} \mid \text{constraint}\}$.

3.2 SNARK Protocols in Vector Form

We distinguish between proofs and arguments of knowledge. A proof protocol satisfies completeness and soundness, whereas an argument of knowledge satisfies completeness and knowledge soundness (Definitions 9, 10, and 12 in Appendix A). Combining proof protocols with a knowledge-sound PCS and applying the Fiat–Shamir transform yields a non-interactive argument of knowledge.

We express circuit satisfiability as vector identities using the PLONK and R1CS constraint systems.

Definition 1 (PLONK constraint system for Circ-SAT [2,32]). *An arithmetic circuit with N gates and n_{io} public inputs and outputs (I/O) is specified by five selector vectors $\boldsymbol{q}_L, \boldsymbol{q}_R, \boldsymbol{q}_M, \boldsymbol{q}_O, \boldsymbol{q}_C \in \mathbb{F}^N$ and an augmented wiring permutation $\sigma : [3N + n_{\mathsf{io}}] \to [3N + n_{\mathsf{io}}]$, encoding wire-copy constraints and connections to the public inputs and outputs. Given a public-I/O vector $\mathbf{io} \in \mathbb{F}^{n_{\mathsf{io}}}$, let $\boldsymbol{w}_L, \boldsymbol{w}_R, \boldsymbol{w}_O \in \mathbb{F}^N$ denote the left, right, and output wire assignments, respectively, and define the augmented assignment $\boldsymbol{w} \coloneqq (\boldsymbol{w}_L \,\|\, \boldsymbol{w}_R \,\|\, \boldsymbol{w}_O \,\|\, \mathbf{io}) \in \mathbb{F}^{3N+n_{\mathsf{io}}}$. The wire assignment satisfies the circuit when:*

$$\boldsymbol{q}_L \circ \boldsymbol{w}_L + \boldsymbol{q}_R \circ \boldsymbol{w}_R + \boldsymbol{q}_M \circ (\boldsymbol{w}_L \circ \boldsymbol{w}_R) + \boldsymbol{q}_O \circ \boldsymbol{w}_O + \boldsymbol{q}_C = \mathbf{0}_N, \tag{2}$$

$$\sigma(\boldsymbol{w}) = \boldsymbol{w}. \tag{3}$$

Here, $\sigma(\boldsymbol{w})$ denotes the coordinate permutation of $\boldsymbol{w}$ induced by σ.

Definition 2 (R1CS constraint system for Circ-SAT [9,35]). *After padding, an R1CS with N constraints is specified by three sparse matrices $\boldsymbol{S}_L,\ \boldsymbol{S}_R,\ \boldsymbol{S}_O \in \mathbb{F}^{N\times N}$. Let $\boldsymbol{w} = (\boldsymbol{w}^{\mathsf{mul}} \,\|\, \mathbf{io}) \in \mathbb{F}^N$ be the assignment, where $\boldsymbol{w}^{\mathsf{mul}}$ collects multiplication gate values. The circuit is satisfied if and only if:*

$$(\boldsymbol{S}_L\boldsymbol{w}) \circ (\boldsymbol{S}_R\boldsymbol{w}) = \boldsymbol{S}_O\boldsymbol{w}.$$

The following tensor representation identifies an MLE evaluation with a linear functional on its evaluation vector.

Definition 3 (Tensor span). *Let $n \in 2^{\mathbb{N}}$ and let $\boldsymbol{x} = (x_1, \ldots, x_{\log_2 n}) \in \mathbb{F}^{\log_2 n}$. The* tensor span *of $\boldsymbol{x}$ is the length-n vector $\mathsf{ts}(\boldsymbol{x}) \coloneqq \bigotimes_{i=1}^{\log_2 n}(1 - x_i, x_i)^\top \in \mathbb{F}^n$. The same construction applies to $\overline{\boldsymbol{x}} \in \mathbb{E}^{\log_2 n}$. We define the sets of tensor spans over $\mathbb{F}$ and $\mathbb{E}$, respectively, by:*

$$\mathbb{T}(n) \coloneqq \left\{\mathsf{ts}(\boldsymbol{x}) \;\middle|\; \boldsymbol{x} \in \mathbb{F}^{\log_2 n}\right\} \subseteq \mathbb{F}^n, \qquad \overline{\mathbb{T}}(n) \coloneqq \left\{\mathsf{ts}(\overline{\boldsymbol{x}}) \;\middle|\; \overline{\boldsymbol{x}} \in \mathbb{E}^{\log_2 n}\right\} \subseteq \mathbb{E}^n.$$

Products of tensor weights. We also allow tensor weight vectors with arbitrary two-entry factors. Coordinatewise products preserve this representation:

$$\mathsf{ts}(\overline{\boldsymbol{x}}) \circ \mathsf{ts}(\overline{\boldsymbol{y}}) = \bigotimes_{j=1}^{\log_2 n} \left((1-\overline{x}_j)(1-\overline{y}_j), \overline{x}_j\overline{y}_j\right)^\top. \tag{4}$$

When $(1-\overline{x}_j)(1-\overline{y}_j) \neq 0$ for every j, factoring out these entries gives:

$$\mathsf{ts}(\overline{\boldsymbol{x}}) \circ \mathsf{ts}(\overline{\boldsymbol{y}}) = \left(\prod_j (1-\overline{x}_j)(1-\overline{y}_j)\right) \bigotimes_j \begin{pmatrix} 1 \\ \frac{\overline{x}_j}{1-\overline{x}_j} \\ \frac{\overline{y}_j}{1-\overline{y}_j} \end{pmatrix}.$$

To express the result as a scaled MLE query, set $\overline{d}_j = (1-\overline{x}_j)(1-\overline{y}_j) + \overline{x}_j\overline{y}_j$. If every $\overline{d}_j \neq 0$, then:

$$\mathsf{ts}(\overline{\boldsymbol{x}}) \circ \mathsf{ts}(\overline{\boldsymbol{y}}) = \left(\prod_j \overline{d}_j\right) \mathsf{ts}((\overline{x}_j\overline{y}_j/\overline{d}_j)_j).$$

Our protocols retain the two-entry factors in Eq. (4), which also covers zero entries and vanishing normalization factors without division.

Lemma 1 (Schwartz-Zippel-DeMillo-Lipton Lemma). *Let $\phi(\boldsymbol{x}) : \mathbb{F}^n \to \mathbb{F}$ be a nonzero n-variate polynomial such that the degree of ϕ in each variable is at most d. Let $\boldsymbol{x} \xleftarrow{\$} \mathbb{X}^n$ for some finite set $\mathbb{X} \subseteq \mathbb{F}$. Then:*

$$\Pr_{\boldsymbol{x} \xleftarrow{\$} \mathbb{X}^n} [\phi(\boldsymbol{x}) = 0] \leq \frac{n \cdot d}{|\mathbb{X}|}.$$

Definition 4 (Inner-product proof). *Let m be a positive power of two. Suppose that the prover $\mathcal{P}$ is bound to two vectors $\overline{\boldsymbol{a}}, \overline{\boldsymbol{b}} \in \mathbb{E}^m$, and that the verifier $\mathcal{V}$ holds the corresponding commitments. Given a claimed value $\overline{v} \in \mathbb{E}$, an* inner-product proof *is an interactive protocol for the relation:*

$$\mathcal{L}_{\mathsf{ip}} \coloneqq \left\{ \overline{v} \in \mathbb{E},\ \overline{\boldsymbol{a}}, \overline{\boldsymbol{b}} \in \mathbb{E}^m \;\middle|\; \overline{v} = \langle \overline{\boldsymbol{a}}, \overline{\boldsymbol{b}} \rangle \right\}.$$

Either sumcheck [62] or folding [24] can reduce the inner-product relation to two terminal MLE claims.

Fact 1 (**Inner-product reduction protocol).** *There is an inner-product reduction proof protocol with prover cost $4m + O(\log m)$ multiplications in $\mathbb{E}$, communication $(2\log_2 m + 2)$ $\mathbb{E}$-elements, and verifier cost $O(\log m)$ $\mathbb{E}$-multiplications. Over $\log_2 m$ rounds, the verifier samples challenges collected in $\overline{\boldsymbol{x}} \in \mathbb{E}^{\log_2 m}$, and the protocol reduces the inner-product relation to two terminal MLE claims:*

$$\overline{v}_1 \stackrel{?}{=} \langle \overline{\boldsymbol{a}}, \mathsf{ts}(\overline{\boldsymbol{x}}) \rangle \quad \wedge \quad \overline{v}_2 \stackrel{?}{=} \left\langle \overline{\boldsymbol{b}}, \mathsf{ts}(\overline{\boldsymbol{x}}) \right\rangle,$$

where the prover sends $\overline{v}_1, \overline{v}_2 \in \mathbb{E}$. The soundness error is at most $\frac{2\log_2 m}{|\mathbb{E}|}$.

Definition 5 (Multilinear polynomial commitment scheme in matrix form). *Let $n \in 2^{\mathbb{N}}$. A multilinear PCS over $\mathbb{F}$ with evaluations over $\mathbb{E}$ consists of algorithms* $(\mathsf{Setup}, \mathsf{Comm}, \mathsf{Open})$ *such that:*

- $\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda, n)$ *takes the security parameter and the committed vector length n, and outputs public parameters* pp.
- $C \leftarrow \mathsf{Comm}(\mathsf{pp}, \boldsymbol{a}; \rho)$ *takes a vector $\boldsymbol{a} \in \mathbb{F}^n$ and fresh randomness ρ, and outputs a commitment C. We write $\mathsf{Comm}(\boldsymbol{a}; \rho)$ when* pp *is clear from context.*
- *The opening protocol* $\mathsf{bit} \leftarrow \mathsf{Open}(\mathsf{pp}, C, \overline{v}, \overline{\boldsymbol{x}} : \boldsymbol{a}, \rho)$ *is an argument of knowledge between $\mathcal{P}$ and $\mathcal{V}$ for the relation:*

$$\mathcal{R}_{\mathsf{open}} \coloneqq \left\{ \left(\mathsf{pp}, C, \overline{v}, \overline{\boldsymbol{x}}\right) : \left(\boldsymbol{a}, \rho\right) \;\middle|\; C = \mathsf{Comm}(\mathsf{pp}, \boldsymbol{a}; \rho) \wedge \overline{v} = \langle \boldsymbol{a}, \mathsf{ts}(\overline{\boldsymbol{x}}) \rangle \right\}.$$

At the end of the protocol, $\mathcal{V}$ outputs $\mathsf{bit} \in \{\mathsf{true}, \mathsf{false}\}$.

3.3 Extension-Field Arithmetic

Fact 2 (Base-field embedding). *Let $\mathbb{E}/\mathbb{F}$ be an extension. The canonical inclusion $\mathcal{I} : \mathbb{F} \hookrightarrow \mathbb{E}$ is injective and induces an injective inclusion $\mathbb{F}[X] \hookrightarrow \mathbb{E}[X]$.*

Definition 6 (Extension-field basis). *An ordered tuple $\mathcal{B} = \left(\overline{\beta}_1, \ldots, \overline{\beta}_\kappa\right) \in \mathbb{E}^\kappa$ is a* basis *of $\mathbb{E}$ over $\mathbb{F}$ if every $\overline{\alpha} \in \mathbb{E}$ admits a unique representation:*

$$\overline{\alpha} = \textstyle\sum_{i=1}^{\kappa} a_i \overline{\beta}_i, \qquad a_1, \ldots, a_\kappa \in \mathbb{F}.$$

Fact 3 (Extension-field equality by coordinates). *Let $\overline{\boldsymbol{\beta}} = (\overline{\beta}_1, \ldots, \overline{\beta}_\kappa)^\top$ be a basis of $\mathbb{E}$ over $\mathbb{F}$. For any matrices $\boldsymbol{A}, \boldsymbol{A'} \in \mathbb{F}^{m \times \kappa}$, $\boldsymbol{A}\overline{\boldsymbol{\beta}} = \boldsymbol{A'}\overline{\boldsymbol{\beta}}$ if and only if $\boldsymbol{A} = \boldsymbol{A'}$. In particular, two elements of $\mathbb{E}$ are equal if and only if their coordinates with respect to $\overline{\boldsymbol{\beta}}$ are equal.*

Besides extension-field bases, we also pack with polynomial bases, namely κ linearly independent polynomials in $\mathbb{F}[X]$ of degree at most $\kappa - 1$. Such polynomials form a basis of $\mathbb{F}[X]_{<\kappa}$, and hence also of $\mathbb{E}[X]_{<\kappa}$ over $\mathbb{E}$. A canonical choice is the Lagrange basis.

Definition 7 (Lagrange basis over the extension field). *Assume that $q \geq \kappa$, and fix an ordered set of distinct roots $\mathcal{H} = \{\gamma_1, \ldots, \gamma_\kappa\} \subseteq \mathbb{F}$. The indices $j \in [\kappa]$ label the chosen field elements. The vanishing polynomial of $\mathcal{H}$ is:*

$$\Psi(X) \coloneqq \textstyle\prod_{j=1}^{\kappa} (X - \gamma_j) \in \mathbb{F}[X].$$

For each $j \in [\kappa]$, define the Lagrange basis polynomial $\psi_j(X) \coloneqq \prod_{\substack{i \in [\kappa] \\ i \neq j}} \frac{X - \gamma_i}{\gamma_j - \gamma_i} \in \mathbb{F}[X]$. These polynomials satisfy:

$$\psi_j(\gamma_i) = \delta_{ij} \qquad \text{for all } i, j \in [\kappa].$$

Moreover, $\{\psi_j(X)\}_{j=1}^{\kappa}$ forms an $\mathbb{E}$-basis for the polynomials in $\mathbb{E}[X]$ of degree less than κ. Every $g(X) \in \mathbb{E}[X]$ with $\deg g < \kappa$ admits the unique representation:

$$g(X) = \textstyle\sum_{j=1}^{\kappa} \overline{a}_j \psi_j(X), \qquad \overline{a}_j = g(\gamma_j) \in \mathbb{E}.$$

4 Building Blocks

This section presents the bitwidth-aware building blocks of `Mikros`. Each building block uses algebraic vector packing to reduce a relation over $\mathbb{F}$ to a shorter relation over $\mathbb{E}$, which is then proved by an existing extension-field protocol. Throughout, we assume extension-field protocols for inner and Hadamard products whose prover cost is linear in the vector length (Fact 1). We first illustrate algebraic vector packing with inner-product proofs (Section 4.1). We then present packed Hadamard-product proofs (Section 4.2) and packed permutation and lookup arguments (Section 4.3). Section 5 combines these building blocks into end-to-end SNARKs.

4.1 Packed Inner-Product Proof

We illustrate algebraic vector packing by showing how it reduces a length-N inner-product claim over $\mathbb{F}$ to a length-m claim over $\mathbb{E}$, where $N = m\kappa$ and $\kappa \geq 2$. Without loss of generality, we assume that m is a power of two, since the vectors can be padded with zeros. Consider the claim:

$$c = \langle \boldsymbol{a}, \boldsymbol{b} \rangle, \ \ \boldsymbol{a}, \boldsymbol{b} \in \mathbb{F}^N.$$

We partition each vector into κ blocks of length m, and arrange these blocks as:

$$\boldsymbol{A} \coloneqq \left[\boldsymbol{a}^{(1)} \mid \ldots \mid \boldsymbol{a}^{(\kappa)} \right], \qquad \boldsymbol{B} \coloneqq \left[\boldsymbol{b}^{(1)} \mid \ldots \mid \boldsymbol{b}^{(\kappa)} \right] \in \mathbb{F}^{m \times \kappa}. \tag{5}$$

We use the monomial basis $\boldsymbol{\phi}_A(X) = (1, X, \ldots, X^{\kappa-1})^\top$ and the reciprocal monomial basis $\boldsymbol{\phi}_B(X) = (1, X^{-1}, \ldots, X^{-(\kappa-1)})^\top$ to pack the columns into a polynomial vector and a Laurent polynomial vector, respectively:

$$\boldsymbol{g}_A(X) \coloneqq \textstyle\sum_{j=1}^{\kappa} \boldsymbol{a}^{(j)} X^{j-1} \in \mathbb{F}[X]^m, \quad \boldsymbol{g}_B(X) \coloneqq \textstyle\sum_{j=1}^{\kappa} \boldsymbol{b}^{(j)} X^{-(j-1)} \in \mathbb{F}[X^{-1}]^m.$$

Their inner product defines the Laurent polynomial:

$$f(X) \coloneqq \langle \boldsymbol{g}_A(X), \boldsymbol{g}_B(X) \rangle \,. \tag{6}$$

Its constant coefficient is $\sum_{j=1}^{\kappa} \langle \boldsymbol{a}^{(j)}, \boldsymbol{b}^{(j)} \rangle = \langle \boldsymbol{a}, \boldsymbol{b} \rangle$, while all cross-column terms have nonzero exponents. Thus, $c = \langle \boldsymbol{a}, \boldsymbol{b} \rangle$ if and only if there exist $h_+, h_- \in \mathbb{F}[X]$, each of degree at most $\kappa - 2$, such that:

$$f(X) = c + X h_+(X) + X^{-1} h_-(X^{-1}). \tag{7}$$

With c and the commitments to $\boldsymbol{A}$ and $\boldsymbol{B}$ fixed, the prover sends the coefficients of h_+ and h_-. The verifier then samples $\overline{x} \xleftarrow{\$} \mathbb{E}^\times$. The prover evaluates both encodings at $\overline{x}$ to obtain:

$$\overline{\boldsymbol{a}} \coloneqq \boldsymbol{g}_A(\overline{x}) \in \mathbb{E}^m, \qquad \overline{\boldsymbol{b}} \coloneqq \boldsymbol{g}_B(\overline{x}) \in \mathbb{E}^m.$$

Evaluating Eq. (7) at $\overline{x}$ reduces the polynomial identity to the following length-m extension-field inner-product claim:

$$\left\langle \overline{\boldsymbol{a}}, \overline{\boldsymbol{b}} \right\rangle = c + \overline{x} h_+(\overline{x}) + \overline{x}^{-1} h_-(\overline{x}^{-1}). \tag{8}$$

The verifier computes the right-hand side from c, the auxiliary polynomials, and $\overline{x}$. The parties then apply the extension-field inner-product protocol of Fact 1, reducing this claim to two MLE claims on the length-m packed vectors. These terminal claims are proved using a packed PCS with respect to the original witness commitments. Algorithm 1 in Appendix B.1 gives the detailed protocol.

Proposition 1 (Packed inner-product proof). *Algorithm 1 is complete for the base-field inner-product relation. With the terminal MLE claims verified, its soundness error is at most:*

$$(2\kappa - 2)/(|\mathbb{E}| - 1) + 2\log_2 m/|\mathbb{E}|.$$

Its prover time is $O(N\kappa + \kappa^3)\mathsf{bb}$ and its peak prover memory is $O(N + \kappa^2)\mathbb{F}$.

We prove Proposition 1 in Appendix C.1.

4.2 Packed Hadamard-Product Proof

We next apply polynomial packing, using a different choice of basis polynomials, to a weighted Hadamard-product relation:

$$\overline{c} = \left\langle \overline{\boldsymbol{\xi}}, \boldsymbol{a} \circ \boldsymbol{b} \right\rangle \in \mathbb{E}, \qquad \overline{\boldsymbol{\xi}} = \overline{\boldsymbol{r}} \otimes \overline{\boldsymbol{l}} \in \mathbb{E}^N, \tag{9}$$

where $\boldsymbol{a}, \boldsymbol{b} \in \mathbb{F}^N$, $\overline{\boldsymbol{l}} \in \mathbb{E}^m$, $\overline{\boldsymbol{r}} \in \mathbb{E}^\kappa$, and $N = m\kappa$. The row weight $\overline{\boldsymbol{l}}$ is supplied as a tensor product of $\log_2 m$ two-entry factors, as in Eq. (4). When expressed using MLE polynomials, this weighted Hadamard relation corresponds to the degree-two zerocheck setting considered by Dao and Thaler [26] and Gruen [39].

The tensor-structured weight vector factors as $\overline{\boldsymbol{\xi}} = \overline{\boldsymbol{r}} \otimes \overline{\boldsymbol{l}}$. Using the block matrices $\boldsymbol{A}, \boldsymbol{B} \in \mathbb{F}^{m \times \kappa}$ defined in Eq. (5), we can therefore rewrite Eq. (9) as:

$$\overline{c} = \overline{\boldsymbol{l}}^\top (\boldsymbol{A} \circ \boldsymbol{B}) \overline{\boldsymbol{r}} \in \mathbb{E}.$$

To prove this identity, the prover sends $\overline{\boldsymbol{d}}^\top \coloneqq \overline{\boldsymbol{l}}^\top (\boldsymbol{A} \circ \boldsymbol{B}) \in \mathbb{E}^\kappa$. The verifier checks $\overline{c} \stackrel{?}{=} \langle \overline{\boldsymbol{d}}, \overline{\boldsymbol{r}} \rangle$. We then encode these κ column claims at distinct interpolation nodes to verify them jointly, writing $\overline{\boldsymbol{d}} = (\overline{d}^{(1)}, \ldots, \overline{d}^{(\kappa)})^\top$.

We choose both $\boldsymbol{\phi}_A(X)$ and $\boldsymbol{\phi}_B(X)$ to be the Lagrange basis $(\psi_1(X), \ldots, \psi_\kappa(X))^\top$ from Definition 7, and let $\Psi(X)$ denote the corresponding vanishing polynomial. We define:

$$\boldsymbol{g}_A(X) \coloneqq \sum_{j=1}^{\kappa} \boldsymbol{a}^{(j)} \psi_j(X), \quad \boldsymbol{g}_B(X) \coloneqq \sum_{j=1}^{\kappa} \boldsymbol{b}^{(j)} \psi_j(X), \quad g_C(X) \coloneqq \sum_{j=1}^{\kappa} \overline{d}^{(j)} \psi_j(X).$$

When $|\mathbb{F}| \geq 2\kappa-1$, we choose κ distinct interpolation nodes in $\mathbb{F}$, so the Lagrange basis polynomials have coefficients in $\mathbb{F}$. Consequently:

$$\boldsymbol{g}_A(X), \boldsymbol{g}_B(X) \in \mathbb{F}[X]^m, \qquad g_C(X) \in \mathbb{E}[X].$$

We define the residual polynomial:

$$f(X) \coloneqq \left\langle \overline{\boldsymbol{l}}, \boldsymbol{g}_A(X) \circ \boldsymbol{g}_B(X) \right\rangle - g_C(X) \in \mathbb{E}[X].$$

At each interpolation root $\gamma_j \in \mathcal{H}$, the property of the Lagrange basis gives:

$$f(\gamma_j) = \left\langle \overline{\boldsymbol{l}}, \boldsymbol{a}^{(j)} \circ \boldsymbol{b}^{(j)} \right\rangle - \overline{d}^{(j)}.$$

Thus, all κ column claims hold if and only if f vanishes on $\mathcal{H}$, or equivalently, $\Psi(X)$ divides $f(X)$. Since $\deg f \leq 2\kappa - 2$ and $\deg \Psi = \kappa$, this is equivalent to the existence of $h(X) \in \mathbb{E}[X]$ with $\deg h \leq \kappa - 2$ such that

$$f(X) = \Psi(X)h(X). \tag{10}$$

For valid claims, the values of f on $\mathcal{H}$ are known to be zero. When $|\mathbb{F}| \geq 2\kappa-1$, the prover can therefore recover f by evaluating it at $\kappa - 1$ distinct points in $\mathbb{F} \setminus \mathcal{H}$ and interpolating these values together with the known zeros. Dividing the resulting polynomial by Ψ then gives h.

After the prover sends $\overline{\boldsymbol{d}}$ and h, the verifier samples $\overline{x} \xleftarrow{\$} \mathbb{E} \setminus \mathcal{H}$. The Lagrange evaluations give the packing weights:

$$\overline{\boldsymbol{\rho}}(\overline{x}) \coloneqq \big(\psi_1(\overline{x}), \ldots, \psi_\kappa(\overline{x})\big)^\top \in \mathbb{E}^\kappa.$$

The prover computes the packed vectors:

$$\overline{\boldsymbol{a}} \coloneqq \boldsymbol{A}\overline{\boldsymbol{\rho}}(\overline{x}) = \boldsymbol{g}_A(\overline{x}) \in \mathbb{E}^m, \qquad \overline{\boldsymbol{b}} \coloneqq \boldsymbol{B}\overline{\boldsymbol{\rho}}(\overline{x}) = \boldsymbol{g}_B(\overline{x}) \in \mathbb{E}^m,$$

and the verifier directly computes a degree-$(\kappa - 1)$ polynomial value:

$$\overline{c}_x \coloneqq g_C(\overline{x}) = \left\langle \overline{\boldsymbol{d}}, \overline{\boldsymbol{\rho}}(\overline{x}) \right\rangle \in \mathbb{E}.$$

Evaluating Eq. (10) at $\overline{x}$ reduces the Hadamard-product claim to:

$$\left\langle \overline{\boldsymbol{a}}, \overline{\boldsymbol{b}} \circ \overline{\boldsymbol{l}} \right\rangle = \overline{c}_x + \Psi(\overline{x})h(\overline{x}).$$

This is again a length-m inner-product relation over $\mathbb{E}$. Applying the inner-product protocol of Fact 1 reduces it to MLE claims on $\overline{\boldsymbol{a}}$ and $\overline{\boldsymbol{b}} \circ \overline{\boldsymbol{l}}$. At the terminal point $\overline{\boldsymbol{y}}$, their row weights on the original matrices are $\mathsf{ts}(\overline{\boldsymbol{y}})$ and $\overline{\boldsymbol{l}} \circ \mathsf{ts}(\overline{\boldsymbol{y}})$, respectively; both retain tensor weights and fit the packed-opening interface.

Algorithm 2 in Appendix B.1 specifies the protocol.

Small fields via an intermediate field. When $q = |\mathbb{F}| < 2\kappa_0 - 1$, let $\kappa_0 \geq 2$ denote the original extension degree and choose $t = \left\lceil \log_q(4\kappa_0) \right\rceil$ and $\kappa = t \left\lceil \frac{\kappa_0}{t} \right\rceil$.

Then $t \mid \kappa$, $\kappa_0 \le \kappa \le 2\kappa_0$, and $q^t \ge 4\kappa_0 > 2\kappa - 1$. Moreover, $\kappa = \kappa_0 + O(\log_q \kappa_0)$. Taking $\mathbb{E} = \mathbb{F}_{q^\kappa}$ gives:

$$\mathbb{F} \subseteq \mathbb{K} = \mathbb{F}_{q^t} \subseteq \mathbb{E}, \ [\mathbb{E} : \mathbb{K}] = \kappa/t.$$

Choose $\mathcal{H} \subseteq \mathbb{K}$ of size κ and define $\boldsymbol{\phi}(X), \Psi(X), \boldsymbol{g}_A(X), \boldsymbol{g}_B(X), g_C(X)$ as above. Arrange the witnesses as $\boldsymbol{A}, \boldsymbol{B} \in \mathbb{F}^{m \times \kappa}$, with $N = m\kappa$. The witness entries remain over $\mathbb{F}$, while:

$$\boldsymbol{g}_A(X), \boldsymbol{g}_B(X) \in \mathbb{K}[X]^m, \qquad g_C(X) \in \mathbb{E}[X].$$

We implement the interpolation procedure using precomputed addition and multiplication tables over $\mathbb{K}$, which also handle inputs in $\mathbb{F} \times \mathbb{K}$. Together, these tables require $O(t^2 q^{2t})$ base-field preprocessing operations and $O(tq^{2t})$ base-field coordinates of storage. Caching the evaluation and interpolation data requires an additional $O(\kappa^2 t)$ coordinates. We store table arguments and outputs as encoded $\mathbb{K}$-elements and charge each table access as $O(1)$.

Under this convention, evaluating the witness encodings and aggregating their row products costs $O(N\kappa)$. Evaluating g_C, interpolating f, and computing the quotient cost $O(\kappa^3/t)$. The online prover time and total storage are:

$$\mathcal{TP} = O(N\kappa + \kappa^3), \qquad \mathcal{SP} = O(N + \kappa^2 t + tq^{2t}).$$

The time counts base-field arithmetic and unit-cost encoded-field table accesses; storage is measured in base-field coordinates. The storage bound includes all precomputed data and is achieved by processing sample points and witness rows sequentially. For $m \ge \kappa$, the online prover time simplifies to $O(N\kappa)$.

The packed polynomial identity, challenge sampling over $\mathbb{E}$, and terminal extension-field inner-product proof carry over to these parameters. The same soundness and complexity bounds for the remaining steps therefore apply.

Proposition 2 (Packed Hadamard-product proof). *Let $q = |\mathbb{F}|$, $N = m\kappa$, and $t = [\mathbb{K} : \mathbb{F}]$, where $\mathbb{K}$ is the interpolation field and may equal $\mathbb{F}$. Algorithm 2 is complete for the weighted Hadamard-product relation. With the terminal MLE claims verified, its soundness error is at most:*

$$(2\kappa - 2)/(|\mathbb{E}| - \kappa) + 2\log_2 N/|\mathbb{E}|.$$

With the precomputed data described above, its online prover cost, excluding terminal MLE openings, is $O(N\kappa + \kappa^3)$ bb. For $t > 1$, this cost also counts unit-cost encoded-$\mathbb{K}$ table accesses. Its peak prover memory, including all precomputed data, is $O\big(N + \kappa^2 t + (t-1)q^{2t}\big)\mathbb{F}$.

We prove Proposition 2 in Appendix C.2; Algorithm 2 specifies the protocol.

4.3 Packed Permutation Proof

We next consider the permutation relation:

$$\boldsymbol{a} \overset{\mathsf{perm}}{=} \boldsymbol{b}, \qquad \boldsymbol{a}, \boldsymbol{b} \in \mathbb{F}^N.$$

It asserts that the two vectors contain the same multiset of entries. Such proofs verify the PLONK copy constraints in Eq. (3) and are used in lookup constructions [60]. After the witness commitments are fixed, the verifier samples $\overline{z} \xleftarrow{\$} \mathbb{E}$. The prover claims a common fingerprint $\overline{\Theta}$ and proves:

$$\overline{\Theta} = \mathsf{GrandProd}(\overline{z}\mathbf{1} + \boldsymbol{a}) = \mathsf{GrandProd}(\overline{z}\mathbf{1} + \boldsymbol{b}).$$

The two fingerprint polynomials are identical if and only if the permutation relation holds. It therefore suffices to prove the two grand-product claims. We describe the proof for $\boldsymbol{a}$.

Grand products from Hadamard products. Arrange $\boldsymbol{a}$ as $\boldsymbol{A} \in \mathbb{F}^{m\times\kappa}$, as in Eq. (5), with $N = m\kappa$. Let $q = |\mathbb{F}|$. Throughout this section we assume, without loss of generality, that m and κ are powers of two with $m \geq \kappa \geq 2$. We first construct a polynomial product tree over $\mathbb{F}$, keeping κ columns and halving the number of rows at each level:

$$\begin{aligned} \boldsymbol{C}_0(Z) &\coloneqq Z \cdot \mathbf{1}_{m\times\kappa} + \boldsymbol{A}, \\ \boldsymbol{C}_{\tau+1}(Z) &\coloneqq \boldsymbol{C}_{\tau,U}(Z) \circ \boldsymbol{C}_{\tau,D}(Z), \qquad \tau \in [0, \log_2 \kappa - 1], \end{aligned} \tag{11}$$

where $\boldsymbol{C}_{\tau,U}(Z)$ and $\boldsymbol{C}_{\tau,D}(Z)$ are the upper and lower row halves of $\boldsymbol{C}_\tau(Z)$. Thus:

$$\boldsymbol{C}_\tau(Z) \in \mathbb{F}[Z]^{(m/2^\tau)\times\kappa}, \qquad \deg_Z \boldsymbol{C}_\tau(Z) = 2^\tau.$$

Each entry of $\boldsymbol{C}_{\log_2\kappa}(Z)$ is a monic degree-κ product $\prod_\ell (Z + a_\ell)$ over the κ rows it aggregates, obtainable by Horner's method in $(\kappa^2/2)\mathsf{bb}$. With these polynomial coefficients over $\mathbb{F}$, the prover can evaluate $\boldsymbol{C}_{\log_2\kappa}(Z)$ using precomputed powers of $\overline{z}$ and $\mathbb{F}$-$\mathbb{E}$ products. Its m entries form the vector $\boldsymbol{c}_{\log_2\kappa}(\overline{z}) \in \mathbb{E}^m$. The remaining levels are computed over $\mathbb{E}$:

$$\boldsymbol{c}_{\tau+1}(\overline{z}) = \boldsymbol{c}_{\tau,U}(\overline{z}) \circ \boldsymbol{c}_{\tau,D}(\overline{z}), \qquad \tau \in [\log_2\kappa, \log_2 N - 1],$$

where U, D now denote vector halves. The final entry is $\boldsymbol{c}_{\log_2 N}(\overline{z}) = \overline{\Theta}$.

The proof traverses these trees in reverse. Starting from $\overline{\Theta}$, weighted Hadamard proofs over $\mathbb{E}$ reduce the last $\log_2 m$ levels to an MLE claim on $\boldsymbol{c}_{\log_2\kappa}(\overline{z})$. Factoring its tensor weight into column and row weights gives:

$$\overline{v}_{\log_2\kappa} = \overline{\boldsymbol{l}}_{\log_2\kappa}^\top \boldsymbol{C}_{\log_2\kappa}(\overline{z})\overline{\boldsymbol{r}}_{\log_2\kappa}.$$

At each subsequent level, the recursive claim has the same form:

$$\overline{v}_{\tau+1} = \overline{\boldsymbol{l}}_{\tau+1}^\top \boldsymbol{C}_{\tau+1}(\overline{z})\overline{\boldsymbol{r}}_{\tau+1}, \qquad \overline{\boldsymbol{l}}_{\tau+1} \in \mathbb{E}^{m/2^{\tau+1}}, \quad \overline{\boldsymbol{r}}_{\tau+1} \in \mathbb{E}^\kappa,$$

with a tensor-structured row weight and fixed column weights.

Base-field interpolation: $q \geq 2\kappa - 1$. In this case, the interpolation points lie in $\mathbb{F}$. We use the Lagrange basis from Section 4.2:

$$\boldsymbol{\phi}(X) \coloneqq (\psi_1(X), \ldots, \psi_\kappa(X))^\top, \qquad \boldsymbol{g}_\tau(Z, X) \coloneqq \boldsymbol{C}_\tau(Z)\boldsymbol{\phi}(X).$$

Choose $\kappa - 1$ distinct points $\gamma'_1, \ldots, \gamma'_{\kappa-1} \in \mathbb{F} \setminus \mathcal{H}$. For each level $\tau \in [\log_2 \kappa]$ and point γ'_j, fix $X = \gamma'_j$ and retain Z as a variable. Define the polynomial vector:

$$\boldsymbol{\eta}_{\tau,j}(Z) \coloneqq \boldsymbol{g}_{\tau-1,U}(Z, \gamma'_j) \circ \boldsymbol{g}_{\tau-1,D}(Z, \gamma'_j) = \textstyle\sum_{i=0}^{2^\tau} \boldsymbol{p}_{\tau,j,i} Z^i, \quad \boldsymbol{p}_{\tau,j,i} \in \mathbb{F}^{m/2^\tau}.$$

Each entry has degree at most 2^τ in Z. In particular, since $\sum_j \psi_j(X) = 1$:

$$\boldsymbol{g}_0(Z, \gamma'_j) = Z \cdot \mathbf{1}_m + \boldsymbol{A}\boldsymbol{\phi}(\gamma'_j).$$

The prover computes the coefficients $\boldsymbol{p}_{\tau,j,i}$ over $\mathbb{F}$. The weighted polynomial is:

$$F_{\tau,j}(Z) \coloneqq \left\langle \bar{\boldsymbol{l}}_\tau, \boldsymbol{\eta}_{\tau,j}(Z) \right\rangle = \textstyle\sum_{i=0}^{2^\tau} \left\langle \bar{\boldsymbol{l}}_\tau, \boldsymbol{p}_{\tau,j,i} \right\rangle Z^i \in \mathbb{E}[Z].$$

The prover first aggregates the coefficient vectors $\boldsymbol{p}_{\tau,j,i}$ against $\bar{\boldsymbol{l}}_\tau$, obtaining the extension-field coefficients of $F_{\tau,j}(Z)$. It then evaluates this scalar polynomial at $\overline{z}$. The powers $1, \overline{z}, \ldots, \overline{z}^\kappa$ are shared across all levels.

For the transition from $\boldsymbol{C}_{\tau+1}$ to $\boldsymbol{C}_\tau$, where $\tau \in [0, \log_2 \kappa - 1]$, the prover applies Section 4.2 by sending the column claims:

$$\overline{\boldsymbol{u}}_{\tau+1}^\top \coloneqq \bar{\boldsymbol{l}}_{\tau+1}^\top \big(\boldsymbol{C}_{\tau,U}(\overline{z}) \circ \boldsymbol{C}_{\tau,D}(\overline{z}) \big).$$

For each column, the prover first aggregates the coefficients of $\boldsymbol{C}_{\tau+1}(Z)$ against $\bar{\boldsymbol{l}}_{\tau+1}$, then evaluates the resulting scalar polynomial at $\overline{z}$. The verifier checks $\overline{v}_{\tau+1} = \overline{\boldsymbol{u}}_{\tau+1}^\top \overline{\boldsymbol{r}}_{\tau+1}$. As in Section 4.2, define:

$$g_C(X) \coloneqq \overline{\boldsymbol{u}}_{\tau+1}^\top \boldsymbol{\phi}(X), \quad f(X) \coloneqq \left\langle \bar{\boldsymbol{l}}_{\tau+1}, \boldsymbol{g}_{\tau,U}(\overline{z}, X) \circ \boldsymbol{g}_{\tau,D}(\overline{z}, X) \right\rangle - g_C(X).$$

For valid column claims, $f(X)$ vanishes on $\mathcal{H}$. Its off-domain samples are:

$$f(\gamma'_j) = F_{\tau+1,j}(\overline{z}) - g_C(\gamma'_j).$$

Interpolating them together with the zeros on $\mathcal{H}$ gives $f(X)$, and division by $\Psi(X)$ gives $h(X) \in \mathbb{E}[X]$ with $\deg h(X) \leq \kappa - 2$. The prover fixes $\overline{\boldsymbol{u}}_{\tau+1}$ and $h(X)$ before the verifier samples a fresh $\overline{x} \xleftarrow{\$} \mathbb{E} \setminus \mathcal{H}$. The resulting check is:

$$\left\langle \bar{\boldsymbol{l}}_{\tau+1}, \boldsymbol{g}_{\tau,U}(\overline{z}, \overline{x}) \circ \boldsymbol{g}_{\tau,D}(\overline{z}, \overline{x}) \right\rangle = g_C(\overline{x}) + \Psi(\overline{x}) h(\overline{x}).$$

This is a weighted Hadamard claim of length $m/2^{\tau+1}$ over $\mathbb{E}$.

Recursive reduction. We implement each extension-field weighted Hadamard proof by weighted sumcheck over $\mathbb{E}$. The checked polynomial has individual degree at most three, and the two child claims use the same row MLE point $\overline{\boldsymbol{y}}$. After their values $\overline{v}_{\tau,U}, \overline{v}_{\tau,D}$ are fixed, the verifier samples a fresh $\overline{y}_0 \xleftarrow{\$} \mathbb{E}$. The next claim is obtained by setting:

$$\overline{v}_\tau \coloneqq (1 - \overline{y}_0)\overline{v}_{\tau,U} + \overline{y}_0 \overline{v}_{\tau,D}, \qquad \bar{\boldsymbol{l}}_\tau \coloneqq \mathsf{ts}(\overline{y}_0 \parallel \overline{\boldsymbol{y}}), \qquad \overline{\boldsymbol{r}}_\tau \coloneqq \overline{\boldsymbol{\rho}}(\overline{x}).$$

It has the desired form $\overline{v}_\tau = \bar{\boldsymbol{l}}_\tau^\top \boldsymbol{C}_\tau(\overline{z}) \overline{\boldsymbol{r}}_\tau$. The same packed Hadamard-product proof is applied. At level zero, both weight vectors sum to one, leaving the claim:

$$\overline{v}_0 - \overline{z} = \bar{\boldsymbol{l}}_0^\top \boldsymbol{A} \overline{\boldsymbol{r}}_0.$$

Cost over $\mathbb{F}$. At level τ, forming the $\kappa-1$ encodings from $\boldsymbol{C}_{\tau-1}$ costs $O(N\kappa)\,\mathsf{bb}$. Their row products cost $O(m\kappa 2^{\tau})\,\mathsf{bb}$, totaling $O(N\kappa)\,\mathsf{bb}$ across all levels. Coefficient aggregation costs $O(N)\,\mathsf{be}$ per level, including the column claims. The aggregated polynomials have degree at most 2^{τ}, so evaluating them at $\overline{z}$ costs $O(\kappa 2^{\tau})\,\mathsf{ee}$ per level and $O(\kappa^2)\,\mathsf{ee}$ in total. To form the terminal Hadamard vectors, the prover first contracts the coefficients of $\boldsymbol{C}_{\tau}(Z)$ with $\overline{\boldsymbol{\rho}}(\overline{x})$, then evaluates the resulting row polynomials at $\overline{z}$, at a cost of $O(N)\,\mathsf{be}+O(m)\,\mathsf{ee}$ per level. Using shared powers of $\overline{x}$, interpolation and public weight generation cost $O(\kappa^3)\,\mathsf{bb}$ per level. The short Hadamard proofs and extension-field tree have total length $O(m)$. Thus, under schoolbook arithmetic, the prover cost for one grand-product proof is $O(N\kappa\log_2\kappa+\kappa^4)\mathsf{bb}$.

Intermediate-field tables over smaller fields: $q<2\kappa-1$. We use the tabulation procedure of Section 4.2, with $\mathbb{K}=\mathbb{F}_{q^t}\subseteq\mathbb{E}$ and t the least power of two at least $\lceil\log_q(2\kappa-1)\rceil$. All interpolation roots lie in $\mathbb{K}$. The coefficients of $\boldsymbol{C}_{\tau}(Z)$ remain in $\mathbb{F}$, while those of $\boldsymbol{g}_{\tau}(Z,X)$ and $\boldsymbol{\eta}_{\tau,j}(Z)$ lie in $\mathbb{K}$. The polynomial identities and recursive reduction are unchanged. Under the same table-lookup convention, the online prover cost per grand-product proof is $O\Big(N\kappa\log_2\kappa+\frac{\kappa^4}{t}\Big)\,\mathsf{bb}$.

Algorithm 3 in Appendix B.1 specifies the protocol for both cases.

Proposition 3 (Packed permutation proof). *Let $q=|\mathbb{F}|$ and let $N=m\kappa$, where $m\geq\kappa\geq 2$. Let $\mathbb{K}\subseteq\mathbb{E}$ be the interpolation field, with $t=[\mathbb{K}:\mathbb{F}]$, $t\mid\kappa$, and $q^t\geq 2\kappa-1$, taking $t=1$ when $q\geq 2\kappa-1$. Algorithm 3 is complete. Each grand-product reduction has soundness error at most:*

$$\epsilon_{\mathsf{GP}}\leq\frac{\frac{3}{2}\log_2 m(\log_2 m-1)+\log_2 m}{|\mathbb{E}|}+\sum_{\tau=0}^{\log_2\kappa-1}\left(\frac{2\kappa-2}{|\mathbb{E}|-\kappa}+\frac{3\log_2(m/2^{\tau+1})+1}{|\mathbb{E}|}\right).$$

With committed vectors, the permutation argument has soundness error at most:

$$\epsilon_{\mathsf{Perm}}\leq\ \min\{1,(N-1)/|\mathbb{E}|\}+2\epsilon_{\mathsf{GP}}+\epsilon_{\mathsf{PCS}},$$

where ϵ_{PCS} is the joint soundness error of the packed PCS for the two terminal linear claims. With preprocessed packing data and, for $t>1$, the unit-cost field tables above, the online prover cost for one grand product, excluding commitments and terminal openings, is $O\big(N\kappa\log_2\kappa+\kappa^4\big)\ \mathsf{bb}$. The peak prover memory, including all precomputed tables, is: $O\big(N+\kappa^3+(t-1)q^{2t}\big)\mathbb{F}$.

Proof of this proposition is provided in Appendix C.3.

4.4 Packed Polynomial Commitment Scheme

We complete the packed reductions by proving their terminal linear claims with respect to a fixed-basis commitment. Let $N=m\kappa$ and $\boldsymbol{A}\in\mathbb{F}^{m\times\kappa}$. The preceding protocols produce claims of the form:

$$\overline{v}=\langle\boldsymbol{A}\overline{\boldsymbol{\rho}},\overline{\boldsymbol{\xi}}\rangle=\overline{\boldsymbol{\xi}}^{\top}\boldsymbol{A}\overline{\boldsymbol{\rho}},\qquad\overline{\boldsymbol{\rho}}\in\mathbb{E}^{\kappa},\quad\overline{\boldsymbol{\xi}}\in\mathbb{E}^{m},$$

where $\overline{\boldsymbol{\xi}}$ is a tensor-structured row weight. The column weights $\overline{\boldsymbol{\rho}}$ may differ from the fixed $\mathbb{F}$-basis $\overline{\boldsymbol{\beta}} \in \mathbb{E}^\kappa$ used to commit to the packed vector:

$$\overline{\boldsymbol{a}} \coloneqq \boldsymbol{A}\overline{\boldsymbol{\beta}} \in \mathbb{E}^m.$$

To relate the two packings, the prover sends the column evaluations $\overline{\boldsymbol{v}}^\top \coloneqq \overline{\boldsymbol{\xi}}^\top \boldsymbol{A}$. The verifier checks $\overline{v} \stackrel{?}{=} \langle \overline{\boldsymbol{v}}, \overline{\boldsymbol{\rho}} \rangle$. Write the row weights and the claimed column evaluations in base-field coordinates:

$$\overline{\boldsymbol{\xi}} = \boldsymbol{\Xi}\overline{\boldsymbol{\beta}}, \qquad \overline{\boldsymbol{v}}^\top = \overline{\boldsymbol{\beta}}^\top \boldsymbol{U}, \qquad \boldsymbol{\Xi} \in \mathbb{F}^{m \times \kappa}, \quad \boldsymbol{U} \in \mathbb{F}^{\kappa \times \kappa}.$$

Since $\boldsymbol{A}$ has base-field entries, the column claims are equivalent to $\boldsymbol{U} = \boldsymbol{\Xi}^\top \boldsymbol{A}$. Packing this matrix identity with the fixed basis gives:

$$\boldsymbol{U}\overline{\boldsymbol{\beta}} = \boldsymbol{\Xi}^\top \boldsymbol{A}\overline{\boldsymbol{\beta}} = \boldsymbol{\Xi}^\top \overline{\boldsymbol{a}} \in \mathbb{E}^\kappa.$$

After $\overline{\boldsymbol{v}}$ is fixed, a fresh random projection reduces this vector identity to a length-m inner-product claim involving the committed vector $\overline{\boldsymbol{a}}$. Fact 1 then leaves one MLE opening of $\overline{\boldsymbol{a}}$ and one terminal MLE claim of the public weight vector. The verifier computes the latter directly from the tensor factors of $\overline{\boldsymbol{\xi}}$.

Appendix B.2 gives the full construction and its analysis; Algorithm 4 specifies the protocol. Under schoolbook arithmetic and $m \geq \kappa$, the packed opening reduction uses $O(\kappa N)\,\mathsf{bb}$ and $O(N)\mathbb{F}$ memory, given an underlying extension-field PCS that commits and opens in $O(m)\mathsf{ee}$ and uses $O(m)\mathbb{E}$ of memory.

5 Putting Everything Together

We now combine the building blocks of Section 4 into an end-to-end bitwidth-aware SNARK for circuit satisfiability. For a single instance, we use the PLONK constraint system (Definition 1). For multiple instances of the same circuit, the R1CS constraint system is more efficient, since it commits to fewer witness values while its constraint matrices are shared by all instances.

In either system, the constraints encode not only the witness values but also the wire positions. Bitwidth awareness therefore accounts for both the $\lceil \log_2 q \rceil$-bit witness entries, where $q = |\mathbb{F}|$, and the $\lceil \log_2 N \rceil$-bit position labels. We call $\lceil \log_2 q \rceil + \lceil \log_2 N \rceil$ the *effective bitwidth* and aim for single-instance prover time and memory that scale linearly with it. Across multiple instances of the same circuit, however, the wiring is shared, so its encoding and the associated proof work are amortized, and the per-instance cost scales with $\lceil \log_2 q \rceil$ alone.

Sections 5.2 and 5.3 present the single- and multiple-instance constructions.

5.1 Proof for the PLONK Copy Constraint

We reduce the PLONK copy constraint $\sigma(\boldsymbol{w}) = \boldsymbol{w}$ in Eq. (3) to a permutation relation over an intermediate field. Let $\boldsymbol{w} \in \mathbb{F}^L$, where $L = 3N + n_{\mathsf{io}}$, and let $q = |\mathbb{F}|$. The reduction encodes each witness value together with its position.

Joint encoding of values and positions. Choose $\underline{q} = 2^s \le q$, with $s = \Theta(\log_2 q)$, and elements $\delta_1, \dots, \delta_s \in \mathbb{F}$ such that the map $\sum_{\ell=1}^{s} 2^{\ell-1} b_\ell \mapsto \sum_{\ell=1}^{s} \delta_\ell b_\ell$ is injective on $\{0,1\}^s$. For prime q, take $s = \lfloor \log_2 q \rfloor$ and $\delta_\ell = 2^{\ell-1}$. Appendix B.3 gives a choice for every finite field. We reserve $\lceil \log_{\underline{q}} L \rceil$ base-field coordinates for the position and one for the witness value. Choose d as the power of two at least $\lceil \log_{\underline{q}} L \rceil + 1$, and choose an extension degree κ with $2d \le \kappa$. This gives the tower:

$$\mathbb{F} \subset \mathbb{K}_{\mathsf{pos}} \subset \mathbb{E}, \quad [\mathbb{K}_{\mathsf{pos}} : \mathbb{F}] = d, \quad d \mid \kappa, \quad [\mathbb{E} : \mathbb{K}_{\mathsf{pos}}] = \kappa_{\mathsf{pos}} = \kappa/d.$$

We fix an $\mathbb{F}$-basis $\boldsymbol{\theta} = (\theta_1, \dots, \theta_d)^\top \in \mathbb{K}_{\mathsf{pos}}^d$ of $\mathbb{K}_{\mathsf{pos}}$.

Then, we can represent σ by $\boldsymbol{\sigma} = (\sigma(1), \dots, \sigma(L))^\top$, and let $\boldsymbol{i} = (1, \dots, L)^\top$. Write their zero-based positions in radix $\underline{q}$:

$$\boldsymbol{\sigma} - \mathbf{1}_L = \textstyle\sum_{j=1}^{\lceil \log_{\underline{q}} L \rceil} \underline{q}^{j-1} \boldsymbol{\sigma}_{(j)}, \;\; \boldsymbol{i} - \mathbf{1}_L = \sum_{j=1}^{\lceil \log_{\underline{q}} L \rceil} \underline{q}^{j-1} \boldsymbol{i}_{(j)}, \;\; \boldsymbol{\sigma}_{(j)}, \boldsymbol{i}_{(j)} \in [0, \underline{q} - 1]^L.$$

These are integer decompositions. In field expressions, we interpret each digit through the injection above. Consequently, we encode the permutation images and identity positions as:

$$\boldsymbol{\varsigma} \coloneqq \textstyle\sum_{j=1}^{\lceil \log_{\underline{q}} L \rceil} \theta_j \boldsymbol{\sigma}_{(j)} \in \mathbb{K}^L, \qquad \boldsymbol{\iota} \coloneqq \sum_{j=1}^{\lceil \log_{\underline{q}} L \rceil} \theta_j \boldsymbol{i}_{(j)} \in \mathbb{K}_{\mathsf{pos}}^L.$$

The labels $\iota_1, \dots, \iota_L$ are distinct, and $\varsigma_i = \iota_{\sigma(i)}$. The remaining basis coordinate binds each label to a witness value. We define:

$$\boldsymbol{a} \coloneqq \boldsymbol{\varsigma} + \theta_d \boldsymbol{w} \in \mathbb{K}^L, \qquad \boldsymbol{b} \coloneqq \boldsymbol{\iota} + \theta_d \boldsymbol{w} \in \mathbb{K}_{\mathsf{pos}}^L.$$

Lemma 2 (Copy constraints from a permutation relation). *For the encodings above and every $\boldsymbol{w} \in \mathbb{F}^L$:*

$$\sigma(\boldsymbol{w}) = \boldsymbol{w} \quad \Longleftrightarrow \quad \boldsymbol{a} \overset{\mathsf{perm}}{=} \boldsymbol{b}.$$

The basis coordinates make the joint encoding injective. Consequently, the label in a_i can match only the label in $b_{\sigma(i)}$, and equality of these entries enforces $w_i = w_{\sigma(i)}$. Appendix C.5 gives the proof.

Applying the packed permutation proof. With the witness commitments fixed, both operands are determined by $\boldsymbol{w}$ and the public permutation. We treat $\mathbb{K}_{\mathsf{pos}}$ as the base field and apply Algorithm 3 over $\mathbb{E}$ with packing width κ_{pos}, chosen so that $|\mathbb{K}_{\mathsf{pos}}| \ge 2\kappa_{\mathsf{pos}} - 1$. All required interpolation points can therefore be chosen in $\mathbb{K}_{\mathsf{pos}}$. The permutation proof leaves weighted evaluations with arbitrary column weights and tensor row weights on $\boldsymbol{a}$ and $\boldsymbol{b}$. By linearity, these reduce to evaluations of $\boldsymbol{\varsigma}, \boldsymbol{\iota} \in \mathbb{K}_{\mathsf{pos}}^L$ and $\boldsymbol{w} \in \mathbb{F}^L$. The verifier evaluates $\boldsymbol{\iota}$ directly using $O(\kappa_{\mathsf{pos}} + \log L)$ extension-field operations, as shown in Appendix B.3; ordinary MLE evaluations need only $O(\log L)$ operations. The terminal linear claims are verified using the packed PCS.

Extension to lookup arguments. Using a similar technique, we can also obtain an indexed lookup argument; see Appendix B.5.

5.2 Packing for a Single Circ-SAT Instance

The copy constraints are handled by the permutation proof of Section 4.3, so it remains to prove the PLONK gate constraints of Eq. (2). Let $N = m\kappa$, and use the polynomial basis of Section 4.4 for the wire commitments. Before any challenge is sampled, the prover commits to the concatenation of the three packed wire vectors; the selector and position-table commitments are preprocessed, with total packed length $m_{\mathsf{pub}} = (5 + 3\lceil \log_{\underline{q}}(3N + n_{\mathsf{io}})\rceil)m$.

To reduce the N gate equations to a single claim, the verifier samples $\overline{\boldsymbol{x}} \xleftarrow{\$} \mathbb{E}^{\log_2 N}$, sets $\overline{\boldsymbol{\xi}} = \mathsf{ts}(\overline{\boldsymbol{x}})$, and checks one random projection of the gate residual:

$$\left\langle \overline{\boldsymbol{\xi}}, \boldsymbol{q}_L \circ \boldsymbol{w}_L + \boldsymbol{q}_R \circ \boldsymbol{w}_R + \boldsymbol{q}_M \circ (\boldsymbol{w}_L \circ \boldsymbol{w}_R) + \boldsymbol{q}_O \circ \boldsymbol{w}_O + \boldsymbol{q}_C \right\rangle = 0.$$

Writing $\boldsymbol{w}_M = \boldsymbol{w}_L \circ \boldsymbol{w}_R$ for the virtual product vector, the prover sends the partial values $\overline{c}_L, \overline{c}_R, \overline{c}_M, \overline{c}_O, \overline{c}_C$, and the verifier checks that they sum to zero and that each equals its claimed projection:

$$\begin{aligned}
&\overline{c}_L + \overline{c}_R + \overline{c}_M + \overline{c}_O + \overline{c}_C = 0, \\
&\overline{c}_L = \langle \overline{\boldsymbol{\xi}}, \boldsymbol{q}_L \circ \boldsymbol{w}_L \rangle, \quad \overline{c}_R = \langle \overline{\boldsymbol{\xi}}, \boldsymbol{q}_R \circ \boldsymbol{w}_R \rangle, \quad \overline{c}_O = \langle \overline{\boldsymbol{\xi}}, \boldsymbol{q}_O \circ \boldsymbol{w}_O \rangle, \qquad (12) \\
&\overline{c}_M = \langle \overline{\boldsymbol{\xi}}, \boldsymbol{q}_M \circ \boldsymbol{w}_M \rangle, \quad \overline{c}_C = \langle \overline{\boldsymbol{\xi}}, \boldsymbol{q}_C \rangle. \qquad (13)
\end{aligned}$$

Each of the three claims in Eq. (12) has the form of Eq. (9), so Algorithm 2 reduces it to an evaluation of a selector vector and an evaluation of a committed wire vector. The claim on $\overline{c}_C$ is a public evaluation and needs no reduction. The claim on $\overline{c}_M$ requires one more step, since $\boldsymbol{w}_M$ is virtual and never committed: the same reduction leaves an evaluation of $\boldsymbol{q}_M$ and an evaluation of $\boldsymbol{w}_M$, and the latter is itself a weighted Hadamard claim on $\boldsymbol{w}_L$ and $\boldsymbol{w}_R$, which we discharge with a second invocation of Algorithm 2. Finally, we batch the resulting evaluation claims into a witness group and a public group, and open each group with the packed opening reduction of Section 4.4.

Theorem 1 (Single-instance packing). *Assume the parameters above, $m \geq \max\{\kappa, q, n_{\mathsf{io}}\}$, and the PCS assumptions of Proposition 4. The single-instance protocol is a SNARK protocol with knowledge-soundness error at most:*

$$\epsilon_{\mathsf{single}} = O\left(N/|\mathbb{E}|\right) + \epsilon_{\mathsf{pc}}, \qquad (14)$$

where ϵ_{pc} bounds the joint binding and opening error of the underlying PCS calls. For the gate reductions, take $t = 1$ when $q \geq 2\kappa - 1$; otherwise, take t as the least power of two at least $\lceil \log_q(2\kappa - 1)\rceil$. With circuit and interpolation preprocessing, the online efficiency bounds are:

$$\begin{aligned}
\textit{Prover time} \quad & O\left(N\kappa \log_q N \log_2 \kappa\right) \mathsf{bb} + O(\kappa^4) + \mathcal{TP}_{\mathsf{pcs}}(3m + m_{\mathsf{pub}}), \\
\textit{Prover memory} \quad & O(N \log_q N)\,\mathbb{F} + O\big(\kappa^4 + (t-1)q^{2t} + n_{\mathsf{io}}\big) + \mathcal{SP}_{\mathsf{pcs}}(3m + m_{\mathsf{pub}}), \\
\textit{Verifier time} \quad & O\left(\kappa \log_q N \log_2 N + (\log_2 N)^2 + n_{\mathsf{io}}\right) \mathsf{ee} + \mathcal{TV}_{\mathsf{pcs}}(3m + m_{\mathsf{pub}}), \\
\textit{Proof size} \quad & O\left(\kappa \log_q N + \kappa \log_2 \kappa + (\log_2 N)^2\right)\,\mathbb{E} + \mathcal{S}_{\mathsf{pcs}}(3m + m_{\mathsf{pub}}),
\end{aligned} \qquad (15)$$

where $\mathcal{TP}_{\mathsf{pcs}}$, $\mathcal{SP}_{\mathsf{pcs}}$, $\mathcal{TV}_{\mathsf{pcs}}$, and $\mathcal{S}_{\mathsf{pcs}}$ denote the PCS prover time, prover memory, verifier time, and proof size for total packed length $3m + m_{\mathsf{pub}}$, respectively.

The proof is deferred to Appendix C.6. Algorithm 5 gives the protocol.

5.3 Packing Multiple Circ-SAT Instances of the Same Circuit

For multiple executions of the same circuit, packing the R1CS formulation of Definition 2 turns out to be more efficient. The sparse matrices $\boldsymbol{S}_L, \boldsymbol{S}_R, \boldsymbol{S}_O \in \mathbb{F}^{N \times N}$ are shared across all instances, so the sparse-matrix–related prover cost can be amortized over the batch.

Let $\kappa \geq 2$ and $N \geq \max\{\kappa, n_{\mathsf{io}}\}$, and assume that each $\boldsymbol{S}_\star$ has $O(N)$ nonzero entries. For $q < 2\kappa - 1$, we use the small-field variant of the packed Hadamard-product proof in Section 4.2. Consider κ instances with assignments $\boldsymbol{w}^{(j)} \in \mathbb{F}^N$, $j \in [\kappa]$. Using the fixed basis $\overline{\boldsymbol{\beta}}$ of Section 4.4, define:

$$\boldsymbol{W} = \left[\boldsymbol{w}^{(1)} \mid \ldots \mid \boldsymbol{w}^{(\kappa)} \right] \in \mathbb{F}^{N \times \kappa}, \qquad \overline{\boldsymbol{w}} = \boldsymbol{W}\overline{\boldsymbol{\beta}} \in \mathbb{E}^N.$$

The prover commits to the packed $\overline{\boldsymbol{w}}$. All κ instances satisfy the circuit *iff*:

$$(\boldsymbol{S}_L \boldsymbol{w}^{(j)}) \circ (\boldsymbol{S}_R \boldsymbol{w}^{(j)}) = \boldsymbol{S}_O \boldsymbol{w}^{(j)}, \ \forall j \in [\kappa] \quad \iff \quad (\boldsymbol{S}_L \boldsymbol{W}) \circ (\boldsymbol{S}_R \boldsymbol{W}) = \boldsymbol{S}_O \boldsymbol{W}.$$

Packed constraint check. Once the commitment is fixed, the prover and verifier run the packed Hadamard-product proof on the $N \times \kappa$ matrix above. Applying Section 4.2 to the input matrices $\boldsymbol{S}_L\boldsymbol{W}$ and $\boldsymbol{S}_R\boldsymbol{W}$ reduces this check to weighted evaluations of the form:

$$\overline{v}_\star = \overline{\boldsymbol{\xi}}_\star^\top \, \boldsymbol{S}_\star \boldsymbol{W} \, \overline{\boldsymbol{\rho}}_\star, \qquad \star \in \{L, R, O\},$$

where $\overline{\boldsymbol{\xi}}_\star \in \mathbb{E}^N$ is tensor structured. The column weights $\overline{\boldsymbol{\rho}}_O$ are the initial MLE weights, while $\overline{\boldsymbol{\rho}}_L, \overline{\boldsymbol{\rho}}_R \in \mathbb{E}^\kappa$ are the Lagrange weights produced by Section 4.2. We describe how to prove these linear claims for one fixed $\star$.

Proving the linear relations. Following the column-projection step of Section 4.4, write $\overline{\boldsymbol{\xi}}_\star = \boldsymbol{\Xi}_\star \overline{\boldsymbol{\beta}}$, with $\boldsymbol{\Xi}_\star \in \mathbb{F}^{N \times \kappa}$. The prover sends the column evaluations $\overline{\boldsymbol{v}}_\star = \boldsymbol{U}_\star^\top \overline{\boldsymbol{\beta}}$, whose base-field coordinate matrix is claimed to satisfy:

$$\boldsymbol{U}_\star = \boldsymbol{\Xi}_\star^\top \boldsymbol{S}_\star \boldsymbol{W} \in \mathbb{F}^{\kappa \times \kappa}.$$

The verifier extracts $\boldsymbol{U}_\star$ and checks $\overline{v}_\star = \overline{\boldsymbol{v}}_\star^\top \overline{\boldsymbol{\rho}}_\star$. Because $\boldsymbol{S}_\star$ and $\boldsymbol{W}$ have base-field entries, injectivity of basis packing gives:

$$\boldsymbol{U}_\star = \boldsymbol{\Xi}_\star^\top \boldsymbol{S}_\star \boldsymbol{W} \quad \iff \quad \boldsymbol{U}_\star \overline{\boldsymbol{\beta}} = \boldsymbol{\Xi}_\star^\top \boldsymbol{S}_\star \overline{\boldsymbol{w}}.$$

Thus the claim-dependent column weights have been replaced by the fixed basis.

After all three matrices $\boldsymbol{U}_\star$ are fixed, the verifier samples independent $\overline{z} \xleftarrow{\$} \mathbb{E}$ and $\overline{\boldsymbol{r}} \xleftarrow{\$} \mathbb{E}^\kappa$. The challenge $\overline{z}$ combines these three linear identities:

$$(\boldsymbol{U}_O + \overline{z} \cdot \boldsymbol{U}_L + \overline{z}^2 \cdot \boldsymbol{U}_R)\overline{\boldsymbol{\beta}} = (\boldsymbol{\Xi}_O^\top \boldsymbol{S}_O + \overline{z} \cdot \boldsymbol{\Xi}_L^\top \boldsymbol{S}_L + \overline{z}^2 \cdot \boldsymbol{\Xi}_R^\top \boldsymbol{S}_R)\overline{\boldsymbol{w}}.$$

Projecting by $\overline{\boldsymbol{r}}$, the prover proves:

$$\overline{\boldsymbol{r}}^\top(\boldsymbol{U}_O + \overline{z}\cdot\boldsymbol{U}_L + \overline{z}^2\cdot\boldsymbol{U}_R)\overline{\boldsymbol{\beta}} = \left\langle \overline{\boldsymbol{w}}, (\boldsymbol{S}_O^\top\boldsymbol{\Xi}_O + \overline{z}\cdot\boldsymbol{S}_L^\top\boldsymbol{\Xi}_L + \overline{z}^2\cdot\boldsymbol{S}_R^\top\boldsymbol{\Xi}_R)\overline{\boldsymbol{r}}\right\rangle.$$

Fact 1 reduces the scalar identity to an MLE opening of $\overline{\boldsymbol{w}}$ at a random point $\overline{\boldsymbol{y}}$ and a public terminal evaluation. At $\overline{\boldsymbol{y}}$, the prover supplies the three MLE evaluations of the public weight vectors and the verifier checks their $(1, \overline{z}, \overline{z}^2)$-weighted sum. For each $\star \in \{O, L, R\}$, the corresponding value is:

$$\mathsf{ts}(\overline{\boldsymbol{y}})^\top\boldsymbol{S}_\star^\top\boldsymbol{\Xi}_\star\overline{\boldsymbol{r}} = \langle \boldsymbol{S}_\star\mathsf{ts}(\overline{\boldsymbol{y}}), \boldsymbol{\Xi}_\star\overline{\boldsymbol{r}}\rangle.$$

One further inner-product proof leaves two claims at its terminal point $\overline{\boldsymbol{y}}'$:

$$\mathsf{ts}(\overline{\boldsymbol{y}}')^\top\boldsymbol{S}_\star\mathsf{ts}(\overline{\boldsymbol{y}}), \qquad \mathsf{ts}(\overline{\boldsymbol{y}}')^\top\boldsymbol{\Xi}_\star\overline{\boldsymbol{r}}.$$

The first is a sparse matrix MLE claim, proved by the sparse-polynomial evaluation protocol of Spartan against the preprocessed circuit commitment [58]. The second is evaluated using the tensor recursion in Appendix B.2. The sparse matrix-evaluation proofs for $\boldsymbol{S}_L, \boldsymbol{S}_R, \boldsymbol{S}_O$ are performed once for all packed instances. Their prover cost is therefore amortized over the κ instances.

Larger batches. For $\nu\kappa$ instances, form ν packed assignments $\overline{\boldsymbol{w}}^{(1)}, \ldots, \overline{\boldsymbol{w}}^{(\nu)}$. Each block undergoes the packed constraint check above. For terminal claims sharing $\boldsymbol{\Xi}$ and $\boldsymbol{S}_\star$, fix all claimed matrices $\boldsymbol{U}^{(j)}$, then sample independent $\overline{z}_1, \ldots, \overline{z}_\nu \xleftarrow{\$} \mathbb{E}$. Their lincheck identities combine as:

$$\left(\textstyle\sum_{\ell=1}^{\nu}\overline{z}_\ell\boldsymbol{U}^{(\ell)}\right)\overline{\boldsymbol{\beta}} = \boldsymbol{\Xi}^\top\boldsymbol{S}_\star\left(\textstyle\sum_{\ell=1}^{\nu}\overline{z}_\ell\overline{\boldsymbol{w}}^{(\ell)}\right).$$

This equation can be proved with $O(\nu N)$ extension-field operations, and the proof reduces to MLE claims on $\boldsymbol{S}_L$, $\boldsymbol{S}_R$, $\boldsymbol{S}_O$, and $\{\overline{\boldsymbol{w}}^{(\ell)}\}_{\ell=1}^{\nu}$.

Theorem 2 (Multiple-instance packing). *Under the parameters above and the PCS assumptions of Proposition 4, the protocol for κ instances of the same R1CS is a SNARK protocol with knowledge-soundness error at most:*

$$\epsilon_{\mathsf{multi}} = O\left(N/|\mathbb{E}|\right) + \epsilon_{\mathsf{pc}}, \tag{16}$$

where ϵ_{pc} bounds the joint binding and opening error of the witness and sparse-evaluation PCS calls. Let $t \mid \kappa$ be the Hadamard interpolation-field degree, with $q^t \geq 2\kappa - 1$ and $t = 1$ when $q \geq 2\kappa - 1$. With circuit and interpolation preprocessing, the total online costs for the batch are:

$$\begin{aligned}
\textit{Prover time} &\quad O(N\kappa^2)\,\mathsf{bb} + \mathcal{TP}_{\mathsf{pcs}}(N) + \mathcal{TP}_{\mathsf{sprs}},\\
\textit{Prover memory} &\quad O\big(N\kappa + \kappa^2 t + (t-1)q^{2t}\big)\,\mathbb{F} + \mathcal{SP}_{\mathsf{pcs}}(N) + \mathcal{SP}_{\mathsf{sprs}},\\
\textit{Verifier time} &\quad O\left(\kappa\log_2 N + n_{\mathsf{io}}\right)\,\mathsf{ee} + \mathcal{TV}_{\mathsf{pcs}}(N) + \mathcal{TV}_{\mathsf{sprs}},\\
\textit{Proof size} &\quad O\left(\kappa + (\log_2 N)^2\right)\,\mathbb{E} + \mathcal{S}_{\mathsf{pcs}}(N) + \mathcal{S}_{\mathsf{sprs}},
\end{aligned} \tag{17}$$

where $\mathcal{TP}, \mathcal{SP}, \mathcal{TV}, \mathcal{S}$ denote prover time, prover memory, verifier time, and proof size, respectively. The subscript pcs *denotes the aggregate commitment and*

Table 3. Binary Hadamard-product proofs at $N = 2^{30}$.

Metric	SFF (Binius)	Flock	`Mikros`	Flock/`Mikros`
Prover time (ms)	106,331.53	995.23	684.48	*1.45*×
Verifier time (μs)	39.00	396.00	34.00	*11.65*×
Proof size (B)	1,504.00	4,912.00	4,888.00	*1.00*×
Prover peak RSS (GiB)	12.38	0.76	0.89	*0.86*×

All systems use $\kappa = 128$ and run on a single thread. SFF (Binius) denotes the small-field-first zerocheck in Binius. The first two systems skip $t = 7$ sumcheck rounds.

opening costs for the length-N packed witness, and sprs *denotes the total sparse-MLE evaluation costs for $\boldsymbol{S}_L, \boldsymbol{S}_R, \boldsymbol{S}_O$, including their underlying PCS calls.*

*With linear-time, linear-space PCS provers, the sparse-evaluation costs of Spartan are $O(N)$*ee *time and $O(N)\mathbb{E}$ space. The amortized SNARK prover costs are therefore $O(N\kappa)$*bb *time and $O\big(N + \kappa t + (t-1)q^{2t}/\kappa\big)\mathbb{F}$ memory per instance, including the precomputed tables.*

The proof is given in Appendix C.7; Algorithm 6 specifies the protocol.

6 Experimental Evaluation

We evaluate `Mikros` on binary Hadamard-product relations and on batches of hash computations, comparing it against Flock [17], a state-of-the-art bitwidth-aware SNARK. Our Rust implementation and experiment logs are available online.[1] All benchmarks run on a dual-socket server with two Intel Xeon Gold 6342 processors (24 cores each, nominally 2.80 GHz), 384 GiB of RAM, and Ubuntu 22.04.5. For both systems, we employ native CPU optimizations for binary-field arithmetic. GFNI provides $\mathbb{F}_{2^8}$ multiplication and affine transforms, VPCLMULQDQ provides carry-less multiplication for extension-field elements, and AVX-512 vectorizes the bitwise operations, transposes, and pack/unpack steps. For a fair comparison, Flock and `Mikros` share the same Ligerito PCS backend.

Binary Hadamard-product proofs. Table 3 compares our Hadamard-product proof against the degree-2 zerochecks of Flock and of the small-field-first (SFF) method implemented in Binius [27], all on the same $N = 2^{30}$ binary Hadamard product. These measurements cover the PIOP reduction and exclude commitments and PCS openings. `Mikros` proves the relation in 684.48 ms, compared with 995.23 ms for Flock and 106.33 s for Binius, giving prover speedups of 1.45× and 155.35×, respectively. Verifier time is 11.65× faster than Flock due to the avoidance of inversion. Prover peak RSS is slightly higher, 0.89 GiB for `Mikros` versus 0.76 GiB for Flock.

[1] `git@github.com:mirandaprivate/mikros.git`

Table 4. End-to-end arguments for batches of 256 independent hash computations.

	SHA-256			Keccak			BLAKE3		
	Flock	`Mikros`	×	Flock	`Mikros`	×	Flock	`Mikros`	×
Prover time (ms)	174.27	79.54	*2.19*×	255.27	100.15	*2.55*×	103.96	56.13	*1.85*×
Verifier time (ms)	37.47	38.34	*0.98*×	7.62	19.30	*0.39*×	35.14	35.74	*0.98*×
Proof size (KiB)	306.59	367.71	*0.83*×	289.85	380.31	*0.76*×	279.86	355.64	*0.79*×
Peak RSS (MiB)	606.79	606.79	*1.00*×	55.53	55.53	*1.00*×	633.00	632.25	*1.00*×

SHA-256 and BLAKE3 measure compression; Keccak measures its 24-round permutation. Both systems use the same Ligerito Fast PCS and run on a single thread.

Table 5. Single-instance arguments for a binary circuit with $N = 2^{25}$ gates.

	Flock	`Mikros`	`Mikros` excl. perm.	
Prover time (s)	10.12	86.56	6.72	*1.51*×
Verifier time (ms)	4,316.57	24.43	21.78	*198.23*×
Proof size (KiB)	426.49	1,169.23	627.91	*0.68*×
Peak RSS (GiB)	12.07	6.65	—	—

Both run on a single thread. Flock uses public constraint matrices. `Mikros` excl. perm. is `Mikros` with the cost of the permutation argument excluded.

End-to-end packed SNARK for hash computations. We next measure online prover costs for batches of 256 independent SHA-256 compressions, Keccak-f[1600] permutations, and BLAKE3 compressions, containing $\sim 2^{15}$, $\sim 2^{16}$, and $\sim 2^{14}$ gates per instance, respectively. Flock does not commit to the wiring matrices, so in this batched setting both systems take them as explicit public inputs and have the verifier evaluate them directly, making the comparison fair.

Table 4 reports full-batch costs. `Mikros` proves the SHA-256, Keccak, and BLAKE3 batches in 79.54, 100.15, and 56.13 ms, respectively, which are 2.19×, 2.55×, and 1.85× faster than Flock, respectively. Whole-process peak RSS is comparable between the two systems on all three workloads. `Mikros` produces larger proofs and has higher verification times on all three workloads, as reported in Table 4.

End-to-end SNARK for a single Circ-SAT instance. For a single instance we evaluate a simulated binary circuit with 2^{25} gates rather than a hash computation, because a single hash packs to fewer than 2^{10} extension-field elements, too small for the backend Ligerito PCS to process. As Table 5 reports, `Mikros` takes 8.55× the prover time of Flock, but the comparison is not fair: proving the wiring matrix—the permutation argument of Section 4.3—is by far the most expensive part of `Mikros`, and Flock omits it by having the verifier evaluate the MLE of the R1CS matrices directly. Flock thus shifts this cost to the verifier, which is why its single-instance verifier time is higher. Excluding the permutation argument, `Mikros` is 1.51× faster in prover time.

7 Conclusion

We introduced `Mikros`, a SNARK framework built on *algebraic vector packing*, which packs base-field vectors into extension-field vectors and decouples packing from the extension-field protocol that proves the packed relation. The technique applies to a range of relations and to arbitrary small fields rather than binary fields alone, yielding packed inner-product and Hadamard-product proofs, packed polynomial commitment openings, and the first bitwidth-aware grand-product proof, and with it a bitwidth-aware permutation argument. Combining these components gives an end-to-end SNARK for circuit satisfiability whose prover time and memory scale with the effective bitwidth; across instances of the same circuit the wiring encoding amortizes, leaving a per-instance cost proportional to the base-field bitwidth alone. Despite exploiting no binary-field structure, `Mikros` outperforms the state-of-the-art binary-field SNARK on batched hash proving over binary-extension fields.

Acknowledgement

The authors used generative AI tools to assist with coding, language editing, and organizing the exposition. All technical content, including the protocols, proofs, and experimental results, was developed and verified by the authors, who take full responsibility for the paper.

References

1. Abbaszadeh, K., Pappas, C., Katz, J., Papadopoulos, D.: Zero-knowledge proofs of training for deep neural networks. In: CCS. pp. 4316–4330 (2024)
2. Ambrona, M., Schmitt, A.L., Toledo, R.R., Willems, D.: New optimization techniques for PlonK's arithmetization. IACR Cryptology ePrint Archive, Paper 2022/462 (2022)
3. Aranha, D.F., Costache, A., Guimarães, A., Soria-Vazquez, E.: HELIOPOLIS: Verifiable computation over homomorphically encrypted data from interactive oracle proofs is practical. In: ASIACRYPT 2024, Part V. pp. 302–334 (2025)
4. Attema, T., Cramer, R., Kohl, L.: A compressed Σ-protocol theory for lattices. In: CRYPTO Part II. pp. 549–579 (2021)
5. Bagad, S., Dao, Q., Domb, Y., Thaler, J.: Speeding up sum-check proving. IACR Cryptology ePrint Archive, Paper 2025/1117 (2025)
6. Bagad, S., Domb, Y., Thaler, J.: The sum-check protocol over fields of small characteristic. IACR Cryptology ePrint Archive, Paper 2024/1046 (2024)
7. Ben-Sasson, E., Bentov, I., Horesh, Y., Riabzev, M.: Fast Reed-Solomon interactive oracle proofs of proximity. In: ICALP. pp. 14:1–14:17 (2018)
8. Ben-Sasson, E., Chiesa, A., Garman, C., Green, M., Miers, I., Tromer, E., Virza, M.: Zerocash: Decentralized anonymous payments from Bitcoin. In: SP. pp. 459–474 (2014)
9. Ben-Sasson, E., Chiesa, A., Genkin, D., Tromer, E., Virza, M.: SNARKs for C: verifying program executions succinctly and in zero knowledge. In: CRYPTO. pp. 90–108 (2013)
10. Ben-Sasson, E., Chiesa, A., Spooner, N.: Interactive oracle proofs. In: TCC (B2). pp. 31–60 (2016)
11. Ben-Sasson, E., Goldberg, L., Kopparty, S., Saraf, S.: DEEP-FRI: Sampling outside the box improves soundness. In: ITCS. LIPIcs, vol. 151, pp. 5:1–5:32 (2020)
12. Bois, A., Cascudo, I., Fiore, D., Kim, D.: Flexible and efficient verifiable computation on encrypted data. In: PKC Part II. pp. 528–558 (2021)
13. Bootle, J., Chiesa, A., Groth, J.: Linear-time arguments with sublinear verification from tensor codes. In: TCC Part II. pp. 19–46 (2020)
14. Bootle, J., Chiesa, A., Liu, S.: Zero-knowledge IOPs with linear-time prover and polylogarithmic-time verifier. In: EUROCRYPT Part II. pp. 275–304 (2022)
15. Brehm, M., Chen, B., Fisch, B., Resch, N., Rothblum, R.D., Zeilberger, H.: Blaze: Fast SNARKs from interleaved RAA codes. In: EUROCRYPT. pp. 123–152 (2025)
16. Bünz, B., Fisch, B., Szepieniec, A.: Transparent SNARKs from DARK compilers. In: EUROCRYPT Part I. pp. 677–706 (2020)
17. Bünz, B., Rothblum, R.D., Wang, W.: Flock: Fast proving for batch boolean computations. IACR Cryptology ePrint Archive, Paper 2026/1329 (2026)

18. Canetti, R., Chen, Y., Holmgren, J., Lombardi, A., Rothblum, G.N., Rothblum, R.D., Wichs, D.: Fiat-shamir: From practice to theory. In: STOC. pp. 1082–1090 (2019), combines the preprints in Cryptology ePrint Archive, Papers 2018/1004 and 2018/1248
19. Cascudo, I., Costache, A., Cozzo, D., Fiore, D., Guimarães, A., Soria-Vazquez, E.: Verifiable computation for approximate homomorphic encryption schemes. In: CRYPTO Part VII. pp. 643–677 (2025)
20. Cascudo, I., Giunta, E.: On interactive oracle proofs for boolean R1CS statements. In: Financial Cryptography and Data Security. Lecture Notes in Computer Science, vol. 13411, pp. 230–247. Springer (2022), full version: Cryptology ePrint Archive, Paper 2021/694
21. Chen, B., Bünz, B., Boneh, D., Zhang, Z.: HyperPlonk: Plonk with linear-time prover and high-degree custom gates. In: EUROCRYPT Part II. pp. 499–530 (2023)
22. Chiesa, A., Hu, Y., Maller, M., Mishra, P., Vesely, P., Ward, N.P.: Marlin: Preprocessing zkSNARKs with universal and updatable SRS. In: EUROCRYPT Part I. pp. 738–768 (2020)
23. Chiesa, A., Ojha, D., Spooner, N.: Fractal: Post-quantum and transparent recursive proofs from holography. In: EUROCRYPT Part I. pp. 769–793 (2020)
24. Cong, M., Chow, S.S.M., Yiu, S.M., Yuen, T.H.: Scalable zkSNARKs for matrix computations: A generic framework for verifiable deep learning. In: ASIACRYPT 2025, Part V. vol. 16249, pp. 363–395 (2026)
25. Dao, Q., DeStefano, Z., Bagad, S., Domb, Y., Thaler, J.: Speeding up sum-check proving (extended version). Cryptology ePrint Archive, Paper 2026/587 (2026), to appear in ACM CCS 2026
26. Dao, Q., Thaler, J.: Constraint-packing and the sum-check protocol over binary tower fields. IACR Cryptology ePrint Archive, Paper 2024/1038 (2024)
27. Diamond, B.E., Posen, J.: Succinct arguments over towers of binary fields. In: EUROCRYPT (4). vol. 15604, pp. 93–122 (2025)
28. Diamond, B.E., Posen, J.: Polylogarithmic proofs for multilinears over binary towers. In: EUROCRYPT (7). Lecture Notes in Computer Science, vol. 16547, pp. 3–32. Springer (2026)
29. Fiat, A., Shamir, A.: How to prove yourself: Practical solutions to identification and signature problems. In: CRYPTO '86. pp. 186–194 (1987)
30. Fiore, D., Gennaro, R., Pastro, V.: Efficiently verifiable computation on encrypted data. In: CCS. pp. 844–855 (2014)
31. Fiore, D., Nitulescu, A., Pointcheval, D.: Boosting verifiable computation on encrypted data. In: PKC Part II. pp. 124–154 (2020)
32. Gabizon, A., Williamson, Z.J., Ciobotaru, O.: PLONK: Permutations over lagrange-bases for oecumenical noninteractive arguments of knowledge. IACR Cryptol. ePrint Arch. 2019/953 (2019)
33. Garg, S., Goel, A., Jha, S., Mahloujifar, S., Mahmoody, M., Policharla, G., Wang, M.: Experimenting with zero-knowledge proofs of training. In: CCS. pp. 1880–1894 (2023)
34. Garg, S., Goel, A., Wang, M.: How to prove statements obliviously? In: CRYPTO Part X. pp. 449–487 (2024)
35. Gennaro, R., Gentry, C., Parno, B., Raykova, M.: Quadratic span programs and succinct nizks without pcps. In: EUROCRYPT. pp. 626–645 (2013)
36. Goldwasser, S., Kalai, Y.T., Rothblum, G.N.: Delegating computation: Interactive proofs for muggles. J. ACM **62**(4), 27:1–27:64 (2015)

37. Golovnev, A., Lee, J., Setty, S.T.V., Thaler, J., Wahby, R.S.: Brakedown: Linear-time and field-agnostic SNARKs for R1CS. In: CRYPTO Part II. pp. 193–226 (2023)
38. Groth, J.: On the size of pairing-based non-interactive arguments. In: EUROCRYPT Part II. pp. 305–326 (2016)
39. Gruen, A.: Some improvements for the PIOP for ZeroCheck. IACR Cryptology ePrint Archive, Paper 2024/108 (2024)
40. Guo, Y., Liu, X., Huang, K., Qu, W., Tao, T., Zhang, J.: DeepFold: Efficient multilinear polynomial commitment from Reed–Solomon code and its application to zero-knowledge proofs. In: USENIX Security. pp. 3497–3516 (2025)
41. Haböck, U., Levit, D., Papini, S.: Circle STARKs. IACR Cryptology ePrint Archive, Paper 2024/278 (2024)
42. Halevi, S., Polyakov, Y., Shoup, V.: An improved RNS variant of the BFV homomorphic encryption scheme. In: CT-RSA. pp. 83–105 (2019)
43. Jacob, B., Kligys, S., Chen, B., Zhu, M., Tang, M., Howard, A.G., Adam, H., Kalenichenko, D.: Quantization and training of neural networks for efficient integer-arithmetic-only inference. In: CVPR. pp. 2704–2713 (2018)
44. Kate, A., Zaverucha, G.M., Goldberg, I.: Constant-size commitments to polynomials and their applications. In: ASIACRYPT. pp. 177–194 (2010)
45. Kosba, A., Miller, A., Shi, E., Wen, Z., Papamanthou, C.: Hawk: The blockchain model of cryptography and privacy-preserving smart contracts. In: SP. pp. 839–858 (2016)
46. Lee, J.: Dory: Efficient, transparent arguments for generalised inner products and polynomial commitments. In: TCC Part II. pp. 1–34 (2021)
47. Liu, T., Xie, X., Zhang, Y.: zkCNN: Zero knowledge proofs for convolutional neural network predictions and accuracy. In: CCS. pp. 2968–2985 (2021)
48. Liu, T., Zhang, Y.: Efficient SNARKs for boolean circuits via sumcheck over tower fields. IACR Cryptology ePrint Archive, Paper 2025/594 (2025)
49. Lund, C., Fortnow, L., Karloff, H.J., Nisan, N.: Algebraic methods for interactive proof systems. J. ACM **39**(4), 859–868 (1992)
50. Papamanthou, C., Shi, E., Tamassia, R.: Signatures of correct computation. In: TCC. pp. 222–242 (2013)
51. Parno, B., Howell, J., Gentry, C., Raykova, M.: Pinocchio: Nearly practical verifiable computation. Commun. ACM **59**(2), 103–112 (2016)
52. Peshawaria, K., Liu, Z., Fisch, B., Tromer, E.: Laminate: Succinct SIMD-friendly verifiable FHE. IACR Cryptology ePrint Archive, Paper 2025/2285 (2025)
53. Polygon Zero Team: Plonky2: Fast recursive arguments with PLONK and FRI. Technical report (2022), draft, September 7, 2022
54. Qu, W., Guo, Y., Ying, Y., Zhang, J.: VerfCNN, optimal complexity zkSNARK for convolutional neural networks. In: S&P (2026)
55. Qu, W., Sun, Y., Liu, X., Lu, T., Guo, Y., Chen, K., Zhang, J.: zkGPT: An efficient non-interactive zero-knowledge proof framework for LLM inference. In: USENIX Security. pp. 2045–2063 (2025)
56. Riasi, A., Wang, H., Behnia, R., Vo, V., Hoang, T.: Zero-knowledge AI inference with high precision. In: CCS. pp. 1053–1067 (2025)
57. Setty, S., Thaler, J.: BabySpartan: Lasso-based SNARK for non-uniform computation. IACR Cryptology ePrint Archive, Paper 2023/1799 (2023)
58. Setty, S.T.V.: Spartan: Efficient and general-purpose zkSNARKs without trusted setup. In: CRYPTO Part III. pp. 704–737 (2020)
59. Setty, S.T.V., Lee, J.: Quarks: Quadruple-efficient transparent zkSNARKs. IACR Cryptol. ePrint Arch. 2020/1275 (2020)

60. Setty, S.T.V., Thaler, J., Wahby, R.S.: Unlocking the lookup singularity with Lasso. In: EUROCRYPT Part VI. pp. 180–209 (2024)
61. Sun, H., Li, J., Zhang, H.: zkLLM: Zero knowledge proofs for large language models. In: CCS. pp. 4405–4419 (2024)
62. Thaler, J.: Time-optimal interactive proofs for circuit evaluation. In: CRYPTO Part II. pp. 71–89 (2013)
63. Tremblay Thibault, L., Walter, M.: Towards verifiable FHE in practice: Proving correct execution of TFHE's bootstrapping using plonky2. In: CCS. pp. 1113–1126 (2025)
64. Wahby, R.S., Tzialla, I., Shelat, A., Thaler, J., Walfish, M.: Doubly-efficient zkSNARKs without trusted setup. In: SP. pp. 926–943 (2018)
65. Wei, Y., Wang, K., Xiang, B., Zhang, X., Deng, Y., Zhu, X., Wang, H., Lin, L., Wang, L.: SNARKs over small prime fields without extension field multiplication. Cryptology ePrint Archive, Paper 2025/719 (2025), to appear in ASIACRYPT 2026
66. Xie, T., Zhang, J., Cheng, Z., Zhang, F., Zhang, Y., Jia, Y., Boneh, D., Song, D.: zkBridge: Trustless cross-chain bridges made practical. In: CCS. pp. 3003–3017 (2022)
67. Xie, T., Zhang, J., Zhang, Y., Papamanthou, C., Song, D.: Libra: Succinct zero-knowledge proofs with optimal prover computation. In: CRYPTO. pp. 733–764 (2019)
68. Zeilberger, H., Chen, B., Fisch, B.: Basefold: Efficient field-agnostic polynomial commitment schemes from foldable codes. In: CRYPTO. pp. 138–169 (2024)
69. Zhang, X., Wang, R., Liu, Z., Xiang, B., Deng, Y., Fisch, B., Lu, X.: Phalanx: An FHE-friendly SNARK for verifiable computation on encrypted data. In: CCS. pp. 4035–4048 (2025)

A Security Definitions

A.1 Completeness and Soundness

Definition 8 (Proof/Argument/Argument of Knowledge). *A protocol consists of algorithms* $(\mathsf{Setup}, \mathcal{P}, \mathcal{V})$*. Proofs, arguments, and arguments of knowledge require soundness, computational soundness, and knowledge soundness, respectively. Throughout, we require perfect completeness.*

- Setup*: on input* 1^λ*, output public parameters* pp*.*
- $\mathcal{P}$*: on input* pp *and a statement* u*, produce a transcript* π *for membership in a language* $\mathcal{L}$*. For a relation* $\mathcal{R}$*, the prover additionally receives a witness* w*.*
- $\mathcal{V}$*: on input* pp*, a statement* u*, and a transcript* π*, output* true *or* false*.*

Let $(\mathcal{P} \rightleftharpoons \mathcal{V})$ *denote the interaction between* $\mathcal{P}$ *and* $\mathcal{V}$*.*

Definition 9 (Perfect Completeness). *The protocol* $(\mathsf{Setup}, \mathcal{P}, \mathcal{V})$ *for a relation* $\mathcal{R}$ *has perfect completeness if, for every* $(u, w) \in \mathcal{R}$*:*

$$\Pr\left[\mathcal{V}(\mathsf{pp}, u, \pi) = \mathsf{true} \;\middle|\; \mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda),\ \pi \leftarrow \mathcal{P}(\mathsf{pp}, u, w)\right] = 1,$$

where the probability is over the randomness of Setup*,* $\mathcal{P}$*, and* $\mathcal{V}$*.*

Definition 10 (Soundness [18]). *A protocol* $(\mathsf{Setup}, \mathcal{P}, \mathcal{V})$ *for a language* $\mathcal{L}$ *or relation* $\mathcal{R}$ *has soundness error* $\epsilon^{\mathsf{s}}(u)\colon \mathbb{N} \mapsto [0,1]$ *if every computationally unbounded interactive prover* $\mathcal{P}^*$ *satisfies:*

$$\Pr\left[\mathcal{V}(\mathsf{pp}, u, \pi) = \mathsf{true} \;\middle|\; u \notin \mathcal{L} \;/\; \mathcal{R};\ \pi \leftarrow (\mathcal{P}^* \rightleftharpoons \mathcal{V})(\mathsf{pp}, u)\right] \leq \epsilon^{\mathsf{s}}(u).$$

The protocol is sound if $\epsilon^{\mathsf{s}}(u)$ *is negligible.*

Restricting the adversarial prover to probabilistic polynomial time gives computational soundness.

Definition 11 (Computational Soundness). *A protocol* $(\mathsf{Setup}, \mathcal{P}, \mathcal{V})$ *for a language* $\mathcal{L}$ *or relation* $\mathcal{R}$ *has computational soundness error* $\epsilon^{\mathsf{cs}}(u)\colon \mathbb{N} \mapsto [0,1]$ *if every probabilistic polynomial-time (PPT) prover* $\mathcal{P}^*$ *satisfies:*

$$\Pr\left[\mathcal{V}(\mathsf{pp}, u, \pi) = \mathsf{true} \;\middle|\; u \notin \mathcal{L} \;/\; \mathcal{R};\ \pi \leftarrow (\mathcal{P}^* \rightleftharpoons \mathcal{V})(\mathsf{pp}, u)\right] \leq \epsilon^{\mathsf{cs}}(u).$$

The protocol is computationally sound if $\epsilon^{\mathsf{cs}}(u)$ *is negligible.*

By definition, $\epsilon^{\mathsf{cs}}(u) \leq \epsilon^{\mathsf{s}}(u)$.

An argument of knowledge additionally requires extraction of a witness [4]. We use the following normalization of the running time and success probability of the extractor.

Definition 12 (Knowledge Soundness). *A protocol* $(\mathsf{Setup}, \mathcal{P}, \mathcal{V})$ *for a relation* $\mathcal{R}$ *has knowledge-soundness error* $\epsilon^{\mathsf{ks}}(u)\colon \mathbb{N} \mapsto [0,1]$ *if there exists a knowledge extractor* $\mathcal{E}$ *satisfying the following guarantee. Given* u *and black-box access to a prover* $\mathcal{P}^*$*, the extractor runs in expected time polynomial in* $|u|$*, counting each oracle access as one step, and satisfies:*

$$\Pr\left[(u, \mathcal{E}(\mathcal{P}^*, u)) \in \mathcal{R}\right] \geq \epsilon(\mathcal{P}^*, u) - \epsilon^{\mathsf{ks}}(u),$$

where $\epsilon(\mathcal{P}^*, u)$ *is the acceptance probability of* $\mathcal{P}^*$*. The protocol is knowledge sound if* $\epsilon^{\mathsf{ks}}(u)$ *is negligible.*

Knowledge soundness implies computational soundness with error $\epsilon^{\mathsf{cs}}(u) \leq \epsilon^{\mathsf{ks}}(u)$. Otherwise, some false statement u and PPT prover $\mathcal{P}^*$ would satisfy $\epsilon(\mathcal{P}^*, u) > \epsilon^{\mathsf{ks}}(u)$. The extraction guarantee would then yield a valid witness with positive probability, contradicting the falsity of u.

Lemma 3 (Sound reductions with a knowledge-sound PCS). *Let* Π *be a reduction proof protocol for the relation:*

$$\mathcal{R}_\Pi = \{\ \mathsf{public\ input}\ :\ \boldsymbol{w}\ \mid \mathsf{constraint}\ \}.$$

All witness data are collected in $\boldsymbol{w}$ *and committed through an efficiently decodable polynomial encoding* $p_{\boldsymbol{w}}$ *before* Π *begins. Assume that* Π *is sound relative to these fixed witness data: for every* $(u, \boldsymbol{w}) \notin \mathcal{R}_\Pi$*, the probability that its checks pass and all terminal evaluation claims about* $p_{\boldsymbol{w}}$ *are true is at most* $\epsilon^{\mathsf{s}}_\Pi$*. Let* PCS *be knowledge sound, with joint error* $\epsilon^{\mathsf{ks}}_{\mathsf{pc}}$ *for extracting the polynomial fixed*

by the initial commitment consistently with all accepted openings, including at adaptively chosen evaluation points. Then the protocol Π' that runs Π and verifies all terminal evaluation claims using PCS *is knowledge sound for $\mathcal{R}_\Pi$ with knowledge-soundness error at most $\epsilon^{\mathsf{s}}_{\Pi} + \epsilon^{\mathsf{ks}}_{\mathsf{pc}}$.*

Proof. Apply the PCS extractor and decode the committed polynomial to obtain $\boldsymbol{w}$. Except with probability $\epsilon^{\mathsf{ks}}_{\mathsf{pc}}$, all accepted terminal evaluations are consistent with this fixed witness. If $(u, \boldsymbol{w}) \notin \mathcal{R}_\Pi$, soundness of the reduction bounds acceptance by $\epsilon^{\mathsf{s}}_{\Pi}$. A union bound gives the claimed knowledge-soundness error. □

A.2 Round-by-Round Soundness

Definition 13 (Round-by-Round Soundness [18]). *Let $\Pi = (\mathsf{Setup}, \mathcal{P}, \mathcal{V})$ be a $(2\varrho + 1)$-message public-coin interactive proof for a relation $\mathcal{R}$ with language $\mathcal{L}$. Fix the public parameters* pp*, which we suppress below. We record states after prover messages. For a statement $u \in \{0,1\}^*$, let $\pi_0 = (\alpha_0)$ and $\pi_j = (\pi_{j-1} \| x_j \| \alpha_j)$ for $j \in [\varrho]$. Thus π_j contains the first $2j+1$ messages. Write $\mathsf{H}(u, \pi_{j-1})$ for the distribution of the verifier's challenge x_j. The protocol has round-by-round soundness error $\{\epsilon^{\mathsf{rbr}}_j\}_{j \in [\varrho]}$ if there exist deterministic, not necessarily efficient, functions $\{\mathsf{State}_j\}_{j \in [0,\varrho]}$ mapping a statement and transcript prefix to* accept *or* reject*, with the following properties:*

- *If $u \notin \mathcal{L}$, then for every first prover message α_0, $\mathsf{State}_0(u, (\alpha_0)) = \mathsf{reject}$.*
- *For every $j \in [\varrho]$ and prefix π_{j-1} with $\mathsf{State}_{j-1}(u, \pi_{j-1}) = \mathsf{reject}$:*

$$\Pr_{x_j \leftarrow \mathsf{H}(u, \pi_{j-1})} [\exists \alpha_j : \mathsf{State}_j(u, \pi_{j-1} \| x_j \| \alpha_j) = \mathsf{accept}] \leq \epsilon^{\mathsf{rbr}}_j.$$

  The existential quantifier allows the prover to choose its response after seeing x_j.
- *For every full transcript π_ϱ, $\mathsf{State}_\varrho(u, \pi_\varrho) = \mathsf{reject}$ implies $\mathcal{V}(u, \pi_\varrho) = \mathsf{false}$.*

On a false statement, an accepting transcript starts in a rejecting state and ends in an accepting state. It therefore contains a reject-to-accept transition. A union bound over the ϱ challenges gives the following bound.

Lemma 4 (Round-by-Round Soundness Implies Soundness [18]). *Let Π be a $(2\varrho+1)$-message interactive proof with round-by-round soundness errors $\epsilon^{\mathsf{rbr}}_j$ for $j \in [\varrho]$. Then Π has soundness error at most $\sum_{j=1}^{\varrho} \epsilon^{\mathsf{rbr}}_j$.*

B Deferred Technical Details

B.1 Pseudocode

We give the protocols in their order of composition. Algorithms 1, 2, and 3 reduce the vector relations of Section 4 to terminal packed claims. Algorithm 4 proves these claims using the PCS, and Algorithms 5 and 6 combine the reductions for the two Circ-SAT settings of Section 5.

Shared interpolation data. Algorithms 2 and 3 use the interpolation procedure of Section 4.2. Set $t = 1$ and $\mathbb{K} = \mathbb{F}$ when $q \geq 2\kappa - 1$; otherwise, choose $\mathbb{K} = \mathbb{F}_{q^t} \subseteq \mathbb{E}$, where $t \mid \kappa$ and $q^t \geq 2\kappa - 1$. Precompute the Lagrange evaluations, vanishing polynomial, and interpolation data on $2\kappa - 1$ distinct nodes in $\mathbb{K}$, together with the field-operation tables when $t > 1$. Mixed $\mathbb{K} \times \mathbb{E}$ products use these tables element-wise in a fixed $\mathbb{K}$-basis of $\mathbb{E}$. Calls with the same parameters reuse the precomputed data; challenges and sumcheck computations remain over $\mathbb{E}$.

Algorithm 1: Packed inner-product proof

Input: $N = m\kappa$, $\kappa \geq 2$; a fixed claim $c \in \mathbb{F}$ and matrices $\boldsymbol{A}, \boldsymbol{B} \in \mathbb{F}^{m \times \kappa}$
Output: Terminal values $\overline{v}_1, \overline{v}_2$; column weights $\overline{\boldsymbol{\rho}}_A, \overline{\boldsymbol{\rho}}_B \in \mathbb{E}^{\kappa}$; row MLE point $\overline{\boldsymbol{y}} \in \mathbb{E}^{\log_2 m}$; acceptance bit ok

`// Encode the inner product and fix the correction polynomials`

$\mathcal{P}$ defines $\boldsymbol{g}_A(X) \leftarrow \sum_{j=1}^{\kappa} \boldsymbol{a}^{(j)} X^{j-1}$ and $\boldsymbol{g}_B(X) \leftarrow \sum_{j=1}^{\kappa} \boldsymbol{b}^{(j)} X^{-(j-1)}$;
foreach $\tau \in \{-(\kappa-1), \ldots, -1, 1, \ldots, \kappa-1\}$ **do**
  $\mathcal{P}$ computes $f_\tau \leftarrow \sum_{\substack{j,j' \in [\kappa] \\ j-j'=\tau}} \langle \boldsymbol{a}^{(j)}, \boldsymbol{b}^{(j')} \rangle$;
$\mathcal{P}$ sets $h_+(X) \leftarrow \sum_{\tau=1}^{\kappa-1} f_\tau X^{\tau-1}$ and $h_-(X) \leftarrow \sum_{\tau=1}^{\kappa-1} f_{-\tau} X^{\tau-1}$;
$\mathcal{P} \to \mathcal{V}$: the $\kappa - 1$ coefficients of each of h_+ and h_-;
$\mathcal{V} \to \mathcal{P}$: $\overline{x} \xleftarrow{\$} \mathbb{E}^{\times}$;
`// Reduce to a length-`m` extension-field inner product`
$\mathcal{P}$ and $\mathcal{V}$ compute $\overline{\boldsymbol{\rho}}_A \leftarrow (1, \overline{x}, \ldots, \overline{x}^{\kappa-1})^{\top}$ and $\overline{\boldsymbol{\rho}}_B \leftarrow (1, \overline{x}^{-1}, \ldots, \overline{x}^{-(\kappa-1)})^{\top}$;
$\mathcal{P}$ computes $\overline{\boldsymbol{a}} \leftarrow \boldsymbol{A}\overline{\boldsymbol{\rho}}_A = \boldsymbol{g}_A(\overline{x})$ and $\overline{\boldsymbol{b}} \leftarrow \boldsymbol{B}\overline{\boldsymbol{\rho}}_B = \boldsymbol{g}_B(\overline{x})$;
$\mathcal{V}$ computes $\overline{c} \leftarrow c + \overline{x} h_+(\overline{x}) + \overline{x}^{-1} h_-(\overline{x}^{-1})$;
$\mathcal{P}$ and $\mathcal{V}$ apply Fact 1 to $\overline{c} \stackrel{?}{=} \langle \overline{\boldsymbol{a}}, \overline{\boldsymbol{b}} \rangle$, obtaining ok, $\overline{v}_1, \overline{v}_2$, and $\overline{\boldsymbol{y}} \in \mathbb{E}^{\log_2 m}$;
`// Return the terminal packed claims`
Output $\overline{v}_1 \stackrel{?}{=} \mathsf{ts}(\overline{\boldsymbol{y}})^{\top} \boldsymbol{A}\overline{\boldsymbol{\rho}}_A$ and $\overline{v}_2 \stackrel{?}{=} \mathsf{ts}(\overline{\boldsymbol{y}})^{\top} \boldsymbol{B}\overline{\boldsymbol{\rho}}_B$;
return $(\mathsf{ok}, \overline{v}_1, \overline{v}_2, \overline{\boldsymbol{\rho}}_A, \overline{\boldsymbol{\rho}}_B, \overline{\boldsymbol{y}})$;

Algorithm 2: Packed Hadamard-product proof

Input: $N = m\kappa$, $\kappa \geq 2$; a fixed claimed value $\overline{c} \in \mathbb{E}$ and matrices $\boldsymbol{A}, \boldsymbol{B} \in \mathbb{F}^{m \times \kappa}$; column weights $\overline{\boldsymbol{r}} \in \mathbb{E}^{\kappa}$; row weights $\overline{\boldsymbol{l}} = \bigotimes_{j=1}^{\log_2 m} (\overline{l}_{j,0}, \overline{l}_{j,1})^{\top}$; nodes $\mathcal{H} = \{\gamma_1, \ldots, \gamma_\kappa\} \subseteq \mathbb{K}$, with the preprocessed interpolation data of Appendix B.1

Output: Terminal values $\overline{v}_1, \overline{v}_2$; column weights $\overline{\boldsymbol{\rho}} \in \mathbb{E}^{\kappa}$; tensor row weights $\overline{\boldsymbol{\xi}}_1, \overline{\boldsymbol{\xi}}_2 \in \mathbb{E}^{m}$; acceptance bit ok

`// Select the interpolation arithmetic`

Take $\mathbb{K} = \mathbb{F}$ if $q \geq 2\kappa - 1$; otherwise use the precomputed $\mathbb{K}$-arithmetic tables;

`// Encode the weighted column claims`

$\mathcal{P}$ computes $\overline{\boldsymbol{d}}^{\top} \leftarrow \overline{\boldsymbol{l}}^{\top} (\boldsymbol{A} \circ \boldsymbol{B})$;

In the Lagrange basis on $\mathcal{H}$, define $\boldsymbol{g}_A(X) \leftarrow \sum_j \boldsymbol{a}^{(j)} \psi_j(X)$, $\boldsymbol{g}_B(X) \leftarrow \sum_j \boldsymbol{b}^{(j)} \psi_j(X)$, and $g_C(X) \leftarrow \sum_j \overline{d}^{(j)} \psi_j(X)$;

At each additional node $\gamma'_j \in \mathbb{K} \setminus \mathcal{H}$, $\mathcal{P}$ computes $f(\gamma'_j) \leftarrow \langle \overline{\boldsymbol{l}}, \boldsymbol{g}_A(\gamma'_j) \circ \boldsymbol{g}_B(\gamma'_j) \rangle - g_C(\gamma'_j)$, using $\mathbb{K}$ for the row products and $\mathbb{K}$-coordinates of $\mathbb{E}$ for their aggregation;

Interpolate f from these $\kappa - 1$ samples and the zeros on $\mathcal{H}$, then set $h(X) \leftarrow f(X)/\Psi(X)$;

$\mathcal{P} \to \mathcal{V}$: $\overline{\boldsymbol{d}}$ and the $\kappa - 1$ coefficients of h;

$\mathcal{V}$ sets $\mathsf{ok}_0 \leftarrow (\overline{c} \stackrel{?}{=} \langle \overline{\boldsymbol{d}}, \overline{\boldsymbol{r}} \rangle)$;

$\mathcal{V} \to \mathcal{P}$: $\overline{x} \xleftarrow{\$} \mathbb{E} \setminus \mathcal{H}$;

`// Reduce to a length-`m` extension-field inner product`

$\mathcal{P}$ and $\mathcal{V}$ compute $\overline{\boldsymbol{\rho}} \leftarrow (\psi_1(\overline{x}), \ldots, \psi_\kappa(\overline{x}))^{\top}$;

$\mathcal{P}$ computes $\overline{\boldsymbol{a}} \leftarrow \boldsymbol{A}\overline{\boldsymbol{\rho}}$ and $\overline{\boldsymbol{b}} \leftarrow \boldsymbol{B}\overline{\boldsymbol{\rho}}$;

$\mathcal{V}$ computes $\overline{c}_x \leftarrow \langle \overline{\boldsymbol{d}}, \overline{\boldsymbol{\rho}} \rangle = g_C(\overline{x})$;

$\mathcal{P}$ and $\mathcal{V}$ apply Fact 1 to $\langle \overline{\boldsymbol{a}}, \overline{\boldsymbol{b}} \circ \overline{\boldsymbol{l}} \rangle \stackrel{?}{=} \overline{c}_x + \Psi(\overline{x}) h(\overline{x})$, obtaining $\mathsf{ok}_{\mathsf{IP}}$, $\overline{v}_1, \overline{v}_2$, and $\overline{\boldsymbol{y}} \in \mathbb{E}^{\log_2 m}$;

`// Return the terminal packed claims`

Set $\overline{\boldsymbol{\xi}}_1 \leftarrow \mathsf{ts}(\overline{\boldsymbol{y}})$ and $\overline{\boldsymbol{\xi}}_2 \leftarrow \overline{\boldsymbol{l}} \circ \mathsf{ts}(\overline{\boldsymbol{y}})$;

Represent the j-th factor of $\overline{\boldsymbol{\xi}}_2$ by $(\overline{l}_{j,0}(1 - \overline{y}_j), \overline{l}_{j,1}\overline{y}_j)^{\top}$;

Output $\overline{v}_1 \stackrel{?}{=} \overline{\boldsymbol{\xi}}_1^{\top} \boldsymbol{A}\overline{\boldsymbol{\rho}}$ and $\overline{v}_2 \stackrel{?}{=} \overline{\boldsymbol{\xi}}_2^{\top} \boldsymbol{B}\overline{\boldsymbol{\rho}}$;

Set $\mathsf{ok} \leftarrow \mathsf{ok}_0 \wedge \mathsf{ok}_{\mathsf{IP}}$;

return $(\mathsf{ok}, \overline{v}_1, \overline{v}_2, \overline{\boldsymbol{\rho}}, \overline{\boldsymbol{\xi}}_1, \overline{\boldsymbol{\xi}}_2)$;

Algorithm 3: Packed permutation proof

Input: Committed matrices $\boldsymbol{A}, \boldsymbol{B} \in \mathbb{F}^{m \times \kappa}$; parameters of Section 4.3 and the shared interpolation data of Appendix B.1

Output: Two terminal packed claims and an acceptance bit

$\mathcal{V}$ rejects malformed messages and failed scalar or sumcheck checks;

Use $\mathbb{K} = \mathbb{F}$ if $q \geq 2\kappa - 1$, and the tabulated intermediate-field arithmetic otherwise;

After the commitments are fixed, $\mathcal{V} \to \mathcal{P} : \overline{z} \xleftarrow{\$} \mathbb{E}$;

foreach $\boldsymbol{D} \in \{\boldsymbol{A}, \boldsymbol{B}\}$ **do**

  From $\boldsymbol{C}_0(Z) \leftarrow Z\mathbf{1} + \boldsymbol{D}$, construct the coefficient tree over $\mathbb{F}$ by Eq. (11);

  Flatten $\boldsymbol{C}_{\log_2 \kappa}(\overline{z})$ in column-major order into $\boldsymbol{c}_{\log_2 \kappa}(\overline{z})$, then complete the extension-field tree as in Section 4.3;

$\mathcal{P} \to \mathcal{V} : \overline{\Theta} \in \mathbb{E}$, the claimed common root;

foreach $\boldsymbol{D} \in \{\boldsymbol{A}, \boldsymbol{B}\}$ **do**

  Use the product trees associated with $\boldsymbol{D}$;

  `// Unfold the extension-field tree`

  Set $\overline{v} \leftarrow \overline{\Theta}$ and $\overline{\boldsymbol{\xi}} \leftarrow (1)$;

  for $\tau = \log_2 N - 1, \ldots, \log_2 \kappa$ **do**

    Apply cubic weighted sumcheck to $\overline{v} = \langle \overline{\boldsymbol{\xi}}, \boldsymbol{c}_{\tau,U}(\overline{z}) \circ \boldsymbol{c}_{\tau,D}(\overline{z}) \rangle$, keeping the two child MLEs at the same point $\overline{\boldsymbol{y}}$;

    After the child values $\overline{v}_U, \overline{v}_D$ are fixed, sample $\mathcal{V} \to \mathcal{P} : \overline{y}_0 \xleftarrow{\$} \mathbb{E}$ and set $\overline{v} \leftarrow (1 - \overline{y}_0)\overline{v}_U + \overline{y}_0 \overline{v}_D$, $\overline{\boldsymbol{\xi}} \leftarrow \mathsf{ts}(\overline{y}_0 \,\|\, \overline{\boldsymbol{y}})$;

  Set $\overline{v}_{\log_2 \kappa} \leftarrow \overline{v}$ and factor $\overline{\boldsymbol{\xi}} = \overline{\boldsymbol{r}}_{\log_2 \kappa} \otimes \overline{\boldsymbol{l}}_{\log_2 \kappa}$;

  `// Unfold the coefficient tree, aggregating before evaluation`

  for $\tau = \log_2 \kappa - 1, \ldots, 0$ **do**

    Reconstruct the required coefficient layers and set $\boldsymbol{g}_\tau(Z, X) \leftarrow \boldsymbol{C}_\tau(Z)\boldsymbol{\phi}(X)$;

    Compute $\overline{\boldsymbol{u}}_{\tau+1}^\top = \overline{\boldsymbol{l}}_{\tau+1}^\top \boldsymbol{C}_{\tau+1}(\overline{z})$ by aggregating Z-coefficients before evaluating at $\overline{z}$;

    Set $g_C(X) \leftarrow \overline{\boldsymbol{u}}_{\tau+1}^\top \boldsymbol{\phi}(X)$;

    At each additional node γ'_j, form the coefficients of $\boldsymbol{\eta}_{\tau+1,j}(Z)$ in $\mathbb{K}$. Aggregate them with $\overline{\boldsymbol{l}}_{\tau+1}$ to obtain $F_{\tau+1,j}(Z)$, then evaluate at $\overline{z}$;

    Interpolate f from $f(\gamma'_j) = F_{\tau+1,j}(\overline{z}) - g_C(\gamma'_j)$ and its zeros on $\mathcal{H}$, then set $h(X) \leftarrow f(X)/\Psi(X)$, of degree at most $\kappa - 2$;

    $\mathcal{P} \to \mathcal{V} : \overline{\boldsymbol{u}}_{\tau+1}, h$; check $\overline{v}_{\tau+1} = \overline{\boldsymbol{u}}_{\tau+1}^\top \overline{\boldsymbol{r}}_{\tau+1}$;

    Sample $\mathcal{V} \to \mathcal{P} : \overline{x} \xleftarrow{\$} \mathbb{E} \setminus \mathcal{H}$ and compute $\boldsymbol{g}_\tau(\overline{z}, \overline{x})$ by contracting coefficients with $\overline{\boldsymbol{\rho}}(\overline{x})$ before evaluating at $\overline{z}$;

    Apply cubic weighted sumcheck to $\langle \overline{\boldsymbol{l}}_{\tau+1}, \boldsymbol{g}_{\tau,U}(\overline{z}, \overline{x}) \circ \boldsymbol{g}_{\tau,D}(\overline{z}, \overline{x}) \rangle = g_C(\overline{x}) + \Psi(\overline{x})h(\overline{x})$, obtaining child values $\overline{v}_{\tau,U}, \overline{v}_{\tau,D}$ at the same row MLE point $\overline{\boldsymbol{y}}$;

    After both values are fixed, sample $\mathcal{V} \to \mathcal{P} : \overline{y}_0 \xleftarrow{\$} \mathbb{E}$ and set $\overline{v}_\tau \leftarrow (1 - \overline{y}_0)\overline{v}_{\tau,U} + \overline{y}_0 \overline{v}_{\tau,D}$, $\overline{\boldsymbol{l}}_\tau \leftarrow \mathsf{ts}(\overline{y}_0 \,\|\, \overline{\boldsymbol{y}})$, and $\overline{\boldsymbol{r}}_\tau \leftarrow \overline{\boldsymbol{\rho}}(\overline{x})$;

  Output $\overline{v}_0 - \overline{z} = \overline{\boldsymbol{l}}_0^\top \boldsymbol{D} \overline{\boldsymbol{r}}_0$ for packed PCS opening by Section 4.4;

return `true` if all checks succeed;

Algorithm 4: Packed PCS opening protocol

Public input: $\mathsf{pp}_{\mathsf{pc}}, \mathsf{cm}$; $\mathbb{F} \subset \mathbb{E}$ with basis $\overline{\boldsymbol{\beta}}$ and $\overline{\beta}_1 = 1$; $N = m\kappa$, $\ell = \log_2 m$; $\overline{v} \in \mathbb{E}$, $\overline{\boldsymbol{\rho}} \in \mathbb{E}^\kappa$, and $\overline{\boldsymbol{\xi}} = \bigotimes_{j=1}^{\ell} \overline{\boldsymbol{\xi}}^{(j)}$, where $\overline{\boldsymbol{\xi}}^{(j)} = (\overline{\xi}_0^{(j)}, \overline{\xi}_1^{(j)})^\top \in \mathbb{E}^2$

Private input: $\mathcal{P}$: $\boldsymbol{A} \in \mathbb{F}^{m \times \kappa}$ and commitment randomness ρ_{pc}

Output: Acceptance bit ok

$\mathcal{V}$ returns 0 on a malformed message or a failed check, reduction, or opening;

`// Project the column claims to one inner product`

$\mathcal{P}$ sets $\overline{\boldsymbol{a}} \leftarrow \boldsymbol{A}\overline{\boldsymbol{\beta}}$ and decodes $\boldsymbol{\Xi} \in \mathbb{F}^{m \times \kappa}$ from $\overline{\boldsymbol{\xi}} = \boldsymbol{\Xi}\overline{\boldsymbol{\beta}}$;

$\mathcal{P}$ computes $\boldsymbol{U} \leftarrow \boldsymbol{\Xi}^\top \boldsymbol{A}$;

$\mathcal{P} \to \mathcal{V}$: $\overline{\boldsymbol{v}} \leftarrow \boldsymbol{U}^\top \overline{\boldsymbol{\beta}} \in \mathbb{E}^\kappa$;

$\mathcal{V}$ checks $\overline{v} \stackrel{?}{=} \overline{\boldsymbol{v}}^\top \overline{\boldsymbol{\rho}}$;

$\mathcal{V}$ recovers $\boldsymbol{U} \in \mathbb{F}^{\kappa \times \kappa}$ from $\overline{\boldsymbol{v}}^\top = \overline{\boldsymbol{\beta}}^\top \boldsymbol{U}$;

$\mathcal{V} \to \mathcal{P}$: $\overline{\boldsymbol{r}} \xleftarrow{\$} \mathbb{E}^\kappa$;

$\mathcal{V}$ computes $\overline{c} \leftarrow \overline{\boldsymbol{r}}^\top \boldsymbol{U} \overline{\boldsymbol{\beta}}$;

$\mathcal{P}$ computes the operand $\boldsymbol{\Xi}\overline{\boldsymbol{r}} \in \mathbb{E}^m$;

$\mathcal{P}, \mathcal{V}$ apply Fact 1 over $\mathbb{E}$ to $\overline{c} \stackrel{?}{=} \langle \overline{\boldsymbol{a}}, \boldsymbol{\Xi}\overline{\boldsymbol{r}} \rangle$, and obtain terminal values $\overline{w}_{\mathsf{com}}, \overline{w}$ and a point $\overline{\boldsymbol{y}} \in \mathbb{E}^\ell$;

`// Compute the MLE evaluation of the public weight vector`

Use the multiplication matrices $\boldsymbol{M}_{\overline{\alpha}}$ from Eq. (23), applied by extension-field scaling of rows and columns without materializing $\boldsymbol{M}_{\overline{\alpha}}$;

$\mathcal{V}$ initializes $\boldsymbol{W} \leftarrow \boldsymbol{e}_1 \boldsymbol{e}_1^\top \in \mathbb{F}^{\kappa \times \kappa}$, where $\boldsymbol{e}_1 = (1, 0, \ldots, 0)^\top$;

for $j = \ell, \ldots, 1$ **do**

  $\mathcal{V}$ updates $\boldsymbol{W} \leftarrow \boldsymbol{W}\boldsymbol{M}_{\overline{\xi}_0^{(j)}}^\top + \boldsymbol{M}_{\overline{y}_j} \boldsymbol{W} \boldsymbol{M}_{\overline{\xi}_1^{(j)} - \overline{\xi}_0^{(j)}}^\top$;

  `// Use the previous` $\boldsymbol{W}$ `on the right, as in Eq.` (25)

$\mathcal{V}$ checks $\overline{w} \stackrel{?}{=} \overline{\boldsymbol{\beta}}^\top \boldsymbol{W} \overline{\boldsymbol{r}}$;

$\mathsf{ok} \leftarrow \mathsf{pc}.\mathsf{Open}\big(\mathsf{pp}_{\mathsf{pc}}, \mathsf{cm}, \overline{w}_{\mathsf{com}}, \overline{\boldsymbol{y}} : \overline{\boldsymbol{a}}, \rho_{\mathsf{pc}}\big)$;

return ok;

Algorithm 5: SNARK protocol for PLONK with a single instance

Public input: Parameters of Theorem 1; io; the preprocessed public commitment of logical length m_{pub}

Private input: $\mathcal{P}$: $\boldsymbol{w}_L, \boldsymbol{w}_R, \boldsymbol{w}_O \in \mathbb{F}^N$; circuit tables and their opening data

Output: Acceptance bit ok

$\mathcal{V}$ returns 0 on a malformed message or a failed check, reduction, or opening;

`// Commit before sampling challenges`

Arrange $\boldsymbol{w}_\star$ as $\boldsymbol{W}_\star \in \mathbb{F}^{m \times \kappa}$ for $\star \in \{L, R, O\}$;

$\mathcal{P} \to \mathcal{V}$: one commitment to the concatenation of $\boldsymbol{W}_L\overline{\boldsymbol{\beta}}, \boldsymbol{W}_R\overline{\boldsymbol{\beta}}, \boldsymbol{W}_O\overline{\boldsymbol{\beta}}$, of logical length $3m$;

Set the virtual vector $\boldsymbol{w}_M \leftarrow \boldsymbol{w}_L \circ \boldsymbol{w}_R$;

`// Reduce the gate claims; defer all terminal openings`

$\mathcal{V} \to \mathcal{P}$: $\overline{\boldsymbol{x}} \xleftarrow{\$} \mathbb{E}^{\log_2 N}$; set $\overline{\boldsymbol{\xi}} \leftarrow \mathsf{ts}(\overline{\boldsymbol{x}})$;

$\mathcal{P} \to \mathcal{V}$: $\overline{c}_L, \overline{c}_R, \overline{c}_M, \overline{c}_O, \overline{c}_C$ as in Eq. (12) and Eq. (13);

$\mathcal{V}$ checks $\overline{c}_L + \overline{c}_R + \overline{c}_M + \overline{c}_O + \overline{c}_C \stackrel{?}{=} 0$;

foreach $\star \in \{L, R, O\}$ **do**

  Apply Algorithm 2 to $\overline{c}_\star \stackrel{?}{=} \langle \overline{\boldsymbol{\xi}}, \boldsymbol{q}_\star \circ \boldsymbol{w}_\star \rangle$; retain both terminal linear claims;

Apply Algorithm 2 to $\overline{c}_M \stackrel{?}{=} \langle \overline{\boldsymbol{\xi}}, \boldsymbol{q}_M \circ \boldsymbol{w}_M \rangle$; retain its selector claim;

Express its product-vector claim as $\overline{v} \stackrel{?}{=} \overline{\boldsymbol{l}}^\top (\boldsymbol{W}_L \circ \boldsymbol{W}_R)\overline{\boldsymbol{r}}$. Apply Algorithm 2 again and retain both terminals;

Retain $\overline{c}_C \stackrel{?}{=} \langle \overline{\boldsymbol{\xi}}, \boldsymbol{q}_C \rangle$;

`// Reduce the copy claims over` $\mathbb{K}_{\mathsf{pos}}$

Form $\boldsymbol{w} \in \mathbb{F}^L$, including the public I/O, and $\boldsymbol{a} \leftarrow \boldsymbol{\varsigma} + \theta_d \boldsymbol{w}$, $\boldsymbol{b} \leftarrow \boldsymbol{\iota} + \theta_d \boldsymbol{w}$;

Apply Algorithm 3 with base field $\mathbb{K}_{\mathsf{pos}}$, extension $\mathbb{E}$, and packing width κ_{pos}, sampling its fresh challenge $\overline{z} \xleftarrow{\$} \mathbb{E} \setminus \{0\}$;

Expand its two terminal packed claims as in Appendix C.6. Restrict to the wire segments and regroup the tensor factors to packing width κ;

$\mathcal{P} \to \mathcal{V}$: six wire values and $3\lceil \log_{\underline{q}} L \rceil$ permuted-digit values; retain their linear claims;

$\mathcal{V}$ evaluates the public contributions directly and checks both recombinations;

`// Prove the terminal claims`

Prove all retained claims using Algorithm 4, as described in Appendix C.6;

return true if all checks succeed;

Algorithm 6: SNARK protocol for R1CS with packing across instances

Public input: Parameters of Theorem 2; the κ public-I/O vectors; preprocessed sparse commitments to $\boldsymbol{S}_L, \boldsymbol{S}_R, \boldsymbol{S}_O$

Private input: $\mathcal{P}$: $\boldsymbol{W} \in \mathbb{F}^{N \times \kappa}$; the circuit matrices

Output: Acceptance bit `ok`

$\mathcal{V}$ returns 0 on a malformed message or a failed check, reduction, or opening;

$\mathcal{P} \to \mathcal{V}$: a commitment to $\overline{\boldsymbol{w}} \leftarrow \boldsymbol{W}\overline{\boldsymbol{\beta}} \in \mathbb{E}^N$;

`// Bind the public inputs and outputs`

$\mathcal{V} \to \mathcal{P}$: a fresh uniform MLE point in $\mathbb{E}^{\log_2 N}$;

Let $\overline{\boldsymbol{\xi}}$ be its tensor weight vector, set to zero outside the public-I/O positions. $\mathcal{V}$ computes the claimed value $\langle \overline{\boldsymbol{\xi}}, \overline{\boldsymbol{w}} \rangle$ from the packed public I/O;

Apply Fact 1 to this claim. $\mathcal{V}$ computes the terminal MLE evaluation of the public weight vector directly and retains the witness MLE claim;

`// Check a random projection of the R1CS residual`

$\mathcal{V} \to \mathcal{P}$: independent uniform row and column MLE points; set their tensor weights to $\overline{\boldsymbol{l}} \in \mathbb{E}^N$ and $\overline{\boldsymbol{\rho}}_O \in \mathbb{E}^\kappa$, and set $\overline{\boldsymbol{\xi}}_O \leftarrow \overline{\boldsymbol{l}}$;

$\mathcal{P} \to \mathcal{V}$: $\overline{v}_O$, claimed to equal $\overline{\boldsymbol{l}}^\top \boldsymbol{S}_O \boldsymbol{W} \overline{\boldsymbol{\rho}}_O$;

Run Algorithm 2 on the claim $\overline{v}_O \stackrel{?}{=} \overline{\boldsymbol{l}}^\top [(\boldsymbol{S}_L \boldsymbol{W}) \circ (\boldsymbol{S}_R \boldsymbol{W})] \overline{\boldsymbol{\rho}}_O$. Retain its two terminal claims and the output claim, written $\overline{v}_\star \stackrel{?}{=} \overline{\boldsymbol{\xi}}_\star^\top \boldsymbol{S}_\star \boldsymbol{W} \overline{\boldsymbol{\rho}}_\star$ for $\star \in \{L, R, O\}$;

`// Prove the three linear relations`

foreach $\star \in \{L, R, O\}$ **do**

  Write $\overline{\boldsymbol{\xi}}_\star = \boldsymbol{\Xi}_\star \overline{\boldsymbol{\beta}}$. $\mathcal{P}$ computes $\boldsymbol{U}_\star \leftarrow \boldsymbol{\Xi}_\star^\top \boldsymbol{S}_\star \boldsymbol{W}$;

  $\mathcal{P} \to \mathcal{V}$: $\overline{\boldsymbol{v}}_\star = \boldsymbol{U}_\star^\top \overline{\boldsymbol{\beta}}$. $\mathcal{V}$ recovers $\boldsymbol{U}_\star$ and checks $\overline{v}_\star \stackrel{?}{=} \overline{\boldsymbol{v}}_\star^\top \overline{\boldsymbol{\rho}}_\star$;

After all three column messages are fixed, $\mathcal{V} \to \mathcal{P}$: independent $\overline{z} \stackrel{\$}{\leftarrow} \mathbb{E}$ and $\overline{\boldsymbol{r}} \stackrel{\$}{\leftarrow} \mathbb{E}^\kappa$;

Apply Fact 1 to $\overline{\boldsymbol{r}}^\top (\boldsymbol{U}_O + \overline{z}\boldsymbol{U}_L + \overline{z}^2 \boldsymbol{U}_R) \overline{\boldsymbol{\beta}} \stackrel{?}{=} \langle \overline{\boldsymbol{w}}, (\boldsymbol{S}_O^\top \boldsymbol{\Xi}_O + \overline{z}\boldsymbol{S}_L^\top \boldsymbol{\Xi}_L + \overline{z}^2 \boldsymbol{S}_R^\top \boldsymbol{\Xi}_R) \overline{\boldsymbol{r}} \rangle$. Retain its witness MLE claim at $\overline{\boldsymbol{y}}$;

$\mathcal{P} \to \mathcal{V}$: three claimed values $\langle \boldsymbol{S}_\star \mathsf{ts}(\overline{\boldsymbol{y}}), \boldsymbol{\Xi}_\star \overline{\boldsymbol{r}} \rangle$. $\mathcal{V}$ checks their $(1, \overline{z}, \overline{z}^2)$-weighted sum against the terminal MLE evaluation of the public weight vector, in order O, L, R;

For each value, apply Fact 1. At its terminal point $\overline{\boldsymbol{y}}'$, prove the sparse matrix MLE claim $\mathsf{ts}(\overline{\boldsymbol{y}}')^\top \boldsymbol{S}_\star \mathsf{ts}(\overline{\boldsymbol{y}})$ by the sparse-evaluation protocol of Spartan; evaluate $\mathsf{ts}(\overline{\boldsymbol{y}}')^\top \boldsymbol{\Xi}_\star \overline{\boldsymbol{r}}$ using Appendix B.2;

Prove both retained witness MLE claims using the PCS opening protocol for the length-N commitment;

return `true` if all checks succeed;

B.2 Packed Polynomial Commitment Scheme

This appendix details the packed PCS of Section 4.4. We first give the packed opening algorithm, which proves a linear claim about $\boldsymbol{A} \in \mathbb{F}^{m \times \kappa}$ with tensor row weights through a single MLE opening of its fixed-basis packing $\boldsymbol{A}\overline{\boldsymbol{\beta}}$. The verifier computes the terminal MLE evaluation of the public weight vector by a recursion on $\kappa \times \kappa$ base-field matrices. As an alternative, we also present a bitwidth-aware PCS based on the code-switching construction.

Solution 1: From a PCS over the Extension Field

Parameters. Let $N = m\kappa$, $[\mathbb{E} : \mathbb{F}] = \kappa$, and $\ell = \log_2 m$, where m is a power of two and $\kappa \geq 2$. Fix an $\mathbb{F}$-basis $\overline{\boldsymbol{\beta}} = (\overline{\beta}_1, \ldots, \overline{\beta}_\kappa)^\top$ of $\mathbb{E}$, with $\overline{\beta}_1 = 1$, and let pc be a multilinear PCS for vectors in $\mathbb{E}^m$. Store extension-field elements in this basis, so packing and coordinate extraction only reorganize base-field coordinates.

Setup and commitment. Run $\mathsf{pc.Setup}(1^\lambda, m)$ to obtain $\mathsf{pp}_{\mathsf{pc}}$, and include the fields, basis, and dimensions N, m, κ in the public parameters. For the witness matrix $\boldsymbol{A} = \left[\, \boldsymbol{a}^{(1)} \mid \ldots \mid \boldsymbol{a}^{(\kappa)} \,\right] \in \mathbb{F}^{m \times \kappa}$, pack each row into one extension-field element, obtaining $\overline{\boldsymbol{a}} = \boldsymbol{A}\overline{\boldsymbol{\beta}} \in \mathbb{E}^m$. Compute the commitment $\mathsf{cm} = \mathsf{pc.Comm}(\mathsf{pp}_{\mathsf{pc}}, \overline{\boldsymbol{a}}; \rho_{\mathsf{pc}})$, where ρ_{pc} is the commitment randomness, and retain $\boldsymbol{A}$ and ρ_{pc}. The fixed basis makes binding to $\overline{\boldsymbol{a}}$ equivalent to binding to $\boldsymbol{A}$.

Opening relation. Given column weights $\overline{\boldsymbol{\rho}} \in \mathbb{E}^\kappa$, tensor row weights $\overline{\boldsymbol{\xi}} \in \mathbb{E}^m$, and a claimed value $\overline{v} \in \mathbb{E}$, the protocol proves:

$$\overline{v} = \langle \boldsymbol{A}\overline{\boldsymbol{\rho}}, \overline{\boldsymbol{\xi}} \rangle = \overline{\boldsymbol{\xi}}^\top \boldsymbol{A}\overline{\boldsymbol{\rho}}. \tag{18}$$

The row weights are supplied as $\overline{\boldsymbol{\xi}} = \bigotimes_{j=1}^{\ell} \overline{\boldsymbol{\xi}}^{(j)}$, where each factor is $\overline{\boldsymbol{\xi}}^{(j)} = (\overline{\xi}_0^{(j)}, \overline{\xi}_1^{(j)})^\top \in \mathbb{E}^2$. For an ordinary MLE claim, $\overline{\boldsymbol{\rho}}$ and $\overline{\boldsymbol{\xi}}$ are the tensor spans of the column and row parts of the evaluation point. Arbitrary two-entry row factors also support the element-wise products of tensor weights returned by the packed Hadamard reduction.

Reduction to an inner product. The commitment uses the fixed weights $\overline{\boldsymbol{\beta}}$, whereas the linear claim uses $\overline{\boldsymbol{\rho}}$. To relate these packings, the prover claims the column evaluations $\overline{\boldsymbol{v}}^\top = \overline{\boldsymbol{\xi}}^\top \boldsymbol{A}$. Write the row weights and claimed column values in base-field coordinates:

$$\overline{\boldsymbol{\xi}} = \boldsymbol{\Xi}\overline{\boldsymbol{\beta}}, \qquad \overline{\boldsymbol{v}}^\top = \overline{\boldsymbol{\beta}}^\top \boldsymbol{U}, \qquad \boldsymbol{\Xi} \in \mathbb{F}^{m \times \kappa}, \quad \boldsymbol{U} \in \mathbb{F}^{\kappa \times \kappa}.$$

The prover computes $\boldsymbol{U} = \boldsymbol{\Xi}^\top \boldsymbol{A}$ and sends $\overline{\boldsymbol{v}} = \boldsymbol{U}^\top \overline{\boldsymbol{\beta}}$. The verifier extracts $\boldsymbol{U}$ from this message and checks $\overline{v} = \overline{\boldsymbol{v}}^\top \overline{\boldsymbol{\rho}}$. The public tensor factors specify $\boldsymbol{\Xi}$ implicitly.

Since $\boldsymbol{A}$ has base-field entries, the remaining claim $\overline{\boldsymbol{v}}^\top = \overline{\boldsymbol{\xi}}^\top \boldsymbol{A}$ is equivalent to $\boldsymbol{U} = \boldsymbol{\Xi}^\top \boldsymbol{A}$. Packing the rows of this identity yields the committed vector $\overline{\boldsymbol{a}}$:

$$\boldsymbol{U} = \boldsymbol{\Xi}^\top \boldsymbol{A} \quad \Longleftrightarrow \quad \boldsymbol{U}\overline{\boldsymbol{\beta}} = \boldsymbol{\Xi}^\top \overline{\boldsymbol{a}}. \tag{19}$$

After receiving $\overline{\boldsymbol{v}}$, the verifier samples $\overline{\boldsymbol{r}} \xleftarrow{\$} \mathbb{E}^{\kappa}$, independently of the original row and column weights, and projects Eq. (19) to one inner-product claim:

$$\overline{c} = \overline{\boldsymbol{r}}^{\top} \boldsymbol{U} \overline{\boldsymbol{\beta}} = \langle \overline{\boldsymbol{a}}, \boldsymbol{\Xi} \overline{\boldsymbol{r}} \rangle. \tag{20}$$

The verifier computes $\overline{c}$ by packing the rows of $\boldsymbol{U}$ and taking their inner product with $\overline{\boldsymbol{r}}$.

Apply Fact 1 over $\mathbb{E}$ to Eq. (20). At its terminal point $\overline{\boldsymbol{y}} \in \mathbb{E}^{\ell}$, the prover supplies $\overline{w}_{\mathsf{com}}, \overline{w} \in \mathbb{E}$ for the claims:

$$\overline{w}_{\mathsf{com}} = \langle \overline{\boldsymbol{a}}, \mathsf{ts}(\overline{\boldsymbol{y}}) \rangle, \qquad \overline{w} = \mathsf{ts}(\overline{\boldsymbol{y}})^{\top} \boldsymbol{\Xi} \overline{\boldsymbol{r}}. \tag{21}$$

The first claim is proved using the underlying PCS. The verifier checks the second directly from the public tensor factors.

Public MLE evaluation. Write $\mathsf{ts}(\overline{\boldsymbol{y}}) = \boldsymbol{Y} \overline{\boldsymbol{\beta}}$, where $\boldsymbol{Y} \in \mathbb{F}^{m \times \kappa}$. It suffices to compute the small matrix:

$$\boldsymbol{W} = \boldsymbol{Y}^{\top} \boldsymbol{\Xi} \in \mathbb{F}^{\kappa \times \kappa}, \qquad \overline{w} = \overline{\boldsymbol{\beta}}^{\top} \boldsymbol{W} \overline{\boldsymbol{r}}. \tag{22}$$

The tensor structure allows the verifier to compute $\boldsymbol{W}$ without forming either length-m coordinate table.

For $\overline{\alpha} \in \mathbb{E}$, let $\boldsymbol{M}_{\overline{\alpha}} \in \mathbb{F}^{\kappa \times \kappa}$ be the coordinate matrix of multiplication by $\overline{\alpha}$, characterized by:

$$\overline{\boldsymbol{\beta}}^{\top} \boldsymbol{M}_{\overline{\alpha}} = \overline{\alpha}\, \overline{\boldsymbol{\beta}}^{\top}. \tag{23}$$

Thus left multiplication by $\boldsymbol{M}_{\overline{\alpha}}$ scales each coordinate column by $\overline{\alpha}$ in $\mathbb{E}$, and right multiplication by $\boldsymbol{M}_{\overline{\alpha}}^{\top}$ scales each coordinate row. In particular, $\boldsymbol{M}_{1-\overline{\alpha}} = \boldsymbol{I} - \boldsymbol{M}_{\overline{\alpha}}$.

First consider MLE row weights $\overline{\boldsymbol{\xi}} = \mathsf{ts}(\overline{\boldsymbol{x}})$, with $\overline{\boldsymbol{x}} \in \mathbb{E}^{\ell}$. Let $\boldsymbol{Y}_j, \boldsymbol{\Xi}_j$ be the coordinate matrices for the suffix factors $j, \ldots, \ell$:

$$\boldsymbol{Y}_j \overline{\boldsymbol{\beta}} = \bigotimes_{i=j}^{\ell} \left(1 - \overline{y}_i, \overline{y}_i\right)^{\top}, \qquad \boldsymbol{\Xi}_j \overline{\boldsymbol{\beta}} = \bigotimes_{i=j}^{\ell} \left(1 - \overline{x}_i, \overline{x}_i\right)^{\top}.$$

Here $\boldsymbol{Y}_1 = \boldsymbol{Y}$, $\boldsymbol{\Xi}_1 = \boldsymbol{\Xi}$, and $\boldsymbol{Y}_{\ell+1} = \boldsymbol{\Xi}_{\ell+1} = \boldsymbol{e}_1^{\top}$, where $\boldsymbol{e}_1 = (1, 0, \ldots, 0)^{\top} \in \mathbb{F}^{\kappa}$. Splitting on the first suffix bit gives:

$$\boldsymbol{Y}_j = \begin{bmatrix} \boldsymbol{Y}_{j+1} \boldsymbol{M}_{1-\overline{y}_j}^{\top} \\ \boldsymbol{Y}_{j+1} \boldsymbol{M}_{\overline{y}_j}^{\top} \end{bmatrix}, \qquad \boldsymbol{\Xi}_j = \begin{bmatrix} \boldsymbol{\Xi}_{j+1} \boldsymbol{M}_{1-\overline{x}_j}^{\top} \\ \boldsymbol{\Xi}_{j+1} \boldsymbol{M}_{\overline{x}_j}^{\top} \end{bmatrix}. \tag{24}$$

Consequently, the verifier computes $\boldsymbol{W}_j = \boldsymbol{Y}_j^{\top} \boldsymbol{\Xi}_j$ by initializing $\boldsymbol{W}_{\ell+1} = \boldsymbol{e}_1 \boldsymbol{e}_1^{\top}$ and processing $j = \ell, \ldots, 1$:

$$\begin{aligned} \boldsymbol{W}_j &= \boldsymbol{M}_{1-\overline{y}_j} \boldsymbol{W}_{j+1} \boldsymbol{M}_{1-\overline{x}_j}^{\top} + \boldsymbol{M}_{\overline{y}_j} \boldsymbol{W}_{j+1} \boldsymbol{M}_{\overline{x}_j}^{\top} \\ &= \boldsymbol{W}_{j+1} \boldsymbol{M}_{1-\overline{x}_j}^{\top} + \boldsymbol{M}_{\overline{y}_j} \boldsymbol{W}_{j+1} \boldsymbol{M}_{2\overline{x}_j - 1}^{\top}. \end{aligned} \tag{25}$$

The output is $\boldsymbol{W} = \boldsymbol{W}_1$. For general tensor factors $\overline{\boldsymbol{\xi}}^{(j)}$, replace $1-\overline{x}_j$ and $\overline{x}_j$ in the first line by $\overline{\xi}_0^{(j)}$ and $\overline{\xi}_1^{(j)}$, respectively; the second line then uses $\boldsymbol{M}_{\overline{\xi}_0^{(j)}}$ and $\boldsymbol{M}_{\overline{\xi}_1^{(j)}-\overline{\xi}_0^{(j)}}$. Each step of the second line uses at most 3κ extension-field multiplications, implemented by row and column scaling without materializing $\boldsymbol{M}_{\overline{\alpha}}$. The recursion takes $O(\kappa\ell)$ extension-field multiplications and $O(\kappa^2\ell)$ base-field additions and coordinate moves, using $O(\kappa^2)$ base-field coordinates of working memory. It applies in every characteristic.

The verifier then packs the columns of $\boldsymbol{W}$ and checks $\overline{w} = \overline{\boldsymbol{\beta}}^{\top}\boldsymbol{W}\overline{\boldsymbol{r}}$, as required by Eq. (22).

Terminal extension-field opening. Finally, the parties prove the MLE claim for $\overline{\boldsymbol{a}}$ at $\overline{\boldsymbol{y}}$ with claimed value $\overline{w}_{\mathsf{com}}$ by invoking $\mathsf{pc.Open}(\mathsf{pp}_{\mathsf{pc}}, \mathsf{cm}, \overline{w}_{\mathsf{com}}, \overline{\boldsymbol{y}} : \overline{\boldsymbol{a}}, \rho_{\mathsf{pc}})$. The verifier accepts if the scalar checks, the inner-product reduction, and the underlying opening accept. Algorithm 4 in Appendix B.1 summarizes the protocol.

Proposition 4 (Packed polynomial commitment scheme). *Let* pc *bind the prover to a vector in* $\mathbb{E}^m$*, with evaluation-soundness error* ϵ_{pc} *for adaptively chosen MLE claims about that vector. The packed commitment is binding and the opening protocol is complete, with knowledge-soundness error at most:*

$$\epsilon_{\mathsf{PCS}} \coloneqq \epsilon_{\mathsf{pc}} + \frac{1+2\ell}{|\mathbb{E}|}. \tag{26}$$

Throughout, ϵ_{pc} *denotes the error of the underlying extension-field PCS and* ϵ_{PCS} *that of the packed PCS built from it. The opening reduction requires no additional lower bound on* $|\mathbb{F}|$*.*

Excluding the underlying PCS opening, the prover's arithmetic cost is:

$$\mathcal{TP} = O(\kappa N)\,\mathsf{bb} + O(N)\,\mathsf{be} + O(m+\kappa)\,\mathsf{ee}. \tag{27}$$

The verifier's arithmetic cost is:

$$\mathcal{TV} = O\big(\kappa^2(1+\ell)\big)\,\mathsf{bb} + O\big(\kappa(1+\ell)\big)\,\mathsf{ee}. \tag{28}$$

When $m \geq \kappa$*, schoolbook arithmetic gives* $O(\kappa N)$ *base-field operations for the prover. Peak prover memory is* $O(N+\kappa^2)$ *base-field coordinates, or* $O(N)$ *when* $m \geq \kappa$*, excluding the workspace of the underlying PCS.*

With $\overline{v}$ *already part of the public statement, the proof transcript, excluding the underlying opening, consists of* $\kappa + 2\ell + 2$ *extension-field elements.*

The proof is deferred to Appendix C.4.

Solution 2: From a Linear Code over the Base Field

Our second bitwidth-aware PCS instantiation starts from a linear code over the base field. By linearity, applying the same generator matrix to packed messages induces a code over $\mathbb{E}$, which serves as the source encoding for the code-switching protocol [15], with a foldable code such as the BaseFold code [68] as the target code.

Let $\mathcal{C} : \mathbb{F}^m \to \mathbb{F}^n$ be a linear code with generator matrix $\boldsymbol{G} \in \mathbb{F}^{n\times m}$, and let its κ-way interleaving encode the columns of $\boldsymbol{A} \in \mathbb{F}^{m\times\kappa}$ independently. Packing the resulting codeword in the fixed basis gives:

$$\boldsymbol{C} = \boldsymbol{G}\boldsymbol{A} \in \mathbb{F}^{n\times\kappa}, \qquad \overline{\boldsymbol{c}} = \boldsymbol{C}\overline{\boldsymbol{\beta}} = \boldsymbol{G}(\boldsymbol{A}\overline{\boldsymbol{\beta}}) = \mathcal{C}_{\mathbb{E}}(\overline{\boldsymbol{a}}),$$

where $\mathcal{C}_{\mathbb{E}}$ is the scalar extension of $\mathcal{C}$ defined by the same generator matrix, which has the same rate and relative distance. Each row of $\boldsymbol{C}$ represents one symbol of the extension-field codeword, so a commitment to these rows is a source codeword oracle for $\overline{\boldsymbol{a}} = \boldsymbol{A}\overline{\boldsymbol{\beta}}$.

To open this commitment, code switching converts the source oracle to a target-code encoding of the same length-m message $\overline{\boldsymbol{a}}$ over $\mathbb{E}$, which supports efficient MLE openings. These openings instantiate pc: the packed opening reduction handles the challenge-dependent weights $\overline{\boldsymbol{\rho}}$, and the code-switching PCS proves the terminal MLE claim for the fixed-basis packing.

If $\mathcal{C}$ is constant-rate and linear-time encodable, forming $\boldsymbol{C}$ costs $O(N)$ base-field operations, and the committed data occupies $O(N \log_2 |\mathbb{F}|)$ bits. The source encoding thus retains the small-field representation, while the target code operates on only $m = N/\kappa$ extension-field symbols.

B.3 Evaluating the MLE of the Identity Labels in the Permutation Argument

We evaluate the identity labels $\boldsymbol{\iota} \in \mathbb{K}^L_{\mathsf{pos}}$ of Section 5.1 directly from the position bits, without constructing the length-L table. Let $\underline{q} = 2^s$ and $\mu = \lceil \log_2 L \rceil$.

Choosing the coefficients. We first choose $\delta_1, \ldots, \delta_s$ over an arbitrary finite field $\mathbb{F}$, as required by the position encoding. Write $q = p^e$, where $p = \mathrm{char}(\mathbb{F})$, and fix an $\mathbb{F}_p$-basis $(\varepsilon_1, \ldots, \varepsilon_e)$ of $\mathbb{F}$, with $\varepsilon_1 = 1$. Choose:

$$s = e\lfloor \log_2 p \rfloor, \qquad \delta_{(j-1)\lfloor \log_2 p\rfloor + \ell} = 2^{\ell-1}\varepsilon_j, \quad j \in [e], \quad \ell \in [\lfloor \log_2 p \rfloor].$$

Within each basis coordinate, the subset sums are the distinct integers from 0 to $2^{\lfloor \log_2 p\rfloor} - 1 < p$. Independence of the ε_j therefore makes all 2^s encoded values distinct. The choice also gives $\frac{1}{2}\log_2 q \le s \le \log_2 q$, hence $\underline{q} \le q$ and $s = \Theta(\log q)$. For a prime field, this gives $\delta_\ell = 2^{\ell-1}$ and $s = \lfloor \log_2 q \rfloor$.

Proposition 5 (Evaluating position labels). *The MLE of $\boldsymbol{\iota} \,\|\, \boldsymbol{0}_{2^\mu - L}$ can be evaluated in $O(\log L)$ extension-field operations. More generally, arrange this vector in column-major order with κ_{pos} columns, where $\kappa_{\mathsf{pos}} \mid 2^\mu$. Its contraction with arbitrary column weights and tensor row weights takes $O(\kappa_{\mathsf{pos}} + \log L)$ extension-field operations.*

We derive the MLE and weighted-evaluation bounds below. The case $L = 1$ is immediate; assume $L \ge 2$ below.

Recursive evaluation. For $\overline{\boldsymbol{x}} \in \mathbb{E}^s$, let $\overline{v}_s(\overline{\boldsymbol{x}})$ be the MLE of the table whose entry at $1 + \sum_{\ell=1}^s 2^{\ell-1}x_\ell$ is $\sum_{\ell=1}^s \delta_\ell x_\ell$, for $x_\ell \in \{0,1\}$. The second half adds

δ_s to every entry of the first half. The remaining tensor factors sum to one, so splitting off the first factor gives:

$$\begin{aligned}\overline{v}_s(\overline{\boldsymbol{x}}) &= (1-\overline{x}_1)\overline{v}_{s-1}(\overline{x}_2,\ldots,\overline{x}_s) + \overline{x}_1\left(\overline{v}_{s-1}(\overline{x}_2,\ldots,\overline{x}_s) + \delta_s\right) \\ &= \overline{v}_{s-1}(\overline{x}_2,\ldots,\overline{x}_s) + \delta_s\overline{x}_1 \\ &= \sum_{\ell=1}^{s} \delta_{s-\ell+1}\overline{x}_\ell.\end{aligned} \tag{29}$$

Starting from $\overline{v}_1(\overline{x}_1) = \delta_1\overline{x}_1$, each step uses the preceding value once and adds one term. The total cost is $O(s)$ extension-field operations.

Power-of-two length. Suppose $L = 2^\mu$, with positions listed in lexicographic binary order, most significant bit first. Each vector $\boldsymbol{i}_{(j)}$ from Section 5.1 depends on a block of at most s position bits. All other bits repeat the same values and contribute a total tensor weight of one. Applying Eq. (29) to each block yields:

$$\left\langle \boldsymbol{i}_{(j)}, \mathsf{ts}(\overline{\boldsymbol{x}}) \right\rangle = \sum_{\ell=\max\{1,\mu-js+1\}}^{\mu-(j-1)s} \delta_{\mu-(j-1)s-\ell+1}\overline{x}_\ell, \qquad j \in [\lceil \log_{\underline{q}} L \rceil]. \tag{30}$$

The lower limit accounts for an incomplete most significant digit. By linearity, the identity-label evaluation is:

$$\begin{aligned}\left\langle \boldsymbol{\iota}, \mathsf{ts}(\overline{\boldsymbol{x}}) \right\rangle &= \sum_{j=1}^{\lceil \log_{\underline{q}} L \rceil} \theta_j \left\langle \boldsymbol{i}_{(j)}, \mathsf{ts}(\overline{\boldsymbol{x}}) \right\rangle \\ &= \sum_{\ell=1}^{\mu} \theta_{1+\lfloor(\mu-\ell)/s\rfloor}\delta_{1+(\mu-\ell) \bmod s}\overline{x}_\ell.\end{aligned}$$

Each position bit contributes exactly one term, giving $O(\mu) = O(\log L)$ extension-field operations.

General length. For $L < 2^\mu$, write its binary expansion as $L = \sum_{j=1}^{\omega} 2^{\mu_j}$, where $\mu_1 > \cdots > \mu_\omega \geq 0$ and $\omega \leq \mu = \lceil \log_2 L \rceil$. Let $L_j = \sum_{u<j} 2^{\mu_u}$. The intervals $[L_j, L_j + 2^{\mu_j})$ partition $[0, L)$; each fixes the first $\mu - \mu_j$ bits and leaves the last μ_j bits free.

By Eq. (30), the bit of weight $2^{\mu-\ell}$ belongs to coordinate $1 + \lfloor(\mu-\ell)/s\rfloor$ of the basis $\boldsymbol{\theta}$, with coefficient $\delta_{1+(\mu-\ell) \bmod s}$. Hence:

$$\iota_1 = 0, \qquad \iota_{1+2^{\mu-\ell}} = \delta_{1+(\mu-\ell) \bmod s}\theta_{1+\lfloor(\mu-\ell)/s\rfloor}, \quad \ell \in [\mu].$$

These μ single-bit labels take $O(\mu)$ operations to compute. The interval-start labels follow from $\iota_{L_1+1} = 0$ and $\iota_{L_{j+1}+1} = \iota_{L_j+1} + \iota_{1+2^{\mu_j}}$ for $j < \omega$: moving to the next interval sets one previously zero bit. Within an interval, each free bit adds its single-bit label in the same way.

Let $\overline{\xi}_j = (\mathsf{ts}((\overline{x}_1,\ldots,\overline{x}_{\mu-\mu_j})))_{1+L_j/2^{\mu_j}}$ be the tensor weight of its fixed bits. Applying the power-of-two calculation to each interval gives:

$$\left\langle \boldsymbol{\iota} \parallel \mathbf{0}_{2^\mu - L}, \mathsf{ts}(\overline{\boldsymbol{x}}) \right\rangle = \sum_{j=1}^{\omega} \overline{\xi}_j \left(\iota_{L_j+1} + \sum_{\ell=\mu-\mu_j+1}^{\mu} \iota_{1+2^{\mu-\ell}}\overline{x}_\ell \right).$$

Precompute the suffix sums of $\iota_{1+2^{\mu-\ell}}\overline{x}_\ell$, then scan the bits of L once to compute the prefix weights $\overline{\xi}_j$, reusing their common factors. Each interval contributes one stored suffix sum and one interval-start label, so its remaining cost is $O(1)$. The total is $O(\log L)$ extension-field operations and $O(\log L)$ working field elements.

Weighted evaluation of packed labels. Each label $\iota_i \in \mathbb{K}_{\mathsf{pos}}$ already combines all its position coordinates in the basis $\boldsymbol{\theta}$. Arrange the padded label vector in column-major order with κ_{pos} columns. For column weights $\overline{\boldsymbol{r}}$ and tensor row weights $\overline{\boldsymbol{l}}$, the weighted evaluation of this complete label vector is:

$$\left\langle \boldsymbol{\iota} \parallel \mathbf{0}_{2^\mu - L}, \overline{\boldsymbol{r}} \otimes \overline{\boldsymbol{l}} \right\rangle .$$

When both weights are MLE tensors, this is the ordinary MLE of the padded $\boldsymbol{\iota}$. The packed opening reduction also allows arbitrary $\overline{\boldsymbol{r}}$.

First consider all 2^μ position labels before zero padding. The column bits are the most significant bits, and the row bits are the remaining bits. The bitwise formula above splits each label into a column contribution and a row contribution, collected in $\boldsymbol{\iota}_{\mathsf{col}}$ and $\boldsymbol{\iota}_{\mathsf{row}}$. The full label matrix therefore has the following weighted evaluation:

$$\overline{\boldsymbol{l}}^\top \big(\boldsymbol{\iota}_{\mathsf{row}}\mathbf{1}^\top + \mathbf{1}\boldsymbol{\iota}_{\mathsf{col}}^\top\big)\overline{\boldsymbol{r}} = \langle \boldsymbol{\iota}_{\mathsf{row}}, \overline{\boldsymbol{l}}\rangle(\mathbf{1}^\top\overline{\boldsymbol{r}}) + (\mathbf{1}^\top\overline{\boldsymbol{l}})\langle \boldsymbol{\iota}_{\mathsf{col}}, \overline{\boldsymbol{r}}\rangle.$$

Generate the column contributions by successively adding each column-bit label to a copy of the current list, using $O(\kappa_{\mathsf{pos}})$ additions in total. For the rows, compute the weight sum and label-weighted sum for every bit suffix. Each tensor factor updates these two quantities in constant time, without requiring its entries to sum to one.

Zero padding retains only the first L positions: complete columns followed by at most one partially filled column. All complete columns reuse the two full-row sums above. For the partial column, restrict both sums to its retained row prefix. The binary interval decomposition used for general lengths computes these prefix sums from the stored suffix quantities in $O(\log L)$ operations. Combining the retained columns with $\overline{\boldsymbol{r}}$ costs $O(\kappa_{\mathsf{pos}})$ operations. Thus the weighted evaluation takes $O(\kappa_{\mathsf{pos}} + \log L)$ extension-field operations and the same number of working field elements, without division.

B.4 Complexity Bounds for the Hadamard Comparison

Table 2 compares proofs of $\boldsymbol{a} \circ \boldsymbol{b} = \boldsymbol{c}$ for length-N vectors. The degree-two constraint $a_i b_i - c_i$ gives the weighted Hadamard relation of Section 4.2, together with a linear check of the output vector. The bounds below include equality weights, this output check, witness-dependent preprocessing, and all reduction rounds, but exclude commitments and openings. We count base-field operations under schoolbook extension-field multiplication and measure peak memory in base-field coordinates, including inputs and read-only tables. Public preprocessing is accounted for separately. Table accesses are charged per coordinate, except for the unit-cost encoded-field lookups specified below.

Small-field-first. For $0 \le t_{\mathsf{skip}} \le \log_2 N$, retain the inputs and stream the first t_{skip} rounds using a quadratic grid with $3^{t_{\mathsf{skip}}}$ entries. We combine the small-field-first approach [6] with the equality-factor and streaming refinements of Bagad et al. [5, Theorem 6.1 and Section 6.2] and Dao et al. [25, Sections 5–6 and Appendix C]. For each of the $N/2^{t_{\mathsf{skip}}}$ suffixes, the grid requires $O(3^{t_{\mathsf{skip}}})$ mixed-field products. The suffix weights and conventional sumcheck tail use $O(N/2^{t_{\mathsf{skip}}})$ extension-field operations. Including contraction and storage of the grid gives:

$$\begin{aligned} \mathcal{TP} &= O\big(\kappa N(3/2)^{t_{\mathsf{skip}}} + \kappa^2 N/2^{t_{\mathsf{skip}}} + \kappa^2 3^{t_{\mathsf{skip}}}\big)\,, \\ \mathcal{SP} &= O\big(N + \kappa N/2^{t_{\mathsf{skip}}} + \kappa 3^{t_{\mathsf{skip}}}\big)\,. \end{aligned} \tag{31}$$

For this streamed implementation, the table suppresses the final terms when $N \ge 6^{t_{\mathsf{skip}}}$.

Univariate skip. We instantiate Gruen's protocol on a size-$2^{t_{\mathsf{skip}}}$ multiplicative subgroup and disjoint evaluation cosets admitting $O(2^{t_{\mathsf{skip}}} t_{\mathsf{skip}})$-time, linear-space transforms over $\mathbb{F}$ [39, Sections 5.1 and 6]. The domain assumption is stronger than $q \ge 2^{t_{\mathsf{skip}}+1} - 1$. Including the transforms, equality weighting, binding, first-round message processing, and a conventional sumcheck tail gives:

$$\begin{aligned} \mathcal{TP} &= O\big(N t_{\mathsf{skip}} + \kappa N + \kappa^2 N/2^{t_{\mathsf{skip}}} + \kappa^2 2^{t_{\mathsf{skip}}}\big)\,, \\ \mathcal{SP} &= O\big(N + \kappa N/2^{t_{\mathsf{skip}}} + \kappa 2^{t_{\mathsf{skip}}}\big)\,. \end{aligned} \tag{32}$$

The four time terms account for transforms over $N/2^{t_{\mathsf{skip}}}$ rows, mixed-field weighting and binding, the conventional tail, and first-round message processing, respectively. The final terms are absorbed when $N \ge 4^{t_{\mathsf{skip}}}$. Gruen also permits general interpolation sets [39, Section 5.1, footnote 18], but their transform cost must replace $N t_{\mathsf{skip}}$. When the domains exist, $2^{t_{\mathsf{skip}}} = \Theta(\kappa)$ and $N = \Omega(\kappa^2)$ yield $O(\kappa N)$ arithmetic and $O(N)$ RAM.

Flock's lookup-based skip. We combine the binary LDE and output-check techniques of Flock [17, Sections 4.2 and 4.4] with conventional sumcheck. Fix byte-sized chunks, $\mathbb{F}_{256} \subseteq \mathbb{E}$, and $3 \le t_{\mathsf{skip}} \le 7$, so the size-$2^{t_{\mathsf{skip}}}$ input subspace and a disjoint coset fit in the byte field. Each row uses $2^{t_{\mathsf{skip}}}/8$ lookups returning length-$2^{t_{\mathsf{skip}}}$ byte vectors. Reading and XORing these vectors costs $O(N 2^{t_{\mathsf{skip}}})$ bit operations over all rows. Byte-field products cost $O(N)$, and mixed byte-field/extension-field weighting, binding, and the linear output check cost $O(\kappa N)$.

The conventional tail, including its equality weights, costs $O(\kappa^2 N/2^{t_{\mathsf{skip}}})$. Thus:

$$\begin{aligned}\mathcal{TP} &= O\big(N2^{t_{\mathsf{skip}}} + \kappa N + \kappa^2 N/2^{t_{\mathsf{skip}}} + \kappa^2 2^{t_{\mathsf{skip}}}\big)\,,\\ \mathcal{SP} &= O\big(N + \kappa N/2^{t_{\mathsf{skip}}} + \kappa 2^{t_{\mathsf{skip}}}\big)\,.\end{aligned} \tag{33}$$

The storage bound includes the compressed LDE table, of size $O(2^{t_{\mathsf{skip}}})$ bytes for fixed byte chunks. The final terms are absorbed when $N \geq 4^{t_{\mathsf{skip}}}$. The source implementation uses $t_{\mathsf{skip}} = 6$, $\kappa = 128$, and a 16 KiB compressed LDE table. Compression improves storage and cache locality without changing the vector-XOR count. This instantiation isolates the skip construction with a conventional tail, excluding Flock's friendly-challenge and system optimizations.

Binary-tower instantiations. Assume $\mathbb{F} = \mathbb{F}_2$ and $N \geq \kappa$. The Dao–Thaler row uses their structured equality-weight construction [26, Sections 7–8 and Algorithm 2] with conventional sumcheck on an $O(N)$-length table. This composition costs $O(\kappa^2 N)$ time and $O(\kappa N)$ memory.

For Liu–Zhang, use every other level of a Wiedemann tower in the tower zerocheck [48, Section 4.2, Theorem 8 and Protocol 5]. Set $h = \lfloor \log_2 \kappa/2 \rfloor$, so the folded entries at stage $j < h$ have 4^j base-field coordinates. Schoolbook local products and mixed top-field weighting give:

$$\sum_{j=0}^{h-1} O\left(\frac{N}{2^j}\big((4^j)^2 + \kappa 4^j\big)\right) + O\left(\frac{\kappa^2 N}{2^h}\right) = O(\kappa^{3/2} N).$$

To obtain the table's memory bound, generate deterministic prefix weights on demand using the special-generator arithmetic of Dao–Thaler [26, Section 8]. Retain only the current folded inputs and the random-suffix weight tables. Their peak storage is:

$$O\left(N + \max_{0 \leq j \leq h} \frac{N4^j}{2^j} + \frac{\kappa N}{2^h}\right) = O(N\sqrt{\kappa}).$$

The memory bound follows from this streaming schedule.

RMFE–Aurora. We apply the Boolean-R1CS compiler of Cascudo and Giunta [20, full version, Theorem 4 and Appendix A.2] to the N sparse constraints $a_i b_i = c_i$. There are $3N$ variables and $O(N)$ nonzero matrix entries. Assume constant-rate RMFE packing and codes, linear-space quasilinear Reed–Solomon transforms, security $O(\kappa)$, and sufficient instance size to absorb security padding. The compiler's algebraic checks and encoding then use $O(\kappa^2 N + \kappa N \log N)$ base-field operations and $O(\kappa N)$ stored coordinates. These bounds include the compiler's Reed–Solomon encoding.

Packed sumcheck. We use the revised packed ZeroCheck construction of Wei et al. [65, Theorem 3, Section 5.1 and Appendix H.1]. Partition each input into k instances of length $N_0 = N/k$. For fixed constraint degree two, the source gives $O(k^2 N_0) = O(kN)$ base-field operations, counting both multiplications and additions. Its stated parameter condition is that $\mathbb{F}$ is prime and $q > 3(2k-1)+1$.

The equality factor is included in this ZeroCheck cost. We use the revision entitled *SNARKs over Small Prime Fields without Extension Field Multiplication*.

For memory, stream the per-suffix forward-difference computations and accumulate the first-round messages [65, Appendices A and B.3]. The witness, equality tables, and folded inputs occupy $O(N)$ base-field elements; the current challenge matrices and first-round accumulators occupy $O(k^2)$. This gives $O(N + k^2)$ memory. The packing parameter k follows the source's repetition-dependent soundness bound and is distinct from the extension degree κ.

`Mikros` and the small-field cost model. For $q \geq 2\kappa - 1$, let $N = m\kappa$ and $m \geq \kappa$. Proposition 2, together with the linear evaluation of the output vector, gives $O(N\kappa + \kappa^3) = O(N\kappa)$ operations and $O(N + \kappa^2) = O(N)$ stored coordinates.

For smaller fields, Section 4.2 uses $\mathbb{F} \subseteq \mathbb{K} \subseteq \mathbb{E}$. Here $t = [\mathbb{K} : \mathbb{F}]$ is the interpolation-field degree, not the skip parameter t_{skip} of the rows above. Choose the extension degree and interpolation field as in that section. Under its unit-cost encoded-$\mathbb{K}$-lookup convention, the bounds are:

$$\begin{aligned} \mathcal{TP}_{\text{online}} &= O(N\kappa + \kappa^3), \\ \mathcal{SP}_{\text{total}} &= O\big(N + \kappa^2 t + t q^{2t}\big)\,. \end{aligned} \tag{34}$$

Constructing the field-operation tables costs $O(t^2 q^{2t})$ base-field operations. The public tables and cached interpolation data are reusable and included in peak memory. The $O(N\kappa)$ online bound for smaller fields uses this unit-lookup convention. In a coordinate-charged model, reading a $\mathbb{K}$-element entails t base-field coordinates. Moreover, $N \geq \kappa^2$ alone does not absorb the tables in the storage bound.

All parameter choices must satisfy the target soundness. When the stated large-N conditions fail, use the full bounds in Eq. (31), Eq. (32), and Eq. (33).

B.5 Indexed Lookup via Packed Permutation

For the lookup comparison in Table 2, let $\boldsymbol{u} \in \mathbb{F}^N$ contain the lookup values, $\boldsymbol{w} \in \mathbb{F}^M$ the table values, and $\boldsymbol{\sigma} \in [M]^N$ the public indices. The lookup relation is $u_j = w_{\sigma_j}$ for every $j \in [N]$. The index vector and counters are preprocessed; the table values may depend on the witness. We compare algebraic reductions, excluding commitments and terminal opening reductions on both sides.

Lasso in vector form. Let $\zeta_j = |\{i < j : \sigma_i = \sigma_j\}|$ count the previous occurrences of σ_j, and let $\chi_i = |\{j \in [N] : \sigma_j = i\}|$ count all occurrences of table index i. Treating positions and counters as integer labels, the lookup relation is equivalent to equality of the column multisets of the following two matrices [60, full version, Claim 2 and Section 5]:

$$\begin{pmatrix} \boldsymbol{\sigma}^\top & (1,\dots,M) \\ \boldsymbol{u}^\top & \boldsymbol{w}^\top \\ \boldsymbol{\zeta}^\top & \boldsymbol{\chi}^\top \end{pmatrix} \overset{\mathsf{perm}}{=} \begin{pmatrix} (1,\dots,M) & \boldsymbol{\sigma}^\top \\ \boldsymbol{w}^\top & \boldsymbol{u}^\top \\ \mathbf{0}_M^\top & (\boldsymbol{\zeta} + \mathbf{1}_N)^\top \end{pmatrix}.$$

At each table position, the counter chain starts at zero and ends at χ_i; matching consecutive links forces every lookup value at that position to equal w_i. Lasso

checks this permutation using randomized tuple compression and grand products. Additive counters require field characteristic greater than N; the multiplicative-counter variant below handles small characteristic. These checks use $O(M+N)$ challenge-field operations. Schoolbook arithmetic gives $O((M+N)\kappa^2)$ base-field operations. Materializing its product trees and folding tables uses $O((M+N)\kappa)$ base-field coordinates.

Packed lookup reduction. We use the value–position encoding of Section 5.1. Let $q = |\mathbb{F}|$, and choose d as the least power of two such that $d \geq \lceil \log_q M \rceil + 1$ and $q^d - 1 > N$. Assume κ is a power of two with $4d \mid \kappa$, and set $\mathbb{K}_{\mathsf{pos}} = \mathbb{F}_{q^d} \subseteq \mathbb{E}$. The position labels satisfy $\varsigma_j = \iota_{\sigma_j}$, and the joint encodings are $\boldsymbol{a} = \boldsymbol{\varsigma} + \theta_d \boldsymbol{u}$ and $\boldsymbol{b} = \boldsymbol{\iota} + \theta_d \boldsymbol{w}$.

Fix $e \in \mathbb{K}_{\mathsf{pos}}^{\times}$ of order greater than N. Replace each counter c by e^c, so the initial counter becomes 1 and each increment becomes multiplication by e. This is the multiplicative-counter variant of Lasso [60, full version, Remark 2]. Precompute $e^{\boldsymbol{\varsigma}}$ and $e^{\boldsymbol{\chi}}$ by element-wise exponentiation. This encoding is injective on the counter range $\{0, \ldots, N\}$. Choose $\vartheta \in \mathbb{E}$ of degree two over $\mathbb{K}_{\mathsf{pos}}$. Combining each value–position encoding with its counter in the basis $(1, \vartheta)$ gives the equivalent scalar-vector permutation:

$$(\boldsymbol{a} \parallel \boldsymbol{b}) + \vartheta(e^{\boldsymbol{\varsigma}} \parallel e^{\boldsymbol{\chi}}) \overset{\mathsf{perm}}{=} (\boldsymbol{b} \parallel \boldsymbol{a}) + \vartheta(\mathbf{1}_M \parallel e\, e^{\boldsymbol{\varsigma}}).$$

Pad the lookup-value and table segments separately to a common power-of-two length, using identical zero padding on both sides. This changes the length by a constant factor and preserves the tensor structure under the segment swap. Apply Section 4.3 with base field $\mathbb{K}_{\mathsf{pos}}(\vartheta)$ and effective packing width $\kappa' = [\mathbb{E} : \mathbb{K}_{\mathsf{pos}}(\vartheta)] = \kappa/(2d) \geq 2$. The encoded width $2d \log_2 q$ includes the value, position, and counter. With the choice of d above, $d = \Theta(1 + \log_q \max\{M, N\})$. For $\log_2 |\mathbb{E}| \simeq \lambda$, this gives $\kappa' = \Theta(\lambda/(\log_2 q + \log_2 \max\{M, N\}))$, as in Table 2. By linearity, the terminal claims reduce to weighted evaluations of the original $\boldsymbol{u}, \boldsymbol{w}$ and preprocessed public arrays. The PCS proofs of these terminal claims are outside the algebraic cost comparison.

Costs and soundness. Assume $M + N \geq (2d\kappa')^2$ and $q^{2d} \geq 2\kappa' - 1$, so all interpolation nodes lie in $\mathbb{K}_{\mathsf{pos}}(\vartheta)$. Proposition 3 applies with input length $O(M+N)$ and packing width κ'. First count operations over $\mathbb{K}_{\mathsf{pos}}(\vartheta)$. The prover uses $O((M+N)\kappa' \log_2 \kappa' + (\kappa')^4)$ operations and $O(M + N + (\kappa')^3)$ field elements of memory. We measure the final bounds over the original field $\mathbb{F}$. Since $[\mathbb{K}_{\mathsf{pos}}(\vartheta) : \mathbb{F}] = 2d$, schoolbook multiplication costs $O((2d)^2)\,\mathsf{bb}$, and each stored element occupies $2d$ base-field coordinates. Substituting these costs and using $\kappa = 2d\kappa'$ gives:

$$\begin{aligned} \mathcal{TP} &= O\big((M+N)d\kappa \log_2 \kappa' + d^2(\kappa')^4\big)\ \mathsf{bb}, \\ \mathcal{SP} &= O\big(d\big(M + N + (\kappa')^3\big)\big)\ \mathbb{F}. \end{aligned}$$

The memory bound includes inputs, public counter arrays, and interpolation data. If $M + N \geq (\kappa')^3$, the parameter-only terms are absorbed, leaving $O((M+$

$N)d\kappa \log_2 \kappa')\,\mathsf{bb}$ time and $O((M+N)d)\,\mathbb{F}$ memory. The factors $(2d)^2$ and $2d$ account for arithmetic and storage over $\mathbb{F}$, respectively; no unit-cost field-operation tables are assumed here. The permutation bound gives algebraic error $O((M+N)/|\mathbb{E}|)$; its lower-degree recursive checks are absorbed under the size assumption above. The basic Lasso instantiation has the same asymptotic algebraic error [60, full version, Section 5]. Commitment and opening errors must be added in either composition.

C Deferred Proofs

This appendix proves the propositions and theorems in Sections 4 and 5 and the packed PCS proposition in Appendix B.2, using the notation of each statement.

C.1 Proof of Proposition 1

Proof. **Completeness.** Expand the Laurent polynomial in Eq. (6):

$$f(X) = \sum_{\tau=-(\kappa-1)}^{\kappa-1} f_\tau X^\tau, \qquad f_\tau = \sum_{\substack{j,j' \in [\kappa] \\ j-j'=\tau}} \langle \boldsymbol{a}^{(j)}, \boldsymbol{b}^{(j')} \rangle.$$

Its constant coefficient is $f_0 = \langle \boldsymbol{a}, \boldsymbol{b} \rangle$. Thus the auxiliary polynomials in Algorithm 1 satisfy Eq. (7) for a correct inner-product claim. Evaluation at $\overline{x} \neq 0$ gives the length-m claim in Eq. (8).

The terminal values of Fact 1 have the matrix form:

$$\overline{v}_1 = \mathsf{ts}(\overline{\boldsymbol{y}})^\top \boldsymbol{A} \overline{\boldsymbol{\rho}}_A, \quad \overline{\boldsymbol{\rho}}_A = (1, \overline{x}, \ldots, \overline{x}^{\kappa-1})^\top,$$
$$\overline{v}_2 = \mathsf{ts}(\overline{\boldsymbol{y}})^\top \boldsymbol{B} \overline{\boldsymbol{\rho}}_B, \quad \overline{\boldsymbol{\rho}}_B = (1, \overline{x}^{-1}, \ldots, \overline{x}^{-(\kappa-1)})^\top.$$

Both have tensor-structured row weights and fit the packed-opening relation in Eq. (18).

Soundness. Suppose $c \neq \langle \boldsymbol{a}, \boldsymbol{b} \rangle$, and fix the degree-$(\kappa-2)$ polynomials h_+, h_- before sampling $\overline{x}$. The Laurent residual $f(X) - c - X h_+(X) - X^{-1} h_-(X^{-1})$ has a nonzero constant coefficient. Multiplying by $X^{\kappa-1}$ gives a nonzero polynomial of degree at most $2\kappa-2$, so the evaluation check accepts with probability at most $(2\kappa-2)/(|\mathbb{E}|-1)$. Conditioned on a false evaluated claim, Fact 1 contributes at most $2\log_2 m/|\mathbb{E}|$. A union bound gives the stated error, excluding the terminal packed openings.

Efficiency. Each ordered pair of distinct columns contributes a length-m base-field inner product to the correction coefficients, for $m\kappa(\kappa-1) = N(\kappa-1)$ base-field multiplications. Given the powers of $\overline{x}$ and $\overline{x}^{-1}$, packing both matrices uses $2N\,\mathsf{be}$. Generating these powers and evaluating the corrections costs $O(\kappa)\,\mathsf{ee}$; Fact 1 adds $(4m + O(\log m))\,\mathsf{ee}$. Thus the total arithmetic cost is:

$$O(N\kappa)\,\mathsf{bb} + O(N)\,\mathsf{be} + O(m+\kappa)\,\mathsf{ee} = O(N\kappa + \kappa^3)\,\mathsf{bb}.$$

Streaming the cross-column inner products retains only $O(\kappa)$ base-field correction coefficients. The witness matrices and packed vectors use $O(N)$ base-field coordinates, and the challenge powers use $O(\kappa^2)$. An in-place inner-product reduction therefore has peak memory $O(N + \kappa^2)\mathbb{F}$. □

C.2 Proof of Proposition 2

Proof. **Completeness.** The honest column message is $\overline{\boldsymbol{d}}^\top = \bar{\boldsymbol{l}}^\top(\boldsymbol{A} \circ \boldsymbol{B})$, so the scalar check $\bar{c} = \langle \overline{\boldsymbol{d}}, \overline{\boldsymbol{r}} \rangle$ accepts. At every interpolation node, the residual satisfies:

$$f(\gamma_j) = \langle \bar{\boldsymbol{l}}, \boldsymbol{a}^{(j)} \circ \boldsymbol{b}^{(j)} \rangle - \overline{d}^{(j)} = 0, \qquad j \in [\kappa].$$

Hence $f = \Psi h$ for a polynomial h of degree at most $\kappa - 2$. Evaluating this identity at $\overline{x}$ gives:

$$\langle \overline{\boldsymbol{a}}, \bar{\boldsymbol{b}} \circ \bar{\boldsymbol{l}} \rangle = \bar{c}_x + \Psi(\overline{x}) h(\overline{x}).$$

The terminal values of Fact 1 satisfy:

$$\overline{v}_1 = \mathsf{ts}(\overline{\boldsymbol{y}})^\top \boldsymbol{A} \overline{\boldsymbol{\rho}}(\overline{x}), \qquad \overline{v}_2 \quad = \big(\bar{\boldsymbol{l}} \circ \mathsf{ts}(\overline{\boldsymbol{y}})\big)^\top \boldsymbol{B} \overline{\boldsymbol{\rho}}(\overline{x}).$$

With the supplied row weight written as $\bar{\boldsymbol{l}} = \bigotimes_{j=1}^{\log_2 m} (\bar{l}_{j,0}, \bar{l}_{j,1})^\top$, its product with the MLE weight factors as:

$$\bar{\boldsymbol{l}} \circ \mathsf{ts}(\overline{\boldsymbol{y}}) = \bigotimes_{j=1}^{\log_2 m} \big(\bar{l}_{j,0}(1 - \overline{y}_j), \bar{l}_{j,1} \overline{y}_j\big)^\top.$$

Both terminal claims fit Eq. (18), with Lagrange column weights and tensor-structured row weights.

Soundness. Suppose the weighted Hadamard claim is false but the scalar check accepts. Then the column message differs from $\bar{\boldsymbol{l}}^\top(\boldsymbol{A} \circ \boldsymbol{B})$ in at least one coordinate, and $f(\gamma_j) \neq 0$ for some j. For every degree-$(\kappa - 2)$ polynomial h fixed before $\overline{x}$, the residual $f - \Psi h$ is therefore nonzero and has degree at most $2\kappa - 2$. A random $\overline{x} \in \mathbb{E} \setminus \mathcal{H}$ makes this residual vanish with probability at most $(2\kappa - 2)/(|\mathbb{E}| - \kappa)$. Fact 1 contributes at most $2 \log_2 m / |\mathbb{E}|$, giving:

$$\epsilon_{\mathsf{Had}} \leq \frac{2\kappa - 2}{|\mathbb{E}| - \kappa} + \frac{2 \log_2 m}{|\mathbb{E}|} \leq \frac{2\kappa - 2}{|\mathbb{E}| - \kappa} + \frac{2 \log_2 N}{|\mathbb{E}|}.$$

The first inequality bounds Algorithm 2; the second gives the stated bound. The argument conditions on correct terminal evaluations, whose proofs are handled by the calling protocol. The argument applies unchanged when the interpolation nodes lie in $\mathbb{K} \subseteq \mathbb{E}$.

Efficiency over the base field. First suppose $\mathbb{K} = \mathbb{F}$. Computing $\overline{\boldsymbol{d}}$ uses $O(N)\,\mathsf{bb} + O(N)\,\mathsf{be}$. At each of the $\kappa - 1$ additional interpolation nodes, evaluating both witness encodings takes $O(N)\,\mathsf{bb}$. Pointwise multiplication and contraction with $\bar{\boldsymbol{l}}$ then take $O(m)\,\mathsf{bb} + O(m)\,\mathsf{be}$. Over all nodes, these operations cost

$O(N\kappa)\,\mathsf{bb} + O(N)\,\mathsf{be}$. Evaluating g_C, interpolating the residual, and dividing by Ψ contribute $O(\kappa^2)\,\mathsf{be}$.

After $\overline{x}$ is sampled, packing both matrices costs $O(N)\,\mathsf{be}$. Expanding the row factors, forming $\overline{\boldsymbol{b}} \circ \overline{\boldsymbol{l}}$, and running Fact 1 cost $O(m)\,\mathsf{ee}$. The terminal row factors require only element-wise multiplication of their two-entry representations. With precomputed Lagrange coefficients, computing the public column weights and polynomial evaluations costs $O(\kappa^2)\,\mathsf{be} + O(\kappa)\,\mathsf{ee}$. The total is therefore:

$$O(N\kappa)\,\mathsf{bb} + O(N+\kappa^2)\,\mathsf{be} + O(m+\kappa)\,\mathsf{ee} = O(N\kappa+\kappa^3)\,\mathsf{bb}.$$

Intermediate-field tables. Let $[\mathbb{K}:\mathbb{F}] = t > 1$, with $t \mid \kappa$ and $|\mathbb{K}| \geq 2\kappa - 1$. Use the addition and multiplication tables specified in Section 4.2, charging each encoded-$\mathbb{K}$ table access as $O(1)$. Evaluating the witness encodings at all additional nodes requires $O(N\kappa)$ table operations. In $\mathbb{K}$-coordinates, each extension-field element has length κ/t, so weighting and accumulating the $O(N)$ local products uses $O(N\kappa/t)$ lookups. The $O(\kappa^2)$ mixed operations for g_C, interpolation, and division cost $O(\kappa^3/t)$. Changing between the tower coordinates and a fixed base-field basis for the $O(m+\kappa)$ extension-field values costs $O(m\kappa^2+\kappa^3)\,\mathsf{bb}$. The remaining packing and inner-product steps retain their $O(N\kappa+\kappa^3)\,\mathsf{bb}$ bound. Thus the same online time bound holds, excluding construction of the fixed tables.

Memory. Processing rows and sample points sequentially retains $O(\kappa)$ extension-field residual values, followed by $O(\kappa)$ extension-field coefficients after interpolation. The packed vectors and inner-product reduction use $O(m)$ extension-field elements. Including the witness matrices, this is $O(N+\kappa^2)$ base-field coordinates. The fixed interpolation data use $O(\kappa^2 t)$ coordinates, and the tables use $O(tq^{2t})$ when $t > 1$; no field tables are needed when $t = 1$. The peak memory is therefore $O\big(N + \kappa^2 t + (t-1)q^{2t}\big)\mathbb{F}$. □

C.3 Proof of Proposition 3

Proof. **Completeness.** The coefficient tree in Eq. (11) computes the products of 2^τ shifted witness entries in each column. Evaluating its last layer at $\overline{z}$ gives m extension-field entries, whose product is $\mathsf{GrandProd}(\overline{z}\mathbf{1} + \boldsymbol{a})$. The two grand products agree whenever $\boldsymbol{a}$ and $\boldsymbol{b}$ contain the same multiset of entries.

For a transition from level $\tau+1$ to level τ, Lagrange interpolation gives:

$$\boldsymbol{g}_{\tau,U}(Z,\gamma_j) \circ \boldsymbol{g}_{\tau,D}(Z,\gamma_j) = \boldsymbol{C}_{\tau+1}(Z)_{*,j}, \qquad j \in [\kappa].$$

For the honest column claims $\overline{\boldsymbol{u}}_{\tau+1}^\top = \overline{\boldsymbol{l}}_{\tau+1}^\top \boldsymbol{C}_{\tau+1}(\overline{z})$, the residual $f(X)$ therefore vanishes on $\mathcal{H}$. Since its degree is at most $2\kappa-2$, it satisfies $f(X) = \Psi(X)h(X)$ with $\deg h \leq \kappa - 2$. At an additional node, coefficient aggregation commutes with evaluation at $\overline{z}$:

$$\begin{aligned} F_{\tau+1,j}(\overline{z}) &= \sum_{i=0}^{2^{\tau+1}} \langle \overline{\boldsymbol{l}}_{\tau+1}, \boldsymbol{p}_{\tau+1,j,i} \rangle \overline{z}^i \\ &= \left\langle \overline{\boldsymbol{l}}_{\tau+1}, \boldsymbol{g}_{\tau,U}(\overline{z},\gamma'_j) \circ \boldsymbol{g}_{\tau,D}(\overline{z},\gamma'_j) \right\rangle. \end{aligned}$$

Algorithm 3 therefore recovers h from $\kappa-1$ samples. Reencoding $\boldsymbol{C}_\tau(Z)$ at each level keeps the X-degree of $\boldsymbol{g}_\tau(Z,X)$ at most $\kappa-1$.

Each evaluated relation is proved by weighted sumcheck. For length-n child vectors and a tensor weight $\bar{\boldsymbol{l}}$, the sumcheck polynomial is:

$$\widetilde{\bar{\boldsymbol{l}}}(\boldsymbol{Y})\,\widetilde{\overline{\boldsymbol{u}}}(\boldsymbol{Y})\,\widetilde{\overline{\boldsymbol{v}}}(\boldsymbol{Y}). \tag{35}$$

It has individual degree at most three. The two child claims are evaluated at the same point $\overline{\boldsymbol{y}}$. Their affine combination with a fresh $\overline{y}_0$ is the MLE claim on the concatenated parent vector at $\overline{y}_0 \,\|\, \overline{\boldsymbol{y}}$. This preserves the recursive matrix claim $\overline{v}_\tau = \bar{\boldsymbol{l}}_\tau^\top \boldsymbol{C}_\tau(\overline{z})\overline{\boldsymbol{r}}_\tau$. At level zero, the row MLE weights and the column Lagrange weights both sum to one. Removing the shift leaves:

$$\overline{v}_0 - \overline{z} = \bar{\boldsymbol{l}}_0^\top \boldsymbol{A}\overline{\boldsymbol{r}}_0.$$

This is the packed-opening interface of Eq. (18). The same analysis applies to $\boldsymbol{B}$.

Soundness. For a weighted sumcheck on length-n vectors, Eq. (35) contributes at most $3\log_2 n/|\mathbb{E}|$. If at least one of the two terminal child claims is false, their errors form a nonzero affine polynomial in the fresh merge challenge $\overline{y}_0$. The merged claim is then correct with probability at most $1/|\mathbb{E}|$. A transition thus preserves a false claim except with probability at most $(3\log_2 n+1)/|\mathbb{E}|$. The extension-field tree has child lengths $1,2,\ldots,m/2$. Summing its errors gives:

$$\sum_{i=0}^{\log_2 m-1} \frac{3i+1}{|\mathbb{E}|} = \frac{\frac{3}{2}\log_2 m(\log_2 m-1)+\log_2 m}{|\mathbb{E}|}.$$

At coefficient level $\tau+1$, a false incoming matrix claim either fails the scalar check against $\overline{\boldsymbol{u}}_{\tau+1}$, or makes at least one column claim false. In the latter case $f(X)$ does not vanish on all of $\mathcal{H}$, and $f(X)-\Psi(X)h(X)$ is nonzero for every claimed h of degree at most $\kappa-2$. Since the message is fixed before $\overline{x}$ is sampled, this polynomial vanishes at $\overline{x}$ with probability at most $(2\kappa-2)/(|\mathbb{E}|-\kappa)$. The subsequent weighted sumcheck and merge have child length $m/2^{\tau+1}$ and error at most $(3\log_2(m/2^{\tau+1})+1)/|\mathbb{E}|$. Summing over $\tau=0,\ldots,\log_2\kappa-1$ and adding the extension-field-tree error gives the stated ϵ_{GP}. This bound is conditional on a valid proof of the terminal linear claim.

If the committed vectors are not permutations of one another, their fingerprint difference is the nonzero polynomial:

$$\prod_{i=1}^{N}(Z+a_i)-\prod_{i=1}^{N}(Z+b_i).$$

Its degree is at most $N-1$, so the true fingerprints coincide with probability at most $\min\{1,(N-1)/|\mathbb{E}|\}$. Otherwise, at least one claimed grand product is

false. A union bound over the two reductions and the proofs of their terminal claims gives:

$$\epsilon_{\mathsf{Perm}} \leq \min\left\{1, \frac{N-1}{|\mathbb{E}|}\right\} + 2\epsilon_{\mathsf{GP}} + \epsilon_{\mathsf{PCS}}.$$

Efficiency over $\mathbb{F}$. At level $\tau \in [\log_2 \kappa]$, the coefficient matrix of $\boldsymbol{C}_{\tau-1}(Z)$ has $O(m)$ row-degree pairs. Evaluating its κ-column encodings at $\kappa - 1$ additional nodes costs $O(N\kappa)\,\mathsf{bb}$. Multiplying the resulting degree-$2^{\tau-1}$ polynomials costs $O(m\kappa 2^{\tau})\,\mathsf{bb}$. These products, including construction of the coefficient tree, sum to $O(N\kappa)\,\mathsf{bb}$.

The off-domain products and column claims contain $O(N)$ coefficients per level, whose aggregation costs $O(N)\,\mathsf{be}$. There are $O(\kappa)$ aggregated polynomials of degree at most 2^{τ}, so their subsequent evaluation costs:

$$\sum_{\tau=1}^{\log_2 \kappa} O(\kappa 2^{\tau})\,\mathsf{ee} = O(\kappa^2)\,\mathsf{ee}.$$

The shared powers of $\overline{z}$ require $O(\kappa)\,\mathsf{ee}$ once. Forming the vectors for the short weighted sumchecks costs $O(N)\,\mathsf{be} + O(m)\,\mathsf{ee}$ per coefficient level. Indeed, the algorithm contracts the coefficient matrices first:

$$\boldsymbol{g}_{\tau}(\overline{z}, \overline{x}) = \sum_{i=0}^{2^{\tau}} \big([Z^i]\boldsymbol{C}_{\tau}(Z)\big)\overline{\boldsymbol{\rho}}(\overline{x})\,\overline{z}^{\,i}.$$

There are $O(N)$ base-field coefficients to contract and $O(m)$ resulting extension-field coefficients to evaluate. The short sumchecks and the extension-field tree have total length $O(m)$. Interpolation and computation of public weights contribute $O(\kappa^3)\,\mathsf{bb}$ per level. Using $m \geq \kappa$, the total is:

$$O(N\kappa \log_2 \kappa + \kappa^4)\,\mathsf{bb}.$$

The κ^4 term accounts for evaluation of the aggregated extension-field polynomials.

Intermediate-field implementation. For $q < 2\kappa - 1$, use the addition and multiplication tables of Section 4.2 over $\mathbb{K}$, with the same unit-cost lookup convention. Precomputed Lagrange evaluations allow the $O(N\kappa)$ encoding operations at each level to be performed over $\mathbb{K}$. The row-polynomial products likewise use $O(N\kappa)$ table operations in total. The coefficients of $\boldsymbol{C}_{\tau}(Z)$ remain in $\mathbb{F}$; only the off-domain encodings and products use $\mathbb{K}$.

In a compatible tower basis, a mixed $\mathbb{K} \times \mathbb{E}$ product and its accumulation cost $O(\kappa)$ in base-field coordinates. Schoolbook multiplication over $\mathbb{K}$, using the tables for coordinate products, costs $O(\kappa^2/t)$ per extension-field multiplication. The preceding count becomes $O\big(N\kappa \log_2 \kappa + \kappa^4/t\big)\,\mathsf{bb}$, which implies the coarser bound in the proposition.

Space. One coefficient-tree level contains $O(N)$ base-field coefficients. The levels can be reconstructed from $\boldsymbol{C}_0(Z)$ during the reverse traversal: reconstructing

level τ costs $O(N2^{\tau})\,\mathsf{bb}$, and these costs sum to $O(N\kappa)\,\mathsf{bb}$. Thus the prover need not retain all coefficient levels. Processing one additional node at a time uses $O(mt) \subseteq O(N)$ coordinates for its row polynomials. The aggregated polynomials use $O(\kappa^2)$ extension-field coefficients, or $O(\kappa^3)$ base-field coordinates. The extension-field tree and sumcheck tables use $O(m\kappa) = O(N)$ coordinates. Including interpolation data and the intermediate-field tables, peak memory is $O(N + \kappa^3 + tq^{2t})$ base-field coordinates for $t > 1$. When $t = 1$, no field tables are needed, giving $O(N + \kappa^3)$ base-field coordinates. Both cases are $O\big(N + \kappa^3 + (t-1)q^{2t}\big)$ base-field coordinates. The time bounds exclude preprocessing, commitments, and terminal openings. □

C.4 Proof of Proposition 4

Proof. **Binding and completeness.** Each entry of $\boldsymbol{A}\overline{\boldsymbol{\beta}}$ uniquely determines the corresponding row of $\boldsymbol{A}$. This injective encoding preserves binding and, when provided by the underlying PCS, commitment hiding.

For an honest prover, $\boldsymbol{U} = \boldsymbol{\Xi}^{\top}\boldsymbol{A}$, so Eq. (19) and its random projection hold. We show that the verifier also computes the public terminal value exactly.

By Eq. (23), multiplication by $\overline{\alpha}$ acts on a column of base-field coordinates through $\boldsymbol{M}_{\overline{\alpha}}$. This map is $\mathbb{F}$-linear in $\overline{\alpha}$, and $\boldsymbol{M}_1 = \boldsymbol{I}_{\kappa}$. We prove by backward induction that $\boldsymbol{W}_j = \boldsymbol{Y}_j^{\top}\boldsymbol{\Xi}_j$, first for $\overline{\boldsymbol{\xi}} = \mathsf{ts}(\overline{\boldsymbol{x}})$. Normalize $\overline{\boldsymbol{\beta}}$ such that $\overline{\beta}_1 = 1$, and thus the empty suffix has coordinate row $\boldsymbol{e}_1^{\top}$, giving $\boldsymbol{Y}_{\ell+1}^{\top}\boldsymbol{\Xi}_{\ell+1} = \boldsymbol{e}_1\boldsymbol{e}_1^{\top} = \boldsymbol{W}_{\ell+1}$. For the induction step, apply the block identities in Eq. (24) and the induction hypothesis:

$$\begin{aligned}\boldsymbol{Y}_j^{\top}\boldsymbol{\Xi}_j &= \boldsymbol{M}_{1-\overline{y}_j}\boldsymbol{W}_{j+1}\boldsymbol{M}_{1-\overline{x}_j}^{\top} + \boldsymbol{M}_{\overline{y}_j}\boldsymbol{W}_{j+1}\boldsymbol{M}_{\overline{x}_j}^{\top}\\ &= \boldsymbol{W}_{j+1}\boldsymbol{M}_{1-\overline{x}_j}^{\top} + \boldsymbol{M}_{\overline{y}_j}\boldsymbol{W}_{j+1}\boldsymbol{M}_{2\overline{x}_j-1}^{\top}\\ &= \boldsymbol{W}_j.\end{aligned}$$

The second equality uses linearity of $\boldsymbol{M}$; the last is Eq. (25). Replacing $(1-\overline{x}_j, \overline{x}_j)$ by $(\overline{\xi}_0^{(j)}, \overline{\xi}_1^{(j)})$ gives the same induction for arbitrary tensor factors. Taking $j = 1$ gives $\boldsymbol{W} = \boldsymbol{W}_1 = \boldsymbol{Y}^{\top}\boldsymbol{\Xi}$ and hence:

$$\overline{\boldsymbol{\beta}}^{\top}\boldsymbol{W}\overline{\boldsymbol{r}} = (\boldsymbol{Y}\overline{\boldsymbol{\beta}})^{\top}\boldsymbol{\Xi}\overline{\boldsymbol{r}} = \mathsf{ts}(\overline{\boldsymbol{y}})^{\top}\boldsymbol{\Xi}\overline{\boldsymbol{r}}.$$

Thus Eq. (22) computes the terminal MLE evaluation of the public weight vector exactly. Completeness follows from Fact 1 and the underlying PCS opening. For $m = 1$, the initialization suffices, with no recurrence or sumcheck rounds.

For smaller fields, the intermediate-field construction of Section 4.2 still produces column weights in $\mathbb{E}^{\kappa}$ and tensor-structured row weights. Its terminal claims therefore satisfy the same opening interface in Eq. (18).

Knowledge soundness. First fix $\boldsymbol{A}$ before the reduction challenges and assume exact underlying MLE evaluations. Suppose that Eq. (18) is false. If the first scalar check accepts, then $\overline{\boldsymbol{v}}^{\top} \neq \overline{\boldsymbol{\xi}}^{\top}\boldsymbol{A}$, hence $\boldsymbol{U} \neq \boldsymbol{\Xi}^{\top}\boldsymbol{A}$. Injectivity of basis

packing makes $(\boldsymbol{U} - \boldsymbol{\Xi}^\top \boldsymbol{A})\overline{\boldsymbol{\beta}}$ a nonzero column of $\mathbb{E}^\kappa$, fixed before $\overline{\boldsymbol{r}}$ is sampled. Its inner product with the uniform $\overline{\boldsymbol{r}}$ vanishes with probability $1/|\mathbb{E}|$. Outside this event, Eq. (20) is false.

Except with probability $2\ell/|\mathbb{E}|$, Fact 1 then leaves a false terminal claim in Eq. (21). The recurrence computes $\boldsymbol{W}$ from the public tensor factors and the terminal point, so a false MLE claim for the public weight vector is rejected deterministically. If that check passes, the witness MLE claim must be false. Thus the algebraic reduction has error at most $(1 + 2\ell)/|\mathbb{E}|$.

The underlying PCS extractor recovers the committed polynomial and hence its evaluation vector $\overline{\boldsymbol{a}}$. Decoding in $\overline{\boldsymbol{\beta}}$ gives the unique $\boldsymbol{A} \in \mathbb{F}^{m\times\kappa}$ satisfying $\overline{\boldsymbol{a}} = \boldsymbol{A}\overline{\boldsymbol{\beta}}$. The extractor returns $\boldsymbol{A}$ if Eq. (18) holds, and fails otherwise. This decoding and check take polynomial time. Lemma 3 adds the underlying PCS knowledge-soundness error ϵ_{pc} for the adaptively chosen terminal claim, yielding Eq. (26).

Prover complexity. Expanding $\overline{\boldsymbol{\xi}}$ and reading its basis coordinates gives $\boldsymbol{\Xi}$ in $O(m)$ extension-field operations. The matrix product $\boldsymbol{U} = \boldsymbol{\Xi}^\top \boldsymbol{A}$ takes κN base-field multiplications. Its columns give the coordinates of $\overline{\boldsymbol{v}}^\top = \overline{\boldsymbol{\beta}}^\top \boldsymbol{U}$, so packing requires only a change of representation.

Computing $\boldsymbol{\Xi}\overline{\boldsymbol{r}}$ takes N mixed-field multiplications; the extension-field inner-product reduction takes $4m + O(\ell)$ extension-field multiplications. Packing the rows of $\boldsymbol{U}$ and taking their inner product with $\overline{\boldsymbol{r}}$ computes $\bar{c}$ using κ extension-field multiplications. The prover need not construct $\boldsymbol{Y}$ or $\boldsymbol{W}$. These costs give Eq. (27). Under schoolbook arithmetic they are $O(\kappa N+\kappa^3)$ bb, which is $O(\kappa N)$ bb when $m \geq \kappa$.

The matrices $\boldsymbol{A}, \boldsymbol{\Xi}$ occupy $O(N)$ base-field coordinates, and $\boldsymbol{U}$ and the length-κ extension-field vectors occupy $O(\kappa^2)$. A constant number of length-m extension-field buffers also occupy $O(m\kappa) = O(N)$ base-field coordinates. Thus peak prover memory, excluding the underlying PCS workspace, is $O(N + \kappa^2)$, or $O(N)$ when $m \geq \kappa$.

Verifier complexity and proof size. The verifier implements Eq. (25) without forming the multiplication matrices. Right multiplication by $\boldsymbol{M}_{\overline{\alpha}}^\top$ scales each row, interpreted in the fixed basis as an extension-field element, by $\overline{\alpha}$. Left multiplication by $\boldsymbol{M}_{\overline{\alpha}}$ scales each column in the same way. Each scaling therefore costs κ extension-field multiplications. The two terms in each recurrence step require three such scalings, followed by κ^2 base-field additions. Over ℓ steps, the cost is at most $3\kappa\ell$ extension-field multiplications and $O(\kappa^2\ell)$ base-field operations, including coordinate rearrangement. A constant number of intermediate matrices use $O(\kappa^2)$ additional base-field coordinates.

The initial scalar check costs κ extension-field multiplications. To compute $\bar{c}$ and the expected terminal value $\overline{w}$, the verifier packs the rows of $\boldsymbol{U}$ and the columns of $\boldsymbol{W}$, respectively, and takes one length-κ extension-field inner product per target. Packing is coordinate reinterpretation, so these targets cost $O(\kappa)$ extension-field multiplications and $O(\kappa^2)$ coordinate operations in total. The inner-product verifier costs $O(1+\ell)$ extension-field operations, including its terminal scalar check. This gives Eq. (28).

The prover sends κ extension-field elements for $\overline{\boldsymbol{v}}$, followed by the $2\ell + 2$ extension-field elements of Fact 1, including both terminal values. The verifier computes $\boldsymbol{W}$ locally, so no matrix message or base-field proof is sent. Excluding the underlying opening, total prover-to-verifier communication is therefore $\kappa + 2\ell + 2$ extension-field elements. □

C.5 Proof of Lemma 2

Proof. The radix-$\underline{q}$ representations of $0, \ldots, L-1$ use at most $\lceil \log_{\underline{q}} L \rceil$ digits. The digit injection preserves their distinctness. Since $\theta_1, \ldots, \theta_d$ are linearly independent over $\mathbb{F}$, and the last coordinate is reserved for the witness value:

$$a_i = b_j \quad \Longleftrightarrow \quad \big(\sigma(i) = j \text{ and } w_i = w_j\big).$$

Every label occurs exactly once in each operand. Thus equality of the multisets forces $a_i = b_{\sigma(i)}$, and hence $w_i = w_{\sigma(i)}$, for all $i \in [L]$. Conversely, these copy equalities give $a_i = b_{\sigma(i)}$, proving the permutation relation. The encoding is deterministic and injective, so it adds no soundness error. Appending the same zeros to both operands preserves the equivalence by multiset cancellation. □

C.6 Proof of Theorem 1

Proof. **Completeness.** For a satisfying assignment, the five scalar claims in Eq. (12) and Eq. (13) sum to zero. Four packed Hadamard proofs reduce the selector–wire product claims to evaluations of the selectors and $\boldsymbol{w}_L, \boldsymbol{w}_R, \boldsymbol{w}_M, \boldsymbol{w}_O$. The fifth proves the remaining evaluation claim for $\boldsymbol{w}_M = \boldsymbol{w}_L \circ \boldsymbol{w}_R$. All terminal linear claims have arbitrary column weights and tensor row weights, as required by Section 4.4.

Lemma 2 proves the copy equivalence. The two operands have length $L = 3N + n_{\mathsf{io}} \le 4N$; append zeros to reach length $4N$. Their grand products then share the padding factor $\overline{z}^{4N-L}$, which is nonzero because $\overline{z} \xleftarrow{\$} \mathbb{E} \setminus \{0\}$. The two grand-product reductions therefore prove exactly the original permutation relation. Their packing width $\kappa_{\mathsf{pos}} = \kappa/d$ and row count $4N/\kappa_{\mathsf{pos}} = 4md$ are powers of two, with $4md \ge \kappa_{\mathsf{pos}} \ge 2$. Thus the parameters satisfy the interface of Section 4.3, and every honest check accepts.

Terminal copy claims. Each copy terminal is a weighted evaluation of a padded operand, with κ_{pos} arbitrary column weights and $4N/\kappa_{\mathsf{pos}}$ tensor row weights. By linearity, the two terminals involve $\boldsymbol{\varsigma}, \boldsymbol{\iota}$, and $\boldsymbol{w}$. Restricting to one length-N wire segment fixes the first two position bits. The last $\log_2 m$ bits remain tensor row factors. Absorb all other free factors into the column weights to obtain a length-κ column weight vector for the original $m \times \kappa$ wire or digit matrix. This changes the weight representation, not the committed data.

Proposition 5 computes the identity-label contribution in $O(\kappa_{\mathsf{pos}} + \log L)$ extension-field operations. The verifier evaluates the public-I/O values and their

permuted labels directly. Traversing their contiguous interval and reusing tensor prefixes costs $O(\kappa_{\mathsf{pos}} + n_{\mathsf{io}} + \log L)$ operations. No commitment to either contribution is needed.

Batched terminal openings. Write $d_L = \lceil \log_{\underline{q}} L \rceil$. The three fixed-basis wire packings form one witness vector of logical length $3m$, committed before any challenge. The public vector concatenates the five selector packings and the three wire-segment restrictions of each of the d_L permuted-position digit tables. Its logical length is $m_{\mathsf{pub}} = (5 + 3d_L)m$. The public commitment, circuit tables, and packing polynomials are preprocessed.

The gate argument leaves five selector and five wire claims. The two copy terminals add six wire claims and $3d_L$ permuted-digit claims, giving $16 + 3d_L$ linear claims. Apply the column-projection step of Section 4.4 to each claim and check its claimed scalar value. Fix all column messages before sampling independent projection vectors for the different claims. Within each group, zero-extend the projected weight vectors to their committed blocks and sum the resulting inner-product claims. One invocation of Fact 1 per group reduces their sum to one MLE opening of the concatenated vector. The verifier computes the MLE evaluation of the combined public weight vector exactly, using Eq. (25) in Appendix B.2 on each constituent weight vector and retaining its block-prefix weight. Thus there are two underlying PCS openings.

If a column message is false, its coordinate error satisfies $(\boldsymbol{U} - \boldsymbol{\Xi}^\top \boldsymbol{A})\overline{\boldsymbol{\beta}} \neq \boldsymbol{0}$. Consequently, a group containing a false claim has a nonzero linear error polynomial in its independent projection vectors. It vanishes with probability at most $1/|\mathbb{E}|$ per group. The two inner-product reductions add $O(\log N)/|\mathbb{E}|$ error. Here the padded group lengths have logarithm $O(\log N)$, since $d_L < d \leq \kappa/2$.

Soundness. First consider the reduction with exact terminal evaluations. Fix the length-$3m$ packed witness before any challenge. Decoding in the basis $\overline{\boldsymbol{\beta}}$ determines $\boldsymbol{w}_L, \boldsymbol{w}_R, \boldsymbol{w}_O$, and hence $\boldsymbol{w}_M = \boldsymbol{w}_L \circ \boldsymbol{w}_R$. The selectors and position tables are fixed at setup.

A nonzero gate residual vanishes at the random MLE point with probability at most $\log_2 N/|\mathbb{E}|$. Otherwise an accepting scalar sum contains a false claim. Acceptance then requires an error in one of the five Hadamard reductions or in the proofs of terminal claims. For the nested product claim, its value is fixed before the fresh challenges of the fifth reduction.

If a copy constraint fails, the unpadded fingerprint difference is nonzero and has degree at most $L-1$. At the nonzero challenge $\overline{z}$, its padded version vanishes if and only if the unpadded difference does. The fingerprint error is therefore at most $(L-1)/(|\mathbb{E}|-1)$. Let ϵ_{GP} be the bound of Proposition 3, instantiated with length $4N$, packing width κ_{pos}, and base field $\mathbb{K}_{\mathsf{pos}}$.

Knowledge soundness. By knowledge soundness of the PCS, its extractor recovers the committed witness polynomial. The three length-m wire packings in its evaluation table decode in $\overline{\boldsymbol{\beta}}$ to $\boldsymbol{w}_L, \boldsymbol{w}_R, \boldsymbol{w}_O \in \mathbb{F}^N$. The extractor checks their gate and copy constraints, including the public I/O, and returns the resulting assignment if these checks pass; otherwise it fails. Decoding and checking take polynomial time.

For every fixed invalid assignment, the preceding analysis bounds acceptance when terminal evaluations are exact. Lemma 3 compiles this reduction with the PCS proofs of the adaptively chosen terminal MLE claims, adding the joint knowledge-soundness error ϵ_{pc}. Thus:

$$\epsilon_{\mathsf{single}} \leq \epsilon_{\mathsf{pc}} + \frac{L-1}{|\mathbb{E}|-1} + 2\epsilon_{\mathsf{GP}} + 5\left(\frac{2\kappa-2}{|\mathbb{E}|-\kappa} + \frac{2\log_2 N}{|\mathbb{E}|}\right) + O\left(\frac{\log N}{|\mathbb{E}|}\right).$$

The last term includes the initial gate projection and the two batched opening reductions. The grand-product contribution is $O((\kappa_{\mathsf{pos}} \log_2 \kappa_{\mathsf{pos}} + (\log_2 N)^2)/|\mathbb{E}|)$. Since $N = m\kappa \geq \kappa^2$, $L \leq 4N$, and $(\log_2 N)^2 = O(N)$, this proves Eq. (14). Thus the extractor returns a satisfying assignment with probability at least the prover's acceptance probability minus $\epsilon_{\mathsf{single}}$, as required by Definition 12.

Prover time. The five Hadamard reductions cost $O(N\kappa)\mathsf{bb}$. For $q \geq 2\kappa - 1$, interpolation uses base-field nodes. Otherwise, the stated power-of-two choice of t satisfies $t \mid \kappa$, $t \leq \kappa$, and $q^t \geq 2\kappa - 1$. We use interpolation nodes in $\mathbb{F}_{q^t} \subseteq \mathbb{E}$ without changing κ. Proposition 2 gives the same online bound, counting unit-cost table accesses. The $16 + 3d_L$ column projections and the two grouped inner-product reductions cost $O(N\kappa d_L)\mathsf{bb}$.

Choose polynomial bases along $\mathbb{F} \subset \mathbb{K}_{\mathsf{pos}} \subset \mathbb{E}$ for the copy computation. A $\mathbb{K}_{\mathsf{pos}}$-multiplication costs $O(d^2)\mathsf{bb}$. The two grand-product reductions therefore cost:

$$O\left(N\kappa_{\mathsf{pos}} \log_2 \kappa_{\mathsf{pos}} + \kappa_{\mathsf{pos}}^4\right) d^2\mathsf{bb} = O\left(N\kappa d \log_2 \kappa_{\mathsf{pos}} + \frac{\kappa^4}{d^2}\right)\mathsf{bb}.$$

The $\log_2 \kappa_{\mathsf{pos}}$ factor counts the packed coefficient levels; evaluating their aggregated polynomials accounts for the additive term. The least-power-of-two choice gives $d_L + 1 \leq d < 2(d_L + 1)$. Since $\log \underline{q} = \Theta(\log q)$ and $L = \Theta(N)$, both d_L and d are $\Theta(\log_q N)$. Combining these contributions gives the stated $O(N\kappa \log_q N \log_2 \kappa)\mathsf{bb} + O(\kappa^4)$ bound. The copy computation stays in its tower basis; only interface weights and messages are converted to the fixed commitment basis. These conversions cost $O(\kappa^2)\mathsf{bb}$ per extension-field element and are covered by the same bounds.

Verifier time and proof size. The $O(d_L)$ packed column checks and MLE evaluations of the combined public weight vectors cost $O(\kappa d_L \log_2 N)\mathsf{ee}$. The two grand-product reductions add $O(\kappa_{\mathsf{pos}} \log_2 \kappa_{\mathsf{pos}} + (\log_2 N)^2)\mathsf{ee}$. Direct identity-label and public-I/O evaluations contribute $O(\kappa_{\mathsf{pos}} + n_{\mathsf{io}} + \log N)\mathsf{ee}$. These costs give the verifier bound.

The packed column messages contain $O(\kappa d_L)$ extension-field elements. The two grand-product transcripts contain $O(\kappa_{\mathsf{pos}} \log_2 \kappa_{\mathsf{pos}} + (\log_2 N)^2)$ such elements. The remaining Hadamard and batched inner-product transcripts are absorbed by these terms. Using $d_L = O(\log_q N)$ and $\kappa_{\mathsf{pos}} \leq \kappa$ gives the stated proof size.

Prover memory and PCS costs. The grand-product workspace uses $O(dN + d\kappa_{\mathsf{pos}}^3) = O(dN + \kappa^3/d^2)$ base-field coordinates. The witnesses, circuit tables,

and grouped opening buffers occupy $O(Nd_L)$ coordinates. Sequential execution reuses the gate and copy workspaces. The gate interpolation data and field tables use $O\big(\kappa^2 t + (t-1)q^{2t}\big)$ base-field coordinates. Since $t \le \kappa$, including these tables and public I/O gives $O(N \log_q N)\mathbb{F} + O\big(\kappa^4 + (t-1)q^{2t} + n_{\mathsf{io}}\big)$.

The four PCS functions bound aggregate resources for one online witness commitment and one opening of each of the two vectors, of logical lengths $3m$ and m_{pub}. They include domain padding and both retained commitment states; public commitment generation is preprocessed. Their total argument is therefore $3m + m_{\mathsf{pub}}$. This proves Eq. (15). Unlabeled time and memory remainders count base-field operations and base-field elements, respectively. □

C.7 Proof of Theorem 2

Proof. The parameters and sparsity assumptions are those of Section 5.3. The sparse MLE evaluation protocols of Spartan contribute aggregate soundness error $O(N/|\mathbb{E}|)$. Their PCS soundness errors are included in the joint ϵ_{pc} term specified below. The sprs terms include the total online costs of all three sparse-evaluation proofs and their PCS calls.

Completeness. Packing is a bijection between $\boldsymbol{W} \in \mathbb{F}^{N\times\kappa}$ and $\overline{\boldsymbol{w}} = \boldsymbol{W}\overline{\boldsymbol{\beta}}$. The commitment therefore fixes all κ assignments. For a satisfying batch, the packed public-I/O check holds, and:

$$(\boldsymbol{S}_L\boldsymbol{W}) \circ (\boldsymbol{S}_R\boldsymbol{W}) - \boldsymbol{S}_O\boldsymbol{W} = \boldsymbol{0}_{N\times\kappa}.$$

Every honest Hadamard, column-projection, and inner-product identity holds. The witness openings and preprocessed sparse-matrix evaluation proofs therefore accept due to their respective completeness.

Soundness. First consider exact terminal MLE evaluations of the witness and preprocessed circuit polynomials, with $\boldsymbol{W}$ fixed before any challenge. If these assignments violate a constraint, their residual matrix has a nonzero MLE in $\log_2 N + \log_2 \kappa$ variables. Its random row-and-column tensor projection vanishes with probability at most $\log_2(N\kappa)/|\mathbb{E}|$. Otherwise, either the claimed projection of $\boldsymbol{S}_O\boldsymbol{W}$ is false or the weighted Hadamard claim is false. Proposition 2, with total length $N\kappa$, bounds the latter reduction error by:

$$\frac{2\kappa - 2}{|\mathbb{E}| - \kappa} + \frac{2\log_2(N\kappa)}{|\mathbb{E}|}.$$

All three terminal claims concern the fixed virtual matrices $\boldsymbol{S}_\star\boldsymbol{W}$, not independent witnesses.

For each terminal linear claim, the column message determines $\boldsymbol{U}_\star \in \mathbb{F}^{\kappa\times\kappa}$. If its scalar check passes but the evaluation is false, then $\boldsymbol{U}_\star \neq \boldsymbol{\Xi}_\star^\top \boldsymbol{S}_\star\boldsymbol{W}$. Injectivity of basis packing gives:

$$\big(\boldsymbol{U}_\star - \boldsymbol{\Xi}_\star^\top \boldsymbol{S}_\star\boldsymbol{W}\big)\overline{\boldsymbol{\beta}} \neq \boldsymbol{0}.$$

All three errors are fixed before $\overline{z}$ and $\overline{\boldsymbol{r}}$ are sampled. Their $(1, \overline{z}, \overline{z}^2)$-weighted combination is a nonzero vector polynomial of degree at most two whenever one

error is nonzero. It vanishes with probability at most $2/|\mathbb{E}|$. Conditional on a nonzero combined error, projection by the independent uniform vector $\overline{\boldsymbol{r}}$ hides it with probability at most $1/|\mathbb{E}|$.

The first inner-product reduction adds at most $2\log_2 N/|\mathbb{E}|$ error. At its terminal point $\overline{\boldsymbol{y}}$, the verifier checks that the three MLE claims for the public weight vectors combine to its terminal value. If this value is false, at least one of those claims is false. The three further inner-product reductions add at most $6\log_2 N/|\mathbb{E}|$ error. Each ends in a sparse matrix MLE claim $\mathsf{ts}(\overline{\boldsymbol{y}'})^\top \boldsymbol{S}_\star \mathsf{ts}(\overline{\boldsymbol{y}})$ and an MLE evaluation of the public weight vector, $\mathsf{ts}(\overline{\boldsymbol{y}'})^\top \boldsymbol{\Xi}_\star \overline{\boldsymbol{r}}$. Spartan proves the former with respect to the preprocessed circuit commitment; Appendix B.2 evaluates the latter by tensor recursion.

The sparse-evaluation protocols fix their matrix metadata during preprocessing and their evaluation-dependent vectors before their challenges. Addresses and timestamps have fixed injective encodings in $\mathbb{E}$, so distinct integer labels are not reduced modulo the characteristic. Their memory-checking fingerprints have degree $O(N)$, and their product-tree checks contribute $O((\log N)^2)/|\mathbb{E}|$ error. Their aggregate algebraic error is therefore $O(N/|\mathbb{E}|)$ [58, Section 7.2].

Knowledge soundness. By knowledge soundness of the PCS, its extractor recovers the witness polynomial and hence its length-N evaluation vector $\overline{\boldsymbol{w}}$. Decoding each entry in $\overline{\boldsymbol{\beta}}$ gives the unique matrix $\boldsymbol{W} \in \mathbb{F}^{N\times\kappa}$ satisfying $\overline{\boldsymbol{w}} = \boldsymbol{W}\overline{\boldsymbol{\beta}}$. Its columns are the κ candidate assignments. The extractor checks their public I/O and $(\boldsymbol{S}_L\boldsymbol{W}) \circ (\boldsymbol{S}_R\boldsymbol{W}) = \boldsymbol{S}_O\boldsymbol{W}$, returning the batch if these checks pass and failing otherwise. This takes polynomial time; the three virtual matrices require no separate witness extraction.

The preceding analysis bounds acceptance for every fixed invalid batch when terminal evaluations are exact. Its witness-side terminals concern the same precommitted polynomial, and the sparse-evaluation proofs establish the circuit MLE claims. Lemma 3 adds the joint PCS knowledge-soundness error ϵ_{pc} for the adaptive witness and sparse-evaluation calls, giving:

$$\epsilon_{\mathsf{multi}} \le \epsilon_{\mathsf{pc}} + \frac{2\kappa-2}{|\mathbb{E}|-\kappa} + \frac{3\log_2(N\kappa)+11\log_2 N+3}{|\mathbb{E}|} + O\left(\frac{N}{|\mathbb{E}|}\right).$$

Since we assume $\kappa \le N$ and $|\mathbb{E}| = q^\kappa \ge 2\kappa$, this gives Eq. (16). The extractor therefore returns κ satisfying assignments with probability at least the prover's acceptance probability minus $\epsilon_{\mathsf{multi}}$, as in Definition 12.

Prover time and memory. The three virtual matrices $\boldsymbol{S}_\star\boldsymbol{W}$ require $O(N\kappa)$ base-field operations by sparsity. The Hadamard reduction on $N\kappa$ entries costs $O(N\kappa^2+\kappa^3)\mathsf{bb} = O(N\kappa^2)\mathsf{bb}$. This bound also holds for interpolation over $\mathbb{F}_{q^t}$, using the precomputed tables of Proposition 2 and counting unit-cost table accesses.

Forming each $\boldsymbol{U}_\star = \boldsymbol{\Xi}_\star^\top(\boldsymbol{S}_\star\boldsymbol{W})$ costs $O(N\kappa^2)\mathsf{bb}$. Computing $\boldsymbol{\Xi}_\star\overline{\boldsymbol{r}}$ takes $O(N\kappa)\mathsf{be} = O(N\kappa^2)\mathsf{bb}$, and multiplying it by $\boldsymbol{S}_\star^\top$ takes $O(N)$ mixed-field products. The four inner-product proofs for the linear claims and the public-I/O proof use $O(N)\mathsf{ee} = O(N\kappa^2)\mathsf{bb}$ operations in total. The second-stage weight vectors

have the same cost. Thus the algebraic prover time, excluding the witness PCS and sparse-evaluation proofs, is $O(N\kappa^2)\mathsf{bb}$.

The assignment and a constant number of virtual matrices use $O(N\kappa)$ base-field coordinates. The inner-product buffers use $O(N)$ extension-field elements, and the three column messages use $O(\kappa^2)$ base-field coordinates. The interpolation data and field tables add $O\big(\kappa^2 t+(t-1)q^{2t}\big)$ coordinates. Reusing the working buffers gives $O\big(N\kappa+\kappa^2 t+(t-1)q^{2t}\big)\mathbb{F}$ peak memory, including precomputed tables.

Verifier time and communication. The Hadamard and column checks cost $O(\kappa)\mathsf{ee}$. The three tensor-weight recursions cost $O(\kappa \log_2 N)\mathsf{ee}$, including their coordinate operations as in Proposition 4. The five inner-product verifications cost $O(\log N)\mathsf{ee}$, and computing the public-I/O targets and the terminal MLE evaluation of the public weight vector costs $O(n_{\mathsf{io}}+\log N)\mathsf{ee}$. This gives the stated verifier bound.

The Hadamard and column messages contain $O(\kappa)$ extension-field elements; the inner-product transcripts add $O(\log N)$. These fit the stated $O(\kappa+(\log_2 N)^2)\mathbb{E}$ bound.

Commitment costs and amortization. The witness PCS binds one packed vector of logical length N and proves the two retained witness MLE claims. The pcs terms aggregate these calls. The sprs terms aggregate the three sparse-matrix evaluation proofs, including their PCS calls. Circuit commitment generation is offline; retained state and online work are included in the respective memory and time terms. Adding these resources proves Eq. (17).

With linear-cost underlying PCS provers, the sparse-evaluation arithmetic and PCS calls in Spartan use $O(N)\mathsf{ee}$ time and $O(N)\mathbb{E}$ space for the three $O(N)$-sparse matrices [58, Section 7.2]. Thus:

$$\mathcal{TP}_{\mathsf{pcs}}(N)+\mathcal{TP}_{\mathsf{sprs}}=O(N)\mathsf{ee}=O(N\kappa^2)\mathsf{bb},$$
$$\mathcal{SP}_{\mathsf{pcs}}(N)+\mathcal{SP}_{\mathsf{sprs}}=O(N)\mathbb{E}=O(N\kappa)\mathbb{F}.$$

Dividing the total batch time and peak batch memory by κ gives $O(N\kappa)\mathsf{bb}$ time and $O\big(N+\kappa t+(t-1)q^{2t}/\kappa\big)\mathbb{F}$ memory per instance, including precomputed tables. The memory bound is amortized over the batch. □